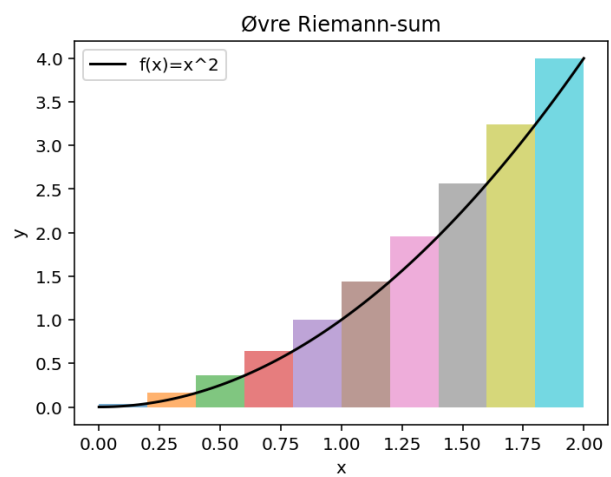
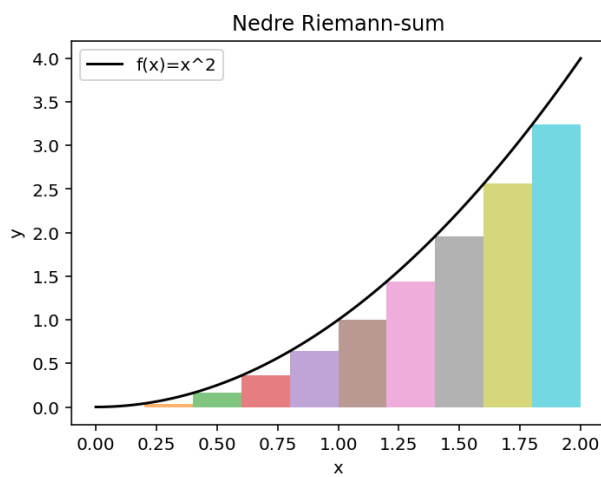


Programmeringsdelen til MAT110

Grunnleggende elementer i Python

Python-programmering i Matematikk 1 (MAT110)



Forord

Dette veiledningsheftet er en del av supplementet til læreboken Matematikk for ingeniørutdanning (1. utgave) [1], og er tilpasset for programmeringsdelen til MAT110.

Innhold

Kapittel 1 er utarbeidet for å dekke kravene til programmeringsdelen i Matematikk 1 (MAT110).

Kapittel 2 er utformet som en håndbok og kan benyttes som referanse dersom man ønsker en nærmere forklaring på hvorfor programmet fungerer slik det gjør.

Videre er det tatt med enkelte emner som går utover pensumkravene i MAT110. Disse er kun ment som **ekstramateriale** (*tilleggsmateriale for selvstudium og fordypning*).

Anbefaling til leseren

Når du lærer matematikk, er det ikke nok å bare pugge formler eller lære seg teknikker for å løse oppgaver. Det er like viktig å spørre seg selv hvorfor en bestemt metode fungerer, og hvordan de ulike begrepene og reglene henger sammen. Når du forstår grunnprinsippene bak en løsning, blir det lettere å se sammenhenger og tilpasse metodene til forskjellige problemer. Matematikk handler ikke bare om å kunne utføre beregninger, men om å forstå «hvorfor» bak løsningene. Jo bedre du forstår de underliggende ideene i et emne, desto lettere vil det være å bruke dem på mer komplekse problemer — og til å anvende matematikk i andre fagfelt, som fysikk, økonomi eller ingeniørfag. Denne forståelsen gir deg et sterkere grunnlag for videre læring og problemløsning.

Takk

En stor takk rettes også til Jonas Gahr Sturtzel Lunde og Kristine Baluka ved UiO, og Jarle Peder Berntsen, Anita Gjesteland ved UiB og Erik Andreas Hanson ved HVL, som har gitt oss tilgang til deres notater, modellerings og programmeringsoppgaver i oppgavesamlingene deres ved UiO og UiB. Jeg vil spesielt sende en stor takk til Erik Andreas Hanson, Marthina Sætre Vahedian, Lorentz T. Syslak, og alle ENT3R-mentorene i Bergen for korrekturlesing av teksten og innspill. Sist og ikke minst vil vi gjerne takke mine kollegaer ved HVL, Trygve Buenes, Harald Spångberg, Erlend Storvik og Ole Bendiksen for å lese gjennom dette heftet, og ha hjulpet å forbedre både teksten og innholdet.

Bergen, 6. august 2025

Amir M. Hashemi

Eventuelle kommentarer eller meldinger om feil i notatene tas imot med glede: ahas@hvl.no.

© Forfatteren 2022–2030.

Innhold

0.1	Hva forventes fra studentene fra programmeringdelen (MAT110)?	3
1	MAT110- Programmering med Python	5
1.1	Komplekse tall	5
1.2	Lineær algebra (matriseregning og rekkereduksjon)	8
1.3	Numerisk derivasjon	15
1.4	Numerisk derivasjon og endelige differanser (ekstramateriale)	19
1.5	Nedstigningsmetoden (gradient descent) for å finne et minimum] (ekstramateriale) . .	22
1.6	Newtons metode	24
1.7	Halveringsmetoden (ekstramateriale)	25
1.8	Numerisk integrasjon	28
1.9	Trapesmetoden	30
1.10	Simpsons metode (ekstramateriale)	31
2	Litt programmering med Python	37
2.1	Å komme i gang med Python	37
2.2	Noen viktige elementer i Python-programmering	38
2.3	Noen av Pythons innebygde funksjoner	39
2.4	Grunnleggende Python-programmering	40
2.5	Datatyper	44
2.6	Variabler	45
2.7	Å sende inn datasett i Python-program	46
2.8	Importering av pakker (numpy, math, pylab, sympy)	48
2.9	Indeksering	50
2.10	Løkker (betingelser og gjentakelser)	51
2.11	Funksjoner	56
2.12	Plotting i Python	63
2.13	Lister	67
2.14	Python-filhåndtering	69
2.15	Laste inn, håndtere og plotte datasett	70
2.16	Modellering: tekopp-temperatur	70
2.17	Oppsummering	73
Fasit		76

Programmering i matematikk 1 (MAT110)

Hva forventes å kunne for programmeringsdelen i MAT110?

Hvor mye skal studentene kunne For programmering i matematikk 1?

Vi leser i emnebeskrivelsen til matematikk 1 (MAT110) at kandidaten etter å ha fullført dette emnet, har kjennskap til matematisk programvare, og kandidatane kan bruke sin matematiske kunnskap, om nødvendig med bruk av matematisk programvare.

Hvor mye må man kunne?

Det forventes ikke at studentene skal kunne programmere (**se oppgave 0.1.1-0.1.3**) men skal kunne:

- *forklare hva programmet utfører generelt, og detaljene i programmet,*
- *forklare hvilket problem programmet løser, og hvilken metode brukes i programmet*
- *forklare hva kan endres om mulig i programmet for å øke presisjonen i programmet, og*
- *tilpasse programmet til en lignende problemstilling (modifisere).*

Derfor legges ikke noe i formelarket som tildeles med eksamensoppgavene noe med programmering.

Hva?

Vi skal bruke Python i matematikk 1, og i følgende temaer kan programmering være aktuelt:

- *å skissere en kurven på formen $y = f(x)$*
- *å løse lineære ligningssystemer ved hjelp av den rekke reduserte matrisen (matriseregning)*
- *å gjennomføre litt matrise algebra, og bestemme invertermatrisen eller den rekke reduserte matrisen for å løse et lineært ligningssystem,*
- *å bestemme funksjonsderiverte i et punkt (stigningstall)*
- *å løse ligninger ved å bruke Newtons metode, en numerisk metode til å regne en tilnærmet løsning til: $f(x) = 0$ som er nært et startpunkt $x = x_0$*
- *å bestemme integral ved hjelp av arealregning (rektangelmetoden og Trapez metoden)*

Oversikt

Her er oversikt over innhold i dette heftet:

Kapittel 1: Programmering i matematikk 1

Kapittel 2: Grunnleggende elementer i Python programmering

Så hvis du vil få litt mer ut fra programmeringsdelen i matemaikk 1, kan du gjerne lese kapittel 2 også. Før du leser kapittel 2, kan du lese neste side om PRIMM-metoden, men det ikke er forventet å kunne skrive program i dette faget.

PRIMM-metoden for å tilrettelegge tekstprogrammering

Her har vi benyttet PRIMM-metoden for å tilrettelegge for læring av tekstprogrammering. PRIMM står for Predict-Run-Investigate-Modify-Make. PRIMM ble introdusert av Sue Sentance i 2017 og trekker sammen kjente effektive undervisningspraksiser i ett strukturert rammeverk.

- **Predict-fasen:** Forklar hva programmet utfører og hvilket problem programmet prøver å løse.
- **Run-fasen:** Kjør programmet for å lære hva programmet utfører, og eventuelt debugge (finne feilen) i programmet.
- **Inverstigate-fasen:** som bør varieres – gode øvelser som fremmer detaljene i programmering, og man lærer om syntaksene og kommandoene steg for steg. Oppgaver der det er lagt inn feil i programmet og man skal finne feilen (feilsøking/debugging) i programmet.
- **Modify-fasen:** Man skal lære hva programmet utfører, og lære seg å gjøre nødvendige endringer for å tilpasse programmet til å løse et nytt problem. Man kan ha oppgaver, der man skal endre litt, og kan ha oppgaver der man skal endre flere deler for å få tilpasset til en ny problemstilling.
- **Make-fasen:** Når man har gått gjennom de første fire trinnene, kan det gis oppgaver der man kan låne kodebiter fra det originale programmet, men det vil ha en ny funksjon, kontekst eller problem som skal løses. Disse fem fasene er kanskje ikke i hver leksjon, avhengig av emnet. Og du kan sykle gjennom de tre, fire første fasene eller alle fem fasene flere ganger.

Å komme i gang med Python

Det er to hovedversjoner av Python, Python 2 og Python 3. Disse versjonene er ganske forskjellige. Her bruker vi Python 3, fordi den er mer semantisk korrekt og støtter nyere funksjoner.

Python er et tekstbasert programmeringsspråk, og for å komme igang med å programmere, så har man flere alternativer blant annet:

- Programmere på nett: Det er flere tjenester på nettet som lar deg skrive og kjøre python-programmer, som for eksempel:

<https://trinket.io/python3>

<https://replit.com/new/python3>

- Installere en kildekode-plattform som både er en kode-editor, men som har innebygget en egen «Python-motor».

<https://www.spyder-ide.org/>

<https://jupyter.org/>

Eksempel på et slikt program er Spyder. Dette er en åpen kildekode-plattform integrert i utviklingsmiljø for vitenskapelig programmering med Python. Spyder integreres med en rekke fremtredende pakker i den vitenskapelige Python-pakken, inkludert numpy, scipy, Matplotlib, pandas, IPython, sympy og Cython, samt annen programvare med åpen kildekode. Det finnes også andre slike program:

<https://www.anaconda.com/>

<https://codewith.mu/en/download>

Hvis du vil lære først mer om programmering, kan du gjerne lese først kapittel 2.

I MAT110 skal vi ikke kunne programmere, men:

- forklare hva programmet utfører,
- hvilket problem programmet løser, og hvilken matematisk metode brukes i programmet
- kunne endre noe i programmet for at den gir bedre nøyaktighet
- kunne endre det trengs for at programmet kan løse en lignende problem (modifisere)

0.1 Hva forventes fra studentene fra programmeringdelen (MAT110)?

Oppg. 0.1.1: En programkode er gitt som:

```
1 import numpy as np
2
3 def f(x):
4     return np.sin(np.pi/2 * x)
5
6 def derivert(a, deltax):
7     return (f(a + deltax) - f(a)) / deltax
8
9 a = 0.5
10 deltax = 0.0001
11
12 df = derivert(a, deltax)
13 print(f"Den deriverte av f i x={a} er tilnærmet: {df}")
```

- a) Hva prøver programkoden å løse? Forklar kort (linje for linje) programmet
- b) Bestem den eksakte verdien og kommenter svaret fra del a).
- c) Hva kan endres i programmet for å få bedre presisjon?
- d) Hva skal endres i programmet for å beregne $g'(\pi/3)$ gitt $g(x) = \cos(x)$?

Oppg. 0.1.2:

- a) Vis at ligningen $x^3 - x - 1 = 0$ har nøyaktig én løsning i intervallet $[1, 2]$. Bruk startverdien $x_0 = 1$ for Newtons metode.
- b) Hva prøver følgende program å bestemme?

```
1 import math
2
3 def f(x):
4     return x**3 - x - 1
5
6 def fderiv(x):
7     return 3*x**2 - 1
8
9 x = 1.0
10 tol = 1e-8
11 max = 100
12
13 for i in range(max):
14     xny = x - f(x)/fderiv(x)
15     if abs(xny - x) < tol:
16         break
17     x = xny
18
19 print("Rot funnet:", x)
20
```

- c) Hvordan kan man forbedre presisjonen i Newtons metode?
- d) Hva må endres i programmet for å løse $\cos(x) - x + 1 = 0$ i $[0, \pi/2]$?

Oppg. 0.1.3: En kode er gitt som nedenfor:

```
1 import numpy as np
2
3 def f(x):
4     return np.sin(np.pi/2 * x)
5
6 n = 4
7 a = 0
8 b = 1
9 dx = (b - a)/n
10
11 integral = 0
12 for i in range(n):
13     xi = a + i*dx
14     integral += f(xi)*dx
15
16 print(f"Tilnærmet verdi av integralet: {integral}")
```

- a) Hva prøver programmet å løse? Hvilken metode programmet?
- b) Bestem den eksakte verdien.
- c) Hvordan kan presisjonen økes?
- d) Hvordan kan vi endre i programmet for å beregne

$$\int_0^2 \cos\left(\frac{\pi}{4}x\right) dx$$

med å dele integrasjonsintervallet i 8 partisjoner?

Kapittel 1

MAT110- Programmering med Python

1.1 Komplekse tall

Komplekse tall skrives som $z = a + bi = re^{i\theta}$, der $i^2 = -1$.

- Den reelle delen er $\operatorname{Re}(z) = a$ og den imaginære delen er $\operatorname{Im}(z) = b$
- Modulus til z : $r = |z| = \sqrt{a^2 + b^2}$
- Konjugat: $\bar{z} = a - bi$

Python-kommandoer for komplekse tall

Kommando	Beskrivelse	Eksempel
<code>import sympy as sp</code>	importerer pakken sympy	<code>i =sp.I</code>
<code>sp.I</code>	Den imaginære enheten i	<code>z = 3 + 4*sp.I</code>
<code>sp.re(z)</code>	Reell del av z	<code>sp.re(z)</code>
<code>sp.im(z)</code>	Imaginær del av z	<code>sp.im(z)</code>
<code>sp.Abs(z)</code>	Modulus $ z $	<code>sp.Abs(z)</code>
<code>sp.conjugate(z)</code>	Konjugat $\bar{z}$	<code>sp.conjugate(z)</code>
<code>sp.arg(z)</code>	Argument θ	<code>sp.arg(z)</code>
<code>z1 + z2</code>	Sum av komplekse tall	<code>(2+3*sp.I)+(1-4*sp.I)</code>
<code>z1 * z2</code>	Produkt av komplekse tall	<code>(2+3*sp.I)*(1-4*sp.I)</code>
<code>sp.simplify(z1/z2)</code>	Forenkle kvotient	<code>sp.simplify((1+2*sp.I)/(3-4*sp.I))</code>

Tabell 1.1: Viktige Python-kommandoer i sympy for komplekse tall

Eksempler - komplekse tall

Eksempel 1.1.1: Forklar kort hva programmet beregner, og beskriv hvilket resultat man får ved å kjøre det.

```
1 import sympy as sp
2 z = 1 + sp.I
3
4 print(sp.Abs(z), sp.arg(z))
```

Programmet bestemmer r og θ : $r = \sqrt{2}$, $\theta = \pi/4$

Eksempel 1.1.2: Forklar kort hva programmet utfører, og hva svaret blir:

```
1 import sympy as sp
2 i = sp.I
3 z1 = 2+3*i
4 z2 = 1-4*i
5 zsum = z1+z2
6 print(zsum)
```

Legger sammen to komplekse tall: $3 - i$

Eksempel 1.1.3: Forklar kort hva programmet beregner, og beskriv hvilket resultat man får ved å kjøre det.

```
1 import sympy as sp
2 z = 5-2*sp.I
3 re, im = sp.re(z), sp.im(z)
4 print(re, im)
```

Den bestemmer og skriver ut den reelle og den imaginære delen til et komplekstall.

$\text{Re}(z) = 5$, $\text{Im}(z) = -2$

◇

Eksempel 1.1.4: Forklar kort hva programmet beregner, og beskriv hvilket resultat man får ved å kjøre det.

```
1 import sympy as sp
2 z1 = 1+2*sp.I
3 z2 = 2+sp.I
4 zprod = z1*z2
5 modz1 = abs(z1)
6 print(zprod, modz1)
```

P Programmet bestemmer og skriver ut modulus(r) og argumentet (θ) til produktet $z_1 z_2$:

$0 + 5i$, $\sqrt{5}$

Eksempel 1.1.5: Hva utfører følgende program?

```
1 import sympy as sp
2 z = 3+4*sp.I
3 zkconj = sp.conjugate(z)
4 print(zkconj)
```

Programmet bestemmer konjugerte til z : $3 - 4i$

Eksempel 1.1.6: Forklar kort hva programmet beregner, og beskriv hvilket resultat man får ved å kjøre det.

```
1 import sympy as sp
2 z1 = 1 + 2*sp.I
3 z2 = 3 - 4*sp.I
4
5 ratio = sp.simplify(z1/z2)
6 print(ratio, sp.N(ratio))
```

Programmet bestemmer z_1/z_2 og forenkler (skriver svaret på standard form).
svar $-0.2 + 0.4i$ (numerisk), som tilsvarer $-1 + 2i$

Eksempel 1.1.7: Hva får man med å kjøre programmet?

```
1 import sympy as sp
2 z = 3 + 4*sp.I
3
4 print(sp.re(z), sp.im(z), sp.Abs(z), sp.conjugate(z))
```

Programmet bestemmer den reelle, imaginære delen til z , modulus (r) og den konjugerte til z ($\bar{z}$):
3, 4, 5, $3 - 4i$

Oppgaver - komplekse tall (bruk programmering)

Oppg. 1.1.1: Legg sammen $z_1 = 1 + 2i$ og $z_2 = 3 - 4i$.

Oppg. 1.1.2: Finn produktet av $2 + i$ og $1 - 3i$.

Oppg. 1.1.3: Finn absoluttverdien av $3 + 4i$.

Oppg. 1.1.4: Finn konjugatet til $2 - 5i$.

Oppg. 1.1.5: Finn real- og imaginærdelen til $7 + 9i$.

Oppg. 1.1.6: Beregn $(1 + 2i)^2$.

Oppg. 1.1.7: Kjør koden og sammenlign med analytisk regning.

- Finn Re , Im , $|z|$ for $z = 5 - 12i$.
- Regn $(2 + 3i) + (4 - 5i)$ og $(2 + 3i)(4 - 5i)$.
- Divider $(3 + 4i)/(1 - 2i)$.
- Finn $\arg(-1 + i)$.
- Skriv $1 + i$ i polar form.
- Finn konjugatet til $7 - 3i$.

1.2 Lineær algebra (matriseregning og rekkereduksjon)

Python-funksjoner til lineær algebra

Funksjon	Beskrivelse	Eksempel
<code>import sympy as sp</code>	importerer pakken sympy	<code>A = sp.Matrix([[1,2],[3,4]])</code>
<code>sp.Matrix(...)</code>	Lager en matrise	<code>A = sp.Matrix([[1,2],[3,4]])</code>
<code>A.row_join(b)</code>	Slår sammen A og b	<code>A.row_join(b)</code>
<code>A.rref()</code>	Redusert trappeform (RREF)	<code>A.rref()</code>
<code>A.det()</code>	Determinant	<code>A.det()</code>
<code>A.inv()</code>	Invers av A	<code>A.inv()</code>
<code>A.rank()</code>	Rangen til A	<code>A.rank()</code>
<code>A.nullspace()</code>	Nullrom til A	<code>A.nullspace()</code>
<code>A.eigenvals()</code>	Eigenverdier	<code>A.eigenvals()</code>
<code>A.eigenvects()</code>	Eigenvektorer	<code>A.eigenvects()</code>
<code>A.LUdecomposition()</code>	LU-faktorisering	<code>A.LUdecomposition()</code>

Tabell 1.2: Viktige Python-funksjoner i sympy for lineær algebra

RREF-funksjonen i Python

Kommandoen `rref()` står for *Reduced Row Echelon Form* (reduert trappeform). Den er tilgjengelig i sympy-pakken, som er et symbolsk matematikkbibliotek for Python. Ved å bruke `rref()` kan man enkelt finne redusert trappeform av en matrise, og den returnerer:

- Selve matrisen i redusert trappeform
- En tuple med posisjonene til pivot-kolonnene

Man importerer alltid først:

```
1 import sympy as sp
```

Eksempel 1.2.1: Finn RREF for en 2×2 matrise:

```
1 import sympy as sp
2
3 A = sp.Matrix([[1, 2],
4               [3, 4]])
5 Arref = A.rref()
6 print(Arref)
```

Dette returnerer matrisen til redusert trappeform: $\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$ og det andre elementet er pivot-kolonnene: (0,1). $\diamond$

Eksempel 1.2.2: Løs et system $Ax = b$ ved å finne RREF av den utvidede matrisen:

```
1 import sympy as sp
2
3 A = sp.Matrix([[1, 1],
4               [2, 3]])
5 b = sp.Matrix([4, 7])
6
7 Ab = A.row_join(b)    # generer [A|b]
8 Abrref = Ab.rref()
9 print(Abrref)
```

Utdata viser redusert trappeform av $[A|b]$, som kan brukes til å lese av løsningen til systemet direkte. $\diamond$

Eksempler - lineær algebra

Eksempel 1.2.3: Vi ser på en homogen ligningssystem $\mathbf{Ax} = 0$ der $\mathbf{A}$ er en 3×3 -matrise med ukjent k :

$$A = \begin{bmatrix} 1 & 2 & -1 \\ 2 & 4 & k \\ 3 & 6 & k-1 \end{bmatrix}.$$

Vi vil drøfte eksistens av løsningen (én entydig-løsning, ingen eller uendelig mange løsninger) ved å bruke programmering. Vi setter opp det homogene systemet $Ax = 0$. Følgende program utfører rekkereduksjon for ligningssystemet:

```
1 import sympy as sp
2
3 k = sp.symbols('k')
4 A = sp.Matrix([[1, 2, -1],
5               [2, 4, k],
6               [3, 6, k-1]])
7 b = sp.Matrix([0, 0, 0])
8
9 Ab = A.row_join(b)
10 rrefAb, pivots = Ab.rref()
11 print("RREF([A|b]) =")
12 print(rrefAb)
13 print("Pivots:", pivots)
```

Ved å kjøre programmet, får vi:

```
1 RREF([A|b]) =
2 Matrix([[1, 2, 0, 0], [0, 0, 1, 0], [0, 0, 0, 0]])
3 Pivots: (0, 2)
```

Rekkereduksjon:

$$\left[\begin{array}{ccc|c} 1 & 2 & -1 & 0 \\ 2 & 4 & k & 0 \\ 3 & 6 & k-1 & 0 \end{array} \right] \sim \left[\begin{array}{ccc|c} 1 & 2 & -1 & 0 \\ 0 & 0 & k+2 & 0 \\ 0 & 0 & k+2 & 0 \end{array} \right].$$

- Hvis $k \neq -2$, får vi en pivot i tredje kolonne, og bare den trivielle løsningen $x = 0$.
- Hvis $k = -2$, kollapser siste rad og vi får

$$\left[\begin{array}{ccc|c} 1 & 2 & -1 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{array} \right],$$

slik at $x_1 + 2x_2 - x_3 = 0 \Rightarrow x_1 = -2x_2 + x_3$. Da har vi *to frie parametre*.

Dermed er løsningsrommet for $k = -2$ gitt ved

$$x = \begin{bmatrix} -2s+t \\ s \\ t \end{bmatrix} = s \begin{bmatrix} -2 \\ 1 \\ 0 \end{bmatrix} + t \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix}, \quad s, t \in \mathbb{R}.$$

◇

Oppgaver - lineær algebra

Oppg. 1.2.1: Gitt følgende program:

```
1 import sympy as sp
2
3 A = sp.Matrix([[2, 1, 3],
4               [1, 0, 2],
5               [3, 2, 1]])
6 b = sp.Matrix([5, 3, 6])
7
8
9 AT = A.T
10 print("A^T =")
11 print(AT)
12
13
14 detA = A.det()
15 print("det(A) =", detA)
16
17 if detA != 0: # betyr hvis det(A) ikke er null.
18     Ainv = A.inv()
19     print("A^{-1} =")
20     print(Ainv)
21 v
22 Abrrref = A.row_join(b).rref()
23 print("RREF av [A|b] =")
24 print(Abrrref)
```

- a) Hva utfører programmet? Hva blir det skrevet ut for `print(A^T)`?
- b) Kan vi forvente en entydig løsning for $Ax = b$?
- c) For `print("A^{-1} =")` blir skrevet ut:

```
1 A^{-1} =
2 Matrix([[ -4/3, 5/3, 2/3], [5/3, -7/3, -1/3], [2/3, -1/3, -1/3]])
```

Bruk dette til å løse ligningssystemet.

- d) Hva får man når man kjører programmet?

Oppg. 1.2.2: Gitt følgende program:

```
1 import sympy as sp
2 A = sp.Matrix([[1,2,1],
3               [2,1,1],
4               [1,1,2]])
5 b = sp.Matrix([0,2,2])
6
7 Ainv = A.inv()
8 print("A^{-1} =")
9 print(Ainv)
10
11 Arref = A.row_join(b).rref()
12 print("RREF av [A|b] =")
13 print(Arref)
```

- a) Hva utfører programmet?
- b) Ved `print("A^{-1} =")` får man

```
1 A^{-1} =
2 Matrix([[ -1/4, 3/4, -1/4], [3/4, -1/4, -1/4], [-1/4, -1/4, 3/4]])
```

Hva får man med å kjøre programmet?

Oppg. 1.2.3: Gitt følgende program:

```
1 import sympy as sp
2
3 A = sp.Matrix([[2,1,3],
4               [1,2,1],
5               [3,1,2]])
6 b = sp.Matrix([10,6,13])
7
8 detA = A.det()
9 print("Det(A) =", detA)
10
11 Arref = A.row_join(b).rref()
12 print("RREF av [A|b] =")
13 print(Arref)
```

- a) Hva utfører programmet?
- b) Kan man forvente å få en entydig løsning? Hva viser programmet når man kjører det?

Oppg. 1.2.4:

- a) Hva utfører følgende program?

```
1 import sympy as sp
2
3 A = sp.Matrix([[1,2,1],
4               [3,1,2],
5               [2,3,1]])
6 b = sp.Matrix([4,10,8])
7
8 Ainv = A.inv()
9 print("A^{-1} =")
10 print(Ainv)
11
12 Arref = A.row_join(b).rref()
13 print("RREF av [A|b] =")
14 print(Arref)
```

- b) Hva får man med å løse systemet via programmet (RREF)?

Oppg. 1.2.5: Gitt programmet

```
1 import sympy as sp
2
3 A = sp.Matrix([[3, 1, -1],
4               [2, 4, 1],
5               [1, -1, 2]])
6 b = sp.Matrix([4, 1, 3])
7
8 Ainv = A.inv()
9 print("A-1 =")
10 print(Ainv)
11
12 Ab = A.row_join(b)
13 R, pivots = Ab.rref()
14 print("RREF av [A|b] =")
15 print(R)
16 print("Pivot-kolonner:", pivots)
17
18 x = Ainv * b
19 print("Losning x =")
20 print(x)
```

- a) Hva utfører programmet? Hva får man med å kjøre programmet?
- b) Forklar linje for linje programmet.

Oppg. 1.2.6: Gitt programmet

```
1 import sympy as sp
2
3 A = sp.Matrix([[1, 1, 0],
4               [0, 1, 1],
5               [0, 0, 1]])
6 b = sp.Matrix([2, -1, 3])
7
8 print("det(A) =", A.det())
9
10 Ainv = A.inv()
11 print("A-1 =")
12 print(Ainv)
13
14 Ab = A.row_join(b)
15 R, pivots = Ab.rref()
16 print("RREF av [A|b] =")
17 print(R)
18 print("Pivot-kolonner:", pivots)
19
20 x = Ainv * b
21 print("Losning x =")
22 print(x)
```

- a) Hva utfører programmet? Hva får man ved å kjøre programmet?
- b) Forklar linje for linje programmet.

Oppg. 1.2.7: Gitt programmet

```
1 import sympy as sp
2
3 A = sp.Matrix([[2, -1, 0],
4               [1, 2, 1],
5               [0, -1, 1]])
6 b = sp.Matrix([3, 0, 5])
7
8 print("det(A) =", A.det())
9
10 Ainv = A.inv()
11 print("A^{-1} =")
12 print(Ainv)
13
14 Ab = A.row_join(b)
15 R, pivots = Ab.rref()
16 print("RREF av [A|b] =")
17 print(R)
18 print("Pivot-kolonner:", pivots)
19
20 x = Ainv * b
21 print("Losning x =")
22 print(x)
```

- a) Hva utfører programmet? Hva får man ved å kjøre programmet?
- b) Forklar linje for linje programmet.

Oppg. 1.2.8: Vi ser på et system der parameteren k inngår i både koeffisientmatrisen og høyresiden:

$$\begin{bmatrix} 1 & k & 1 \\ 2 & 2k & 2 \\ 1 & k & 2 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \end{bmatrix} = \begin{bmatrix} 1 \\ 2 \\ k \end{bmatrix}.$$

Vi undersøker løsningen med SymPy:

```
1 import sympy as sp
2
3 k = sp.symbols('k')
4 A = sp.Matrix([[1, k, 1],
5                [2, 2*k, 2],
6                [1, k, 2]])
7 b = sp.Matrix([1, 2, k])
8
9 Ab = A.row_join(b)
10 rrefAb, pivots = Ab.rref()
11 print("RREF([A|b]) =")
12 print(rrefAb)
13 print("Pivots:", pivots)
```

- Hva gjør programmet?
- Drøft eksistens av løsningen (avhengig av k), det vil si for hvilke verdier av k har ligningssystemet eventuelt én en-entydig løsning, ingen eller uendelig mange løsninger.
- Finn løsningen eksplisitt for $k = 2$.

Oppg. 1.2.9: I et land er det tre sektorer som bidrar til den nasjonale inntekten: Sektor 1 (S_1), Sektor 2 (S_2), og Sektor 3 (S_3). De økonomiske bidragene fra hver sektor er representert ved ukjente variabler I_1 , I_2 , og I_3 , som står for inntektene fra hver sektor i millioner kroner. Tabellen nedenfor viser hvordan hver sektor bidrar i forskjellige deler av økonomien (i millioner kroner):

Region	S_1	S_2	S_3
R_1	12	5	3
R_2	10	7	4
R_3	15	6	5

De totale inntektene i hver region er gitt som:

Region	Totale inntekter (mill. kr)
R_1	31
R_2	36
R_3	42

Anta at inntektene fra sektorene I_1 , I_2 , og I_3 i hver region gir de totale inntektene for hver region. Vi antar at inntektene følger en lineær sammenheng i henhold til tabellen.

- Bruk de totale inntektene for hver region til å sette opp et system med tre lineære ligninger for de ukjente inntektene I_1 , I_2 , og I_3 .
- Sett opp koeffisientmatrisen og regn ut determinanten. Har ligningssystemet en entydig løsning?
- Bruk den inverse matrisen for å finne inntektene I_1 , I_2 , og I_3 .

1.3 Numerisk derivasjon

Ved å finne den deriverte til en funksjon i et punkt $(x, y) = (x_0, f(x_0))$ på kurven til f finner man stigningstallet akkurat der (den oransje tangentlinjen på figuren), som også kan kalles *vekstraten* for dette punktet eller momentan hastighet.

Stigningstallet mellom to punkt $x = x_0$ og $x = x_0 + h$ kan uttrykkes ved:

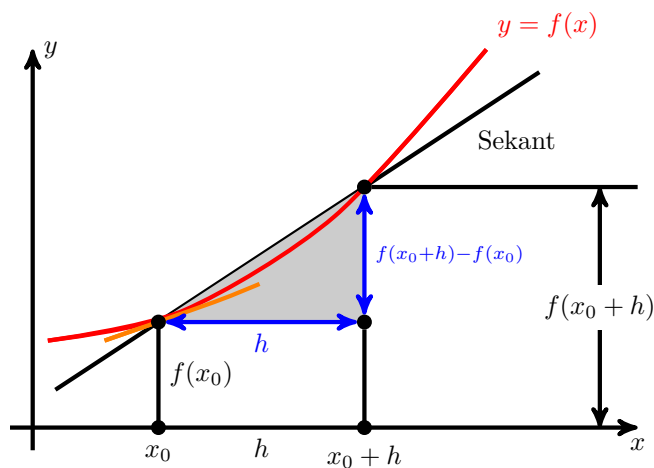
$$\frac{f(x_0 + h) - f(x_0)}{h} \quad (1.1)$$

Hvis $|h|$ blir mindre og mindre, så nærmer sekanten seg mer og mer tangenten, fordi punktet $(x+h, f(x+h))$ kommer nærmere og nærmere $(x, f(x))$:

$$f'(x_0) \simeq \frac{f(x_0 + h) - f(x_0)}{h}$$

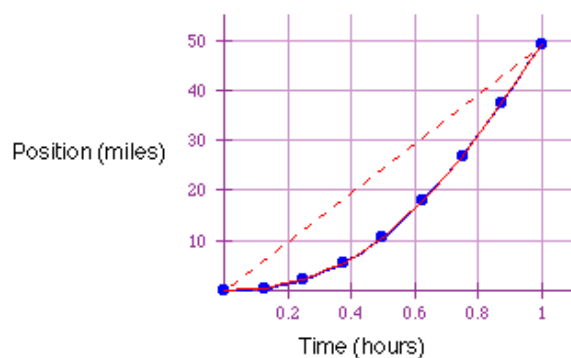
Derivasjon kan fortelle oss hvor raskt en størrelse er i ferd med å endre seg ved et bestemt punkt:

$$\left. \frac{df}{dx} \right|_{x=x_0} = f'(x_0) = \lim_{h \rightarrow 0} \frac{f(x_0 + h) - f(x_0)}{h}$$



Oppgaver - derivasjon

Oppg. 1.3.1: Figuren viser posisjonen til en bil i $0 < t \leq 1$ timer. Det kans leses av grafen:



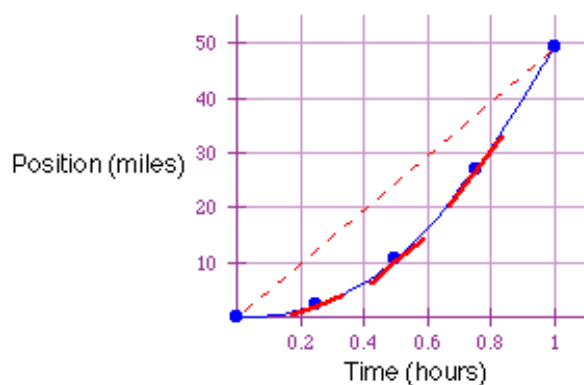
Tid, t (hours)	Posisjon, x (miles)
0.00	0.0
0.25	2.3
0.50	10.6
0.75	26.8
1.00	49.2

a) Bestem gjennomsnittlig fart i intervallet $0.25 \leq t \leq 0.5$.

b) Anta at vi leser av grafen:

Tid, t	Posisjon, x
0.25	2,300
0.26	2,512

Beregn momentan hastighet i $t = 0.25$.



Tid, t (hours)	Momentan hastighet (mph)
0.25	21.2
0.50	49.0
0.75	77.2

c) Forklar med dine ord sammenhengen mellom grafen og de gitte verdiene.

Oppg. 1.3.2:

a) Forklar hva følgende program utfører:

```
1 def f(x):
2     return x**3 + 3*x + 2
3
4 def derivert(a, deltax):
5     return (f(a + deltax) - f(a)) / deltax
6
7 a = 2
8 deltax = 0.0001
9
10 df=derivert(a, deltax)
11 print("Den deriverte av f i punktet", a, "er tilnærmet:", df)
```

b) Forklar kort linje for linje hva programmet utfører.

c) Hva skal endres i programmet for å få bedre presisjon?

d) Hvilke endringer skal man ha i programmet for å beregne $g'(-1)$ gitt $g(x) = 2 * x^5 - 5x + 3$?

Oppg. 1.3.3:

a) Forklar hva følgende program utfører:

```
1 def f(x):
2     return 3*x**4 - x + 1
3
4 def derivert(a, deltax):
5     return (f(a + deltax) - f(a)) / deltax
6
7 a = -2
8 deltax = 0.01
9 df = derivert(a, deltax)
10 print("Den deriverte i punktet", a, "er tilnærmet:", df)
```

b) Forklar kort linje for linje hva programmet utfører.

c) Hva skal endres for å få bedre presisjon?

d) Hvilke endringer skal gjøres for å beregne $m'(1)$ for $m(x) = e^x$?

Eksempel 1.3.1: På en eksamen i 1T får man spørsmål om hva følgende program utfører

```
1 import matplotlib.pyplot as plt
2
3 def f(x):
4     y=x/(1+x**2)
5     return y
6
7 x=0
8 h=0.001
9 t=0
10
11 while f(x)<= f(x+h):
12     t+=1
13     x=x+h
14
15 print(x)
16 print(t)
17
18 x = np.linspace(-3,3, num = 101)
19 plt.plot(x,f(x), '--')
20 plt.grid()
21 plt.xlabel('x')
22 plt.ylabel('y')
23 plt.legend(['f(x)'])
```

I programmet er definert funksjonen $f(x) = \frac{x}{1+x^2}$ og programmet prøver å finne nærmeste minimumspunkt til $x = 0$ med å ta tusen steg fra 0 med steglengde $h = 0.001$. $\diamond$

Oppg. 1.3.4: Skriv kort hva følgende program utfører.

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 def f(x):
5     y=x**3-3*x**2
6     return y
7
8 def derivert(a, deltax):
9     return(f(a + deltax) - f(a))/deltax
10
11 xstart = -2
12 xslutt = 5
13 deltax = .001
14 antallpunkter = 501
15
16 x = np.linspace(xstart, xslutt, antallpunkter)
17 y=f(x)
18 dy = derivert(x, deltax)
19 plt.plot(x,y)
20 plt.plot(x, dy, '--')
21
22 plt.plot(x,y, '--')
23 plt.plot(x,dy, '--')
24 plt.legend(['Funksjonen y=f(x)', 'Den deriverte dy/dx'] )
25 plt.xlabel('x')
26 plt.ylabel('y')
27 plt.grid()
28 plt.show()
```

1.4 Numerisk derivasjon og endelige differanser (ekstramateriale)

Stigningstallet mellom to punkt $x = x_0$ og $x = x_0 + h$ kan uttrykkes ved:

$$\frac{f(x_0 + h) - f(x_0)}{h} \quad (1.2)$$

Hvis $|h|$ blir mindre og mindre, så nærmer sekanten seg mer og mer tangenten, fordi punktet $(x+h, f(x+h))$ kommer nærmere og nærmere $(x, f(x))$:

$$f'(x_0) \simeq \frac{f(x_0 + h) - f(x_0)}{h}$$

La f være en analytisk funksjon. Den taylorutviklingen til f om punktet $x = x_0$ er:

$$\begin{aligned} f(x_0 + h) &= f(x_0) + \frac{f'(x_0)}{1!}h + \frac{f''(x_0)}{2!}h^2 + \frac{f'''(x_0)}{3!}h^3 + \dots \\ \frac{f(x_0 + h) - f(x_0)}{h} &= \frac{f'(x_0)}{1!} + \frac{f''(x_0)}{2!}h + \frac{f'''(x_0)}{3!}h^2 + \dots \end{aligned}$$

som beskriver feilen i estimeringen av sekanten mellom punktene $x = x_0$ og $x = x_0 + h$:

$$\frac{f(x_0 + h) - f(x_0)}{h} - f'(x_0) = \frac{f''(x_0)}{2!}h + \frac{f'''(x_0)}{3!}h^2 + \dots$$

- Tilnærmingen:

$$f'(x_0) \simeq \frac{f(x_0 + h) - f(x_0)}{h}$$

kalles *forlengsdifferanse* og dens største estimeringsfeilen er av orden h . Tilsvarende:

$$\frac{f(x_0) - f(x_0 - h)}{h} = \frac{f'(x_0)}{1!} - \frac{f''(x_0)}{2!}h + \frac{f'''(x_0)}{3!}h^2 + \dots$$

som beskriver feilen i estimeringen av sekanten mellom punktene $x = x_0 - h$ og $x = x_0$:

- Tilnærmingen:

$$f'(x_0) \simeq \frac{f(x_0) - f(x_0 - h)}{h} \quad (1.3)$$

kalles *baklengsdifferanse*, og dens største estimeringsfeilen er av orden h .

$$\frac{f(x_0) - f(x_0 - h)}{h} - f'(x_0) = -\frac{f''(x_0)}{2!}h + \frac{f'''(x_0)}{3!}h^2 + \dots$$

Nå kan vi sette opp:

$$\frac{f(x_0 + h) - f(x_0 - h)}{2h} - f'(x_0) = \frac{f'''(x_0)}{3!}h^2 + \frac{f^{(5)}(x_0)}{3!}h^2 + \dots \quad (1.4)$$

- Tilnærmingen:

$$f'(x_0) \simeq \frac{f(x_0 + h) - f(x_0 - h)}{2h} \quad (1.5)$$

kalles *sentraldifferanse* (endelig differanse), og estimeringsfeilen er av orden h^2 . Lignende endelige differanser kan brukes til å bestemme deriverte av høyere orden:

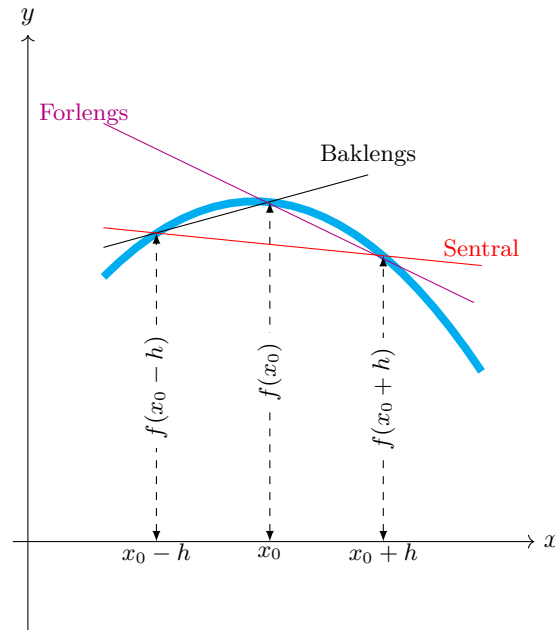
$$\frac{f(x_0 + h) - 2f(x_0) + f(x_0 - h))}{2h} - f''(x_0) = \frac{f^{(4)}(x_0)}{12}h^2 + \dots$$

Da blir:

$$f''(x_0) \approx \frac{f(x_0 + h) - 2f(x_0) + f(x_0 - h))}{h^2}$$

De tre endelige differanse approksimasjonene for å bestemme $f'(x_0)$ er da:

$$\begin{aligned} f'(x_0) &\approx \frac{f(x_0 + h) - f(x_0)}{h}, && \text{forlengsdifferanse,} \\ f'(x_0) &\approx \frac{f(x_0) - f(x_0 - h)}{h}, && \text{baklengsdifferanse,} \\ f'(x_0) &\approx \frac{f(x_0 + h) - f(x_0 - h)}{2h}, && \text{sentraldifferanse.} \end{aligned}$$



Figur 1.1: Endelige differanse-metoder

I figuren 1.1 viser den lilla linjen en forlengsdifferanse, den svarte linjen en baklengsdifferanse og den røde linjen en sentraldifferanse.

I sentraldifferansen er avstanden til hvert av punktene $2h$, mens det for forlengs- og baklengsdifferansen er h . Dermed er i sentraldifferansen delt på $2h$, i stedet for h for å approksimere den deriverte i punktet x_0 .

MERK. Feilleddet i sentraldifferansen er av størrelsesorden h^2 (konstant $\cdot h^2$), mens det for forlengs- og baklengsdifferansen er av størrelsesorden h (konstant $\cdot h$). Dermed er sentraldifferansen en bedre approksimasjon av den deriverte enn forlengs- og baklengsdifferansen. Sentraldifferansen er middelerdien til forlengs- og baklengsdifferansen.

Oppg. 1.4.1: Bestem en tilnærmingsverdi $f'(0.5)$ ved hjelpe de tre differanse-metodene gitt:

-0.25	0.273
0	0.385
0.25	0.448

Oppg. 1.4.2: De tre differansene for å bestemme den deriverte til f i punktet $x = x_0$ kan skrives som

$$\begin{aligned}f'(x_0) &\approx \frac{f(x_0 + h) - f(x_0)}{h}, && \text{forlengsdifferanse,} \\f'(x_0) &\approx \frac{f(x_0) - f(x_0 - h)}{h}, && \text{baklengsdifferanse,} \\f'(x_0) &\approx \frac{f(x_0 + h) - f(x_0 - h)}{2h}, && \text{sentraldifferanse.}\end{aligned}$$

Skriv et program som beregner den deriverte til funksjonen til $f(x) = e^x - 4x$ i $x = 2$ ved hjelp av de tre metodene for $h = 0.1$, $h = 0.01$, $h = 0.001$ og $h = 0.0001$.

Tegn grafen til funksjonen sammen med den numeriske derivasjonen med $h = 0.0001$ i intervallet $[-3, 3]$.

Oppg. 1.4.3: Skriv et program som bruker numerisk derivasjon til å finne tilnærminger til de deriverte av $f(x) = x^2$ i $x = 3$ for $h = 0.1$, $h = 0.01$, $h = 0.001$ og $h = 0.0001$.

Tegn grafen til funksjonen sammen med den numeriske derivasjonen for $h = 0.0001$ i intervallet $[-2, 2]$.

Oppg. 1.4.4: Gitt et python program:

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 n = 6
5
6 x = np.linspace(1,4,num = n+1)
7 deltax = x[1] - x[0]
8 f = 4*x**3 - 2*x
9
10 Sum = 0
11
12 for i in range(len(x)-1):
13     Sum = Sum + ((f[i]+f[i+1])/2)*HVA
14
15 print(Sum)
16
17 N=10000
18 u = np.linspace(1,4,num = N+1)
19 f1 = 4*u**3 - 2*u
20
21 plt.plot(x,f)
22 plt.plot(u,f1, label='f(x)')
23 plt.grid(ls='--')
24 plt.legend()
```

- Forklar hva programmet utfører og skriv kort om detaljene i programmet.
Hva kan det skrives for HVA i programmet?
- Hvordan kan man øke presisjonen for å bestemme Sum i programmet?

1.5 Nedstigningsmetoden (gradient descent) for å finne et minimum] (ekstramateriale)

Mål. Gitt en glatt funksjon $f : \mathbb{R} \rightarrow \mathbb{R}$ ønsker vi å finne et punkt x^* som (lokalt) minimerer f .

Idé. Den deriverte (gradienten i 1D) $f'(x)$ peker i retning av størst økning. For å *minimere* går vi i motsatt retning:

$$x_{k+1} = x_k - \alpha f'(x_k),$$

der $\alpha > 0$ er steglengden (læringsraten).

Algoritme:

- Velg startpunkt x_0 og steglengde $\alpha > 0$.
- Gjenta for $k = 0, 1, 2, \dots$:

$$x_{k+1} = x_k - \alpha f'(x_k).$$

- Stopp når f. eks. $|x_{k+1} - x_k| < \varepsilon$ eller $|f'(x_k)| < \varepsilon$, eller når et maks antall iterasjoner nås.

Valg av steglengde (læringsraten):

- For liten $\alpha \Rightarrow$ kan konvergere, men treg konvergens.
- For stor $\alpha \Rightarrow$ kan overskyte/oscillere og ikke konvergere.
- For kvadratiske funksjoner $f(x) = ax^2 + bx + c$ med $a > 0$ er et trygt spenn $\alpha \in (0, \frac{1}{L})$, der L er Lipschitz-konstanten til f' (for $ax^2 + bx + c$ er $L = 2a$). I vårt eksempel under holder $0 < \alpha < 1$.

Fallgruver.

- *Lokale minima/sadler:* I høyere dimensjoner kan du ende i et lokalt minimum eller en sadel, avhengig av startpunkt.
- *Dårlig skalering:* I smale “daler” blir bevegelsen sikksakk og treg; preconditioning/skalering kan hjelpe.
- *Fast α :* En fast læringsrate kan være suboptimal; linjesøk eller adaptiv α kan gi raskere og mer robust konvergens.

Eksempel 1.5.1: La $f(x) = x^2 + 2x + 1 = (x + 1)^2$. Da er $f'(x) = 2x + 2$. Minimumet er åpenbart $x^* = -1$, men vi bruker nedstigning for å finne det numerisk. Oppdateringen blir

$$x_{k+1} = x_k - \alpha(2x_k + 2).$$

Setter vi $y_k = x_k + 1$, får vi $y_{k+1} = (1 - 2\alpha)y_k$.

Dermed konvergerer $y_k \rightarrow 0$ (altså $x_k \rightarrow -1$) når $|1 - 2\alpha| < 1 \iff 0 < \alpha < 1$.

Python (enkel implementasjon).

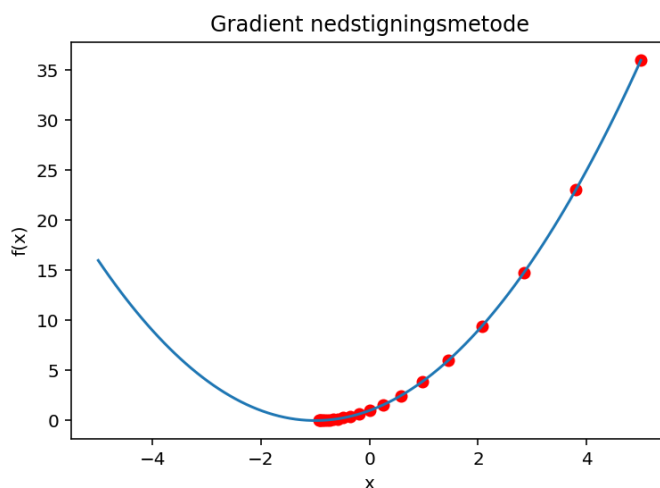
Koden under viser metoden og stopper når trinnene blir små.

```
1 # Nedstigningsmetoden på f(x) = (x + 1)^2
2 def f(x):
3     return x**2 + 2*x + 1      # f(x)
4 def df(x):
5     return 2*x + 2
6
7 x      = 5.0      # startpunkt
8 alpha  = 0.1      # steglengde (0 < alpha < 1 gir konvergens her)
9 tol    = 1e-6     # toleranse for stopping
10 maxit  = 1000    # høyeste antall iterasjoner
11
12 for i in range(1, maxit+1):
13     xny = x - alpha * df(x)
14     if abs(xny - x) < tol: # stoppkriterium
15         x = xny
16         break
17     x = xny
18
19 print(f"Iterasjoner: {i}, x* = {x:.6f}, f(x*) = {f(x):.6e}")
20 # Forventet: x* ~ -1.0 og f(x*) ~ 0.0
```

Ved å kjøre programmet, får man:

```
1 Minimum funnet ved x = -0.9308
```

Hva du skal se etter. Med $\alpha=0.1$ starter vi i $x=5$ og nærmer oss raskt $x^* = -1$. Endrer du α til en større verdi (f.eks. 0.9), vil metoden fortsatt konvergere, men den kan oscillere mer før den roer seg. Setter du $\alpha \geq 1$, vil den *ikke* konvergere i dette eksempelet (fordi $|1 - 2\alpha| \geq 1$).



◇

1.6 Newtons metode

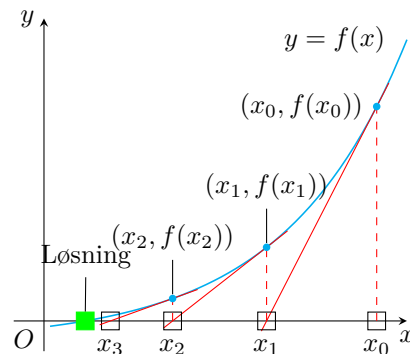
Newtons metode er en numerisk metode til å finne en løsning til ligningen $f(x) = 0$, som ligger i intervallet $[a, b]$.

La f være kontinuerlig på $[a, b]$ og ifølge skjæringssetningen har ligningen minst en løsning dersom $f(a) \cdot f(b) < 0$. Algoritmen begynner med en startverdi $x = x_0$, der $a < x_0 < b$ og man bestemmer hvor tangenten gjennom punktet $(x_0, f(x_0))$ treffer x -aksen og neste punkt blir utgangspunkt for neste iterasjon og da man bestemmer neste x -verdi, x_{n+1} er på samme måte skjæringspunktet mellom tangenten gjennom $(x_n, f(x_n))$:

$$y = f(x_n) + f'(x_n)(x - x_n)$$

og x -aksen ($y = 0$), som blir x_{n+1} ($x = x_{n+1}$):

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$$



Oppg. 1.6.1: Forklar kort hva følgende programkode utfører:

```
1  xn = x0
2  for n in range(0,N):
3      fxn = f(xn)
4      if abs(fxn) < epsilon:
5          print('Svar er funnet etter',n,'iterations.')
6          return xn
7      Dfxn = Df(xn)
8      if Dfxn == 0:
9          print('Ingen svar')
10         return None
11         xn = xn - fxn/Dfxn
12     print('Overstiger maks antall interasjoner. Ingen løsning funnet')
13     return None
14
15 f = lambda x: x**3 - 3*x**2 - 1
16 Df = lambda x: 3*x**2 - 6*x
17
18 Losn= newton(f,Df,0.1,1e-2,100)
19 print(Losn)
```

Oppg. 1.6.2: Forklar kort hva følgende programkode utfører:

```
1  import numpy as np
2
3  def newton(f,df,x0,tol,N):
4      xn = x0
5      for n in range(0,N):
6          fxn = f(xn)
7          if abs(fxn) < tol:
8              print('Losn er funnet etter',n,'iterasjoner.')
9              return xn
10         dfxn = df(xn)
11         if dfxn == 0:
12             print('Den deriverte er null. Ingen losn er funnet.')
13             return None
14         xn = xn - fxn/dfxn
15     print('Maksimalt antall iterasjoner er overskredet. Ingen losn funnet.')
16     return None
```

```

17
18 def f(x):
19     return x**3 - np.cos(x)
20
21 def df(x):
22     return 3*x**2 + np.sin(x)
23
24 Losn = newton(f, df, 1, 1e-10, 10)
25 print(Losn)

```

Oppg. 1.6.3: Vis at ligningen $x + \sin x + 1 = 0$ har *presis* ett nullpunkt x_0 i intervallet $[-\pi/2, 0]$. Bruk Newtons metode tre ganger, med startverdi $x_0 = -1$, til å finne en tilnærmet verdi til dette nullpunktet.

Oppg. 1.6.4: Skriv et program som finner en tilnærmet løsning for ligningen $x^3 - x^2 = 3$ som ligger i intervallet $[0, 2]$ (bruk startverdien $x = 1.5$).

(Hint: programmet fortsetter til $|f(x_n)| < \text{epsilon}$, for en gitt toleranseverdi (epsilon))

1.7 Halveringsmetoden (ekstramateriale)

Halveringsmetoden er en enkel iterativ metode for å finne et nullpunkt til en funksjon, dvs. finne en løsning på likningen $f(x) = 0$. For å bruke metoden må vi kjenne to punkt a og b som er slik at $f(a)$ og $f(b)$ har ulikt fortegn ($f(a) \cdot f(b) < 0$), og funksjonen må være kontinuerlig mellom på intervallet $[a, b]$. Vi starter med å finne midtpunktet m mellom a og b . Hvis $f(m)$ og $f(a)$ har ulikt fortegn ($f(a) \cdot f(m) < 0$), så letter vi videre i intervallet $[a, m]$. Hvis ikke, fortsetter vi med intervallet $[m, b]$. Slik kan vi fortsetter til svaret tilfredsstiller toleransekravet at når funksjonsverdien til midtpunktet er tilstrekkelig nær null (hva som er tilstrekkelig velger du!), er algoritmen ferdigkjørt, og du har estimert nullpunktet for funksjonen.

Oppg. 1.7.1: Skriv kort hva følgende program utfører

```

1 import math
2
3 def f(x):
4     return 2*x*math.cos(x)-1
5
6 a = 0
7 b = 1
8 e = .001
9 m = (a + b)/2
10 n = 0
11
12 while b - a > e:
13     if f(a) * f(m) < 0:
14         b = m
15     elif f(m) == 0:
16         break
17     else:
18         a = m
19     m = (a + b)/2
20     n = n+1
21
22 print(m)
23 print("Antall iterasjoner:", n)

```

Oppg. 1.7.2: Vi ønsker å bestemme en tilnærmet verdi for $\sqrt[3]{x}$ og vil bruke halveringsmetoden for å løse ligningen $x^3 = 5$.

- Skriv et program som finner en tilnærmet løsning for ligningen ved hjelp av halveringsmetoden. I programmet gis endepunktene til et intervallet $[1, 2]$ og en presisjon(e). Sett inn en kommando som viser hvor mange ganger løken kjører.

- b) En student foreslår en algoritme for å finne nullpunktet slik at å nærme seg mot løsningen fra en av endepunktene med en gitt margin (en infinitesimal endring) Δx . Programmet sjekker hver gang om grafen har passert x -aksen, for eksempel $f(\text{endepunkt}) \cdot f(\text{endepunkt} + \text{Deltax})$ ikke har skiftet fortegn fra fortegnet til funksjonsverdi i det endepunktet. Programmet skal angi antall iterasjoner også.
- c) En annen student foreslår en algoritme som bruker tilfeldig søk:

```

1 import random
2 def f(x):
3     return x**3 - 5
4 a = 1
5 b = 2
6 N = 1000
7 n = 0
8 Beste_y = f(a)
9 Beste_x = a
10 for i in range(N):
11     x = random.uniform(a, b)
12     y = f(x)
13     if abs(y) < abs(Beste_y):
14         Beste_y = y
15         Beste_x = x
16         n = n+1
17 print(Beste_x)
18 print("Antall iterasjoner:", n)

```

Ved å kjøre programmet, får vi forskjellige verdier som forventet da vi bruker tilfeldig søk blant 1000 tall i intervallet.

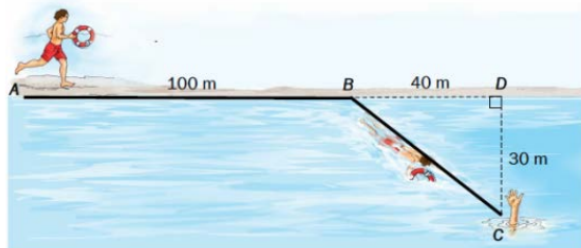
```

1 1.710007277735471
2 Antall iterasjoner: 7
3 1.7099466831544587
4 Antall iterasjoner: 13

```

Oppg. 1.7.3: Vis at ligningen $x + \sin(x) + 1 = 0$ har *presis* ett nullpunkt x_0 i intervallet $[-\pi/2, 0]$. Bruk Newtons metode tre ganger, med startverdi $x_0 = -1$, til å finne en tilnærmet verdi til dette nullpunktet.

Oppg. 1.7.4: (Modelleringsoppgave) En badevakt løper fra A til B og svømmer videre fra B til C for å hjelpe en person i nød. Se skissen nedenfor:



- Bestem avstanden fra B til C.
- Badevakten løper i 20 sekunder og svømmer i 1 min .
Regn ut forholdet mellom farten han har når han løper, og farten han har når han svømmer.
- La nå avstanden mellom A og D være 140 meter, $CD = 30\text{m}$, og $BD = x$.
Badevakten kan løpe på land med farten 5 m/s og kan svømme i vann med en fart på 1 m/s.
Badevakten ønsker å finne hvor på land han skal begynne å svømme for å kunne komme til punktet C så raskt som mulig.
Hva blir den minste tiden da? Ser tiden du får fornuftig ut?
- (Programmeringsdel)** Anta at lengdene AD og CD leses inn fra skjermen, og at vi kjenner farten på land v_1 og i vann v_2 . Vi ønsker å finne det optimale punktet $x = BD$ hvor badevakten skal

begynne å svømme, ved å bruke numerisk derivasjon:

$$t'(x) \approx \frac{t(x+h) - t(x)}{h}, \quad t(x) = \frac{AD - x}{v_1} + \frac{\sqrt{x^2 + CD^2}}{v_2}.$$

Vi ønsker å lage et program som:

1. Leser inn verdier for AD , CD , v_1 og v_2 .
2. Definerer funksjonen $t(x)$.
3. Beregner den numeriske deriverte $t'(x)$ med differansekvtient.
4. Søker etter et x der $t'(x) = 0$ (for eksempel ved å sjekke fortegnsskifte i $t'(x)$).
5. Skriver ut verdien av x og den tilhørende minste tiden.

Et eksempel på Python-kode kan være.

```
1 import numpy as np
2
3 # Leser inn verdier
4 AD = float(input("Lengden AD (m): "))
5 CD = float(input("Lengden CD (m): "))
6 v1 = float(input("Fart på land v1 (m/s): "))
7 v2 = float(input("Fart i vann v2 (m/s): "))
8
9 # Funksjon for total tid
10 def t(x):
11     return (AD - x)/v1 + np.sqrt(x**2 + CD**2)/v2
12
13 # Numerisk derivasjon
14 def dt(x, h=1e-5):
15     return (t(x+h) - t(x)) / h
16
17 # Søker etter nullpunkt til t'(x) i intervallet [0, AD]
18 xverdier = np.linspace(0, AD, 1000)
19 df = [dt(x) for x in xverdier]
20
21 xopt = None
22 for i in range(len(xverdier)-1):
23     if df[i] == 0 or df[i]*df[i+1] < 0:
24         xopt = xverdier[i]
25         break
26
27 if xopt is not None:
28     print(f"Optimalt punkt x: {xopt:.2f} m")
29     print(f"Minste tid: {t(xopt):.2f} sekunder")
30 else:
31     print("Fant ingen nullpunkt i intervallet.")
```

Forklar hva programmet utfører.

1.8 Numerisk integrasjon

Det bestemte integralet $\int_a^b f(x)dx$ lar seg ikke alltid integrere ved å integrere analytisk for å bestemme den eksakte verdien til integralet. Det kan være fordi integranden $f(x)$ er noe som vi ikke kan integrere ved hjelp av analytiske metoder eller $f(x)$ er et resultat av målinger og man ikke har noen analytisk funksjon å integrere. Dermed er det viktig å ha numeriske metoder som kan beregne integralet. Det finnes flere numeriske metoder for å regne ut et bestemt integral. Vi fokuserer her på tre kjente metoder.

Riemannsum (nedre og øvre sum)

La $f : [a, b] \rightarrow \mathbb{R}$ være en kontinuerlig funksjon. Vi ønsker å approksimere integralet

$$I = \int_a^b f(x) dx$$

ved å dele intervallet $[a, b]$ i N like store deler med

$$\Delta x = \frac{b-a}{N}, \quad x_k = a + k \Delta x \quad (k = 0, \dots, N).$$

Riemannsummer:

$$Ln = \sum_{k=0}^{N-1} f(x_k) \Delta x \quad (\text{nedre sum}), \quad Un = \sum_{k=1}^N f(x_k) \Delta x \quad (\text{Øvre sum}),$$

$$Mn = \sum_{k=0}^{N-1} f\left(\frac{x_k + x_{k+1}}{2}\right) \Delta x \quad (\text{midtsum}).$$

Når $N \rightarrow \infty$ vil Ln , Un og Mn alle konvergere mot det eksakte integralet I .

Eksempel 1.8.1: Vi velger $f(x) = x^2$ på $[0, 1]$ med $N = 10$ delintervaller.

```
1 import numpy as np
2
3 # Funksjon f(x) = x^2 defineres
4
5 def f(x):
6     return x**2    # Funksjon f(x) = x^2
7
8 # f = lambda x: x**2
9
10 # Parametre
11 a, b = 0, 1        # start og slutt
12 N = 10             # antall delintervaller
13 dx = (b - a)/N     # delta x
14
15 # x-verdier
16 x = np.linspace(a, b, N+1)
17
18 # Riemann-summer
19 L = sum(f(x[k])      for k in range(N))      * dx    # Nedre sum
20 U = sum(f(x[k+1])    for k in range(N))      * dx    # Øvre sum
21 M = sum(f((x[k]+x[k+1])/2) for k in range(N)) * dx    # Midtsum
22
23 # Skriv resultater i tabell
24 print(" N      Nedre Ln      Midt Mn      Øvre Un")
25 print("-----")
26 print(f"{N:2d}   {L:10.6f}   {M:10.6f}   {U:10.6f}")
```

Kommentar. For $f(x) = x^2$ kjenner vi det eksakte svaret:

$$\int_0^1 x^2 dx = \left[\frac{x^3}{3} \right]_0^1 = \frac{1}{3} \simeq 0.33333.$$

◇

Øvre og nedre sum gir hhv. over- og underestimat; midtsummen gir ofte en raskere konvergens mot det sanne integralet når N økes. Med å kjøre programmet, får vi:

1	N	Venstre Ln	Midt Mn	Høyre Un
2	-----			
3	10	0.285000	0.332500	0.385000

Oppgaver - integrasjon

Oppg. 1.8.1: Bruk programmet ovenfor for å:

- estimere $\int_0^2 (x^2 + 1) dx$ med $N = 5, 10, 50$.
- Sammenligne resultatene for venstre, høyre og midt-rektangelmetoden.
- Diskutere hvordan feilen avhenger av N .

1.9 Trapesmetoden

Vi skal regne ut $\int_a^b f(x)dx$ numerisk. Vi begynner med å dele inn intervallet $[a, b]$ i n like store deler,

$$a = x_0 < x_1 < x_2 < \dots < x_n = b,$$

slik at alle delintervallene $[x_{i-1}, x_i]$ har samme bredde $\Delta x = x_i - x_{i-1}$. Nå kan vi tilnærme grafen til $f(x)$ ved å trekke rette linjestykker mellom punktene $(x_i, f(x_i))$. Idéen er nå å tilnærme integralet $\int_a^b f(x)dx$ ved hjelp av summen av alle arealene (med fortegn) mellom x -aksen og disse linjestykkene. Arealet under hvert linjestykke er et trapes. Vi har da følgende formel:

$$\int_a^b f(x)dx \approx \frac{\Delta x}{2} (f(x_0) + 2f(x_1) + 2f(x_2) + \dots + 2f(x_{n-1}) + f(x_n))$$

der $\Delta x = \frac{b-a}{n}$ og $x_i = a + i\Delta x$.

Legg merke til at i denne metoden er første funksjonsverdi, $f(x_0)$, og siste funksjonsverdi $f(x_n)$ benyttet kun en gang i henholdsvis utregning av første og siste areal. De andre funksjonsverdiene er benyttet to ganger av forrige og neste areal i grafen.

Oppsummering - Trapesmetoden Vi vil approksimere integralet

$$I = \int_a^b f(x) dx$$

ved å dele intervallet $[a, b]$ i N like deler,

$$h = \frac{b-a}{N}, \quad x_k = a + kh, \quad k = 0, \dots, N.$$

Formelen er

$$T_n = \frac{h}{2} (f(x_0) + 2 \sum_{k=1}^{N-1} f(x_k) + f(x_N)).$$

Feilen er av orden $O(h^2)$ hvis f'' er kontinuerlig.

Eksempel 1.9.1 (Trapezmetoden): Vi vil beregne $\int_0^\pi \sin(x) dx = 2$ for $N = 4, 8, 16$.

```
1 import numpy as np
2
3 f = lambda x: np.sin(x)
4 a, b = 0.0, np.pi
5 Nverdier = [4, 8, 16]
6
7 print(" N      Trapez Tn      Feil")
8 print("-----")
9 for N in Nverdier:
10     h = (b - a)/N
11     x = np.linspace(a, b, N+1)
12     T = 0.5*h*(f(x[0]) + 2*sum(f(x[1:-1])) + f(x[-1]))
13     error = abs(T - 2.0)
14     print(f"{N:2d}   {T:12.8f}   {error:10.2e}")
```

Oppg. 1.9.1:

- Bruk koden til å beregne T_n for $N = 4, 8, 16$.
- Sjekk hvordan feilen endres når du dobler N .

1.10 Simpsons metode (ekstramateriale)

I trapesmetoden tilnærmet vi grafen til funksjonen $f(x)$ ved hjelp av linjestykker. Nå skal vi i stedet tilnærme grafen til $f(x)$ ved hjelp av biter av andregradskurver. En rett linje er entydig bestemt av *to punkt* på linjen. Et andregradspolynom er, som vi skal se, entydig bestemt av *tre punkt* på grafen. Derfor skal vi dele inn integrasjonsintervallet $[a, b]$ i et partall antall $2n$ delintervaller,

$$a = x_0 < x_1 < x_2 < \dots < x_{2n} = b$$

og så, på hvert “doble” delintervall $[x_{2i-2}, x_{2i}]$, tilnærme $f(x)$ ved hjelp andregradspolynomet som har de samme verdier som f i punktene x_{2i-2} , x_{2i-1} og x_{2i} . På denne måten tilnærmer vi grafen til f ved hjelp av n andregradskurver. Arealet under hver andregradskurve finner vi ved integrasjon, og så summerer vi alle disse arealene.

Simpsons metode med $2n$ delintervaller gir tilnærmingen

$$\int_a^b f(x)dx \approx \frac{\Delta x}{3}(f(x_0) + 4f(x_1) + 2f(x_2) + 4f(x_3) + 2f(x_4) + 4f(x_5) + \dots + f(x_{2n}))$$

der $\Delta x = \frac{b-a}{2n}$ og $x_i = a + i\Delta x$.

Simpsonmetoden: Krever partall N . Med $h = (b-a)/N$:

$$S_N = \frac{h}{3} \left(f(x_0) + 4 \sum_{k=1, \text{ odd}}^{N-1} f(x_k) + 2 \sum_{k=2, \text{ even}}^{N-2} f(x_k) + f(x_N) \right).$$

Feilen er av orden $O(h^4)$ hvis $f^{(4)}$ er kontinuerlig.

Eksempel 1.10.1 (Simpsonmetoden): Vi beregner $\int_0^\pi \sin(x) dx = 2$ med Simpsonmetoden.

```
1 import numpy as np
2
3 f = lambda x: np.sin(x)
4 a, b = 0.0, np.pi
5 Nverdier = [4, 8, 16]
6
7 print(" N      Simpson S_N      Feil")
8 print("-----")
9 for N in Nverdier:
10     h = (b - a)/N
11     x = np.linspace(a, b, N+1)
12     S = f(x[0]) + f(x[-1])
13     Soddetall = sum(f(x[k]) for k in range(1, N, 2))
14     Spartall = sum(f(x[k]) for k in range(2, N-1, 2))
15     S = (h/3)*(S + 4*Soddetall + 2*Spartall)
16     error = abs(S - 2.0)
17     print(f"{N:2d} {S:12.8f} {error:10.2e}")
```

Oppg. 1.10.1:

- Sammenlign trapes- og Simpsonmetoden for $N = 8, 16$.
- Hvorfor er Simpson mer nøyaktig?

Litt om feilen i trapesmetoden og Simpson metode

Det kan være krevende å vise feilestimat for trapesmetode. Her skal vi kun se på resultatet. For et skikkelig bevis, se: Adams, *Calculus*.

Teorem 1.10.2:

Anta at f er en to ganger deriverbar funksjon, definert på et åpent intervall som inneholder $[a, b]$. Anta også at $|f''(x)| < K$ for alle $x \in [a, b]$. Da er feilen ved å bruke trapesmetoden med n delintervaller er mindre enn $K(b-a)^3/12n^2$:

$$\left| \int_a^b f(x) dx - \frac{1}{2} \left(f(a) + 2 \sum_{i=1}^{n-1} f(a + i\Delta x) + f(b) \right) \cdot \Delta x \right| < \frac{K(b-a)^3}{12n^2}$$

Teorem 1.10.3:

Anta at f er en fire ganger deriverbar funksjon, definert på et åpent intervall som inneholder $[a, b]$. Anta også at $|f^{(4)}(x)| < M$ for alle $x \in [a, b]$. Da er feilen ved å bruke Simpsons metode med n delintervaller er mindre enn $M(b-a)^4/180n^4$:

$$\left| \int_a^b f(x) dx - \frac{1}{3} \left(f(a) + \sum_{i=1}^{n/2} f(x_{2i}) + \sum_{i=1}^{n/2-1} f(x_{2i}) + f(b) \right) \cdot \Delta x \right| < \frac{M(b-a)^5}{180n^4}$$

Oppgaver- Numerisk integrasjon

Oppg. 1.10.2: Et program er gitt som nedenfor:

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 import scipy.integrate as integrate
4
5 a = 0
6 b = 1
7 N = 20
8 Dx = (b - a) / N
9 x = np.linspace(a, b, N+1)
10
11 def f(x):
12     return x / (1 + x**2)
13
14 eksakt, feil = integrate.quad(f, a, b)
15
16 riemann = 0
17 for k in range(N):
18     riemann += f(x[k]) * Dx
19
20 avvik = abs(eksakt - riemann)
21
22 print(f"Venstre Riemann med N={N} gir: {riemann:.6f}")
23 print(f"Eksakt verdi: {eksakt:.6f}")
24 print(f"Avvik: {avvik:.6e}")
25
26 X = np.linspace(a, b, 1000)
27 Y = f(X)
28
29 plt.figure(figsize=(10, 5))
30 plt.plot(X, Y, 'b', label="f(x)")
31 plt.bar(x[:-1], f(x[:-1]), width=dx, alpha=0.3,
32         align='edge', edgecolor='black', label="Riemann-rektangler")
33 plt.title(f"Venstre Riemann-sum med N={N}")
34 plt.xlabel("x")
35 plt.ylabel("f(x)")
36 plt.legend()
37 plt.show()
```

- Forklar kort hva programmet utfører, det vil si hvilken problemstilling programmet prøver å løse, og hvilken metode brukes i programmet.
- Sett opp et Python-program som bestemmer venstre Riemann-sum med antall partisjoner $N = 10$ og $N = 100$. Beregn også estimeringsfeilen ved å sammenligne med eksakt verdi.

$$\int_a^b f(x) dx \approx \Delta x (f(x_0) + f(x_1) + \dots + f(x_{n-1}))$$

Oppg. 1.10.3: Gitt integralet $\int_0^5 \frac{1}{x^2 + 1} dx$.

- Regn ut integralet.
- Sett opp et Python-program som bestemmer nedre (venstre) Riemann sum (nedre sum) med antall partisjoner $N = 10$ og $N = 100$.

$$\int_a^b f(x) dx \approx \Delta x (f(x_0) + f(x_1) + f(x_2) + \dots + f(x_{n-1}))$$

Angi også estimeringsfeilen for hver av delsummene ved å bestemme differensen mellom den høyre og venstre Riemannsummen.

Oppg. 1.10.4: Et program er gitt som nedenfor:

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 import scipy.integrate as integrate
4
5 a = 0
6 b = 2
7 N = 20
8
9 Dx = (b - a) / N
10 x = np.linspace(a, b, N+1)
11
12 def f(x):
13     return x * np.exp(-x**2)
14
15 eksakt, feil = integrate.quad(f, a, b)
16
17 riemann = 0
18 for k in range(N):
19     riemann += f(x[k]) * Dx
20
21 avvik = abs(eksakt - riemann)
22
23 print(f"Venstre Riemann med N={N} gir: {riemann:.6f}")
24 print(f"Eksakt verdi: {eksakt:.6f}")
25 print(f"Avvik: {avvik:.6e}")
26
27 # Plot
28 X = np.linspace(a, b, 1000)
29 Y = f(X)
30
31 plt.figure(figsize=(10, 5))
32 plt.plot(X, Y, 'b', label="f(x)")
33 plt.bar(x[:-1], f(x[:-1]), width=Dx, alpha=0.3,
34         align='edge', edgecolor='black', label="Riemann-rektangler")
35 plt.title(f"Venstre Riemann-sum med N={N}")
36 plt.xlabel("x")
37 plt.ylabel("f(x)")
38 plt.legend()
39 plt.show()
```

- a) Forklar kort hva programmet utfører, det vil si hvilken problemstilling programmet prøver å løse, og hvilken metode brukes i programmet.
- b) Sett opp et Python-program som bestemmer venstre Riemann-sum med antall partisjoner $N = 10$ og $N = 100$. Beregn også estimeringsfeilen ved å sammenligne med eksakt verdi.

$$\int_a^b f(x) dx \approx \Delta x (f(x_0) + f(x_1) + \dots + f(x_{n-1}))$$

Oppg. 1.10.5: Estimeringsavvikene for trapesmetoden og Simpsons regel er gitt ved:

$$|e_T| < \frac{\max_{[a,b]} f''(x)}{12n^2} (b-a)^3$$
$$|e_S| < \frac{\max_{[a,b]} f''''(x)}{180n^4} (b-a)^5$$

Bruk trapesmetoden og Simpsons regel til å beregne en tilnærmingsverdi for integralet og bestem estimeringsavvikene for disse metodene for $n = 4$ når: $\int_0^1 x^4 dx$

Oppg. 1.10.6: Et program er gitt nedenfor:

```
1 import numpy as np
2 import scipy.integrate as integrate
3
4 # Funksjonen vi integrerer
5 def f(x):
6     return np.exp(x**2)
7
8 # Integrasjonsgrenser og delintervaller
9 a, b = 0, 2
10 n = 4
11 dx = (b-a)/n
12 x = np.linspace(a, b, n+1)
13
14 trap = 0
15 for k in range(n):
16     trap += (f(x[k]) + f(x[k+1]))/2 * dx
17
18
19 eksakt, feil = integrate.quad(f, a, b)
20
21 avviktrap = abs(trap - eksakt)
22 print("Tapesmetoden:")
23 print(f"Integral: {trap}, Avvik: {avviktrap}")
24
25
26 simp = f(x[0]) + f(x[-1])
27 for k in range(1, n):
28     if k % 2 == 0:
29         simp += 2*f(x[k])
30     else:
31         simp += 4*f(x[k])
32 simp *= dx/3
33
34 avvik_simp = abs(simp - eksakt)
35 print("Simpsons regel:")
36 print(f"Integral: {simp}, Avvik: {avvik_simp}")
```

- Forklar kort hva programmet gjør og hvilke numeriske metoder som brukes.
- Regn ut integralet for både trapesmetoden og Simpsons regel, og sammenlign avvikene.

Oppg. 1.10.7: Gitt $\int_0^2 e^{x^2} dx$.

- Regn ut integralet ved hjelp av **trapesmetoden** ($n = 4$). Regn også ut estimeringsavviket.
- Regn ut integralet ved hjelp av **Simpsons regel** ($n = 4$). Regn også ut estimeringsavviket.
- Forklar kort hvilken metode som gir best resultat og hvorfor.

Oppg. 1.10.8: Et program er gitt nedenfor

```
1 import numpy as np
2 import scipy.integrate as integrate
3
4 # Funksjonen vi integrerer
5 def f(x):
6     return x**2
7
8
9 a, b = 0, 3
10 n = 6
11 dx = (b-a)/n
12 x = np.linspace(a, b, n+1)
13
14 trap = 0
15 for k in range(n):
16     trap += (f(x[k]) + f(x[k+1]))/2 * dx
17
18 eksakt, feil = integrate.quad(f, a, b)
19
20 avvik_trap = abs(trap - eksakt)
21 print("Trapezmetoden:")
22 print(f"Integral: {trap}, Avvik: {avvik_trap}")
23
24 simp = f(x[0]) + f(x[-1])
25 for k in range(1, n):
26     if k % 2 == 0:
27         simp += 2*f(x[k])
28     else:
29         simp += 4*f(x[k])
30 simp *= dx/3
31
32 avvik_simp = abs(simp - eksakt)
33 print("Simpsons regel:")
34 print(f"Integral: {simp}, Avvik: {avvik_simp}")
```

- Forklar kort hva programmet gjør og hvilke numeriske metoder som brukes.
- Regn ut integralet for både trapesmetoden og Simpsons regel, og sammenlign avvikene.

Kapittel 2

Litt programmering med Python

Python er et objektorientert programmeringsspråk, som ble laget av Guido van Rossum på 80-tallet, og ble først presentert i 1991. Python er et generell programmeringsspråk som kan brukes på alle moderne operativsystemer. Python har en god del fordeler, blant annet er det enkelt å bruke og har mange innebygde funksjoner. Python har en enkel syntaks som ligner på det engelske språket. Python er et ganske moderne programmeringsspråk som kan brukes i forskjellige vitenskapelig kontekster (f.eks. i maskinlæring). Den kan brukes til behandling av tekst, tall, bilder, vitenskapelige data og omtrent alt annet du kan bruke på en datamaskin. Den brukes daglig i driften av f.eks. Googles søkemotor, videodeling YouTube, NASA og New York Børs.

2.1 Å komme i gang med Python

Det er to hovedversjoner av Python, Python 2 og Python 3. Disse versjonene er ganske forskjellige. Her bruker vi Python 3, fordi den er mer semantisk korrekt og støtter nyere funksjoner.

Python er et tekstbasert programmeringsspråk, og for å komme igang med å programmere, så har man flere alternativer blant annet:

- Programmere på nett: Det er flere tjenester på nettet som lar deg skrive og kjøre python-programmer, som for eksempel:

<https://trinket.io/python3>

<https://replit.com/new/python3>

- Installere en kildekode-plattform som både er en kode-editor, men som har innebygget en egen «Python-motor».

<https://www.spyder-ide.org/>

<https://jupyter.org/>

Eksempel på et slikt program er Spyder. Dette er en åpen kildekode-plattform integrert i utviklingsmiljø for vitenskapelig programmering med Python. Spyder integreres med en rekke fremtredende pakker i den vitenskapelige Python-pakken, inkludert numpy, scipy, Matplotlib, pandas, IPython, sympy og Cython, samt annen programvare med åpen kildekode. Det finnes også andre slike program:

<https://www.anaconda.com/>

<https://codewith.mu/en/download>

Konsollvinduet og script i editorvinduet

I dette heftet har vi valgt å bruke Spyder. Vinduet til venstre er editorvinduet og nederst på høyre side er konsollvinduet. I Spyder kan man kjøre Python-kode direkte i konsollvinduet. Dette passer for små og enkle beregninger, for eksempel hvis vi bare skal regne ut et regnestykke som $2 + 5$,

```
1 In[1]: 2+5
2 Out[2]: 7
```

Men når man skriver lengre programmer, er det enklere å skrive koden i et *script*. Et Python-script er en kjørbart fil som inneholder Python-kode. Det vil si at hvis man skriver inn `2+5` i et script, så skjer det ikke noe før man kjører scriptet. Her er et eksempel på et script som skal beregne arealet til et rektangel:

```
1 a = 5
2 b = 3
3 # Regner ut de to røttene
4 A = a*b
5 print('Arealet til et rektangel med sidelengdene', 'a =', a, 'og b=', b, ' er
    lik', A)
```

Dersom vi kjører dette scriptet, vil vi få:

Arealet til et rektangel med sidelengdene a = 5 og b= 3 er lik 15

online-programmering

Det er anbefalt av flere læresteder å bruke:

<https://trinket.io/python3>

når man skal programmere på nettet.

2.2 Noen viktige elementer i Python-programmering

De viktige elementene i Python-scriptet er:

- *Kommentarer*: Når man programmerer, så lønner det seg å forklare hva koden gjør. Det gjør man med tegnet `#` eller `'` eller `"`. Dette tegnet tolker Python som «resten av denne linjen skal ikke leses av programmet». Kommentarer kan også settes mellom enten 'enkle apostrofer' eller doble apostrofer".
- *Streng*: En streng er en sekvens av tegn som er ment å tolkes av Python som tekst.
- *Funksjoner*: en gruppe kodelinjer, som utfører en spesifikk oppgave, for eksempel beregner en matematisk uttrykk eller skriver ut en tekst. En funksjon kan brukes flere steder i ett program, med å kalle funksjonen.
- *Pakker*: For å utføre beregninger, er det nødvendig å importere en passende pakke. Her importerer vi pakken `numpy` som `np`, slik at vi kan bruke for eksempel funksjonen `sqrt` som tilhører pakken som `np.sqrt`.
- *Lister*: Når vi trenger å lagre flere verdier i samme variabel, kan vi benytte lister. Lister er samlinger med tall, strenger, variabler eller annet.
- *Dat typer*: En datatype forteller kompilatoren (tolkeprogrammet) hvilken datatype, som behandles (tekst (`str`), tall (`int`, `float`), vektor (`array`)).
- *Variabler*: Videre definerer vi de tre koeffisientene a, b og c . I Python kalles disse *variabler*, og likhetstegnet angir verdien deres. Også x_1 og x_2 som vi så regner ut, er variabler, og vi ser at i utregningen brukes a, b og c , som har verdiene vi allerede har gitt dem.
- *Indeksering*: Indeksering er prosessen som ordner et sett med data slik at det blir mulig å spore opp datene. Indeksering i Python starter ved indeksen 0.
- *Løkker*: Løkker brukes for å gjenta noe mange ganger etter hverandre

2.3 Noen av Pythons innebygde funksjoner

Python-interpreter har en rekke innebygde funksjoner som alltid kan kalles med å skrive deres navn med riktige input som inngår i funksjonene.

Funksjon	Beskrivelse
<code>help()</code>	Kaller det innebygde hjelpesystemet
<code>print()</code>	Skriver ut data
<code>abs()</code>	Returnerer absoluttverdi av et tall
<code>float()</code>	Returnerer et flyttall
<code>int()</code>	Returnerer et heltall
<code>input()</code>	For å få input fra bruker
<code>input(float())</code>	For å få input (et reelt tall) fra bruker
<code>input(int())</code>	For å få input (et heltall) fra bruker
<code>input(str())</code>	For å få input (tekst) fra bruker
<code>len()</code>	Returnerer lengde av et objekt
<code>list()</code>	Returnerer en liste
<code>range()</code>	Returnerer en sekvens av hele tall, som standard starter fra 0 og inkrementerer tallene med 1
<code>round()</code>	Returnerer rundverdi av et tall

2.4 Grunnleggende Python-programmering

Kommentere i koden med ' ' og

Kommentarer (en forklarende tekst) er det første vi vanligvis møter i koden. En kommentar starter enten med en hash, og varer ut linjen, eller skrives inne i triple anførselstegn, som første linje i eksempelkoden over. Ved triple anførselstegn kan kommentaren vare over flere linjer. Dette er tekst eller kode som ignoreres når koden kjøres. Det er lurt å ha med kommentarer underveis i koden, fordi det gjør den enklere å lese, og det kan være til hjelp når du senere ser tilbake på den.

```
1 # -*- coding: utf-8 -*-
2 """
3 Created on Fri Dec 16 12:39:51 2022
4 @author: ahas
5 """
6 print('Skriv ut noe')
```

Ved å kjøre programmet får vi:

```
1 Skriv ut noe
```

Strenger

En streng er en sekvens av tegn som er ment å tolkes av Python som tekst.

Man kan skrive tekst (strenger) både med apostrof ('... ') og med anførselstegn ("..."):

```
1 print('Denne linjen blir skrevet ut')
2 print(' 'Denne linjen blir skrevet ut '')
```

Man får ikke feilmelding ved å bruke ekstra mellomrom:

```
1 print('Denne linjen blir skrevet ut')
2 print('Denne linjen blir skrevet ut')
```

Man får feilmelding hvis man glemmer noe av syntaksen man skal ta med:

```
1 print(Denne linjen blir skrevet ut')
```

Vi skal se nærmere i notatene at print-funksjonens tolker følgende som streng og som tall:

```
1 print('3+4')
2 print(3+4)
```

Resultatet av den første programsetningen blir 3+4 og den andre 7.

Oppg. 2.4.1: a) Skriv en programkode som skriver ut: Denne setningen skrives ut!

b) Finn feilen i følgende programsetninger:

```
1 Print('Jeg liker programmering')
2 print('Jeg liker programmering!')
```

Oppg. 2.4.2: Hva blir resultat av følgende programsetninger:

a) `print('5+7')`

b) `print(5+7)`

c) `print('5+7')`

d) `Print('5+7')`

e) `print('5+7')`
`print()`

Kalkulator

Vi ønsker at kalkulatoren skal kunne addere (+), subtrahere (-), multiplisere (*) og dividere (/). Vi kaller +, -, * og / for operatorer.

Matematiske operatorer

Operator	Utfører	Eksempel	Resultat
+	Addisjon	$5 + 2$	7
-	Subtraksjon	$5 - 2$	3
*	Multiplikasjon	$5 * 2$	10
/	Divisjon	$5 / 2$	2.5
//	Heltalldivisjon	$13 // 3$	4
**	EkspONENT	$2 ** 3$	8
%	Modulo	$13 \div 4$	1

- a/b gir kvotienten når divisoren a deles på dividenden b .
- $a//b$ gir antall heltall som er i kvotienten (når man deler a på b) når divisoren a deles på dividenden b .
- $a\%b$ gir restleddet når divisoren a deles på dividenden b .

Heltalldivisjon er divisjon uten restledd. Dette betyr at for eksempel både $14//7$ og $15//7$ blir 2, siden $14 : 7 = 2$ og $15 : 7 = 2 + \frac{1}{7}$.

- $a \div b$ (modulo) gir restleddet når man deler a på b .

Eksempel 2.4.1: Her er et eksempel fra konsollvinduet:

```
1 > 2+3+4*5
2 25
3 >3/2
4 1.5
5 >2**3 +1
6 9
7 > 15/7
8 2.142857142857143
9 >15//7
10 2
```

Oppg. 2.4.3: Regn ut disse med Python

a) $3 \cdot 5 + \frac{4}{5}$

b) $2^3 + 3^2 + \frac{1.25}{5}$

c) Hva resultatet til $18\%4$ blir?

Oppg. 2.4.4: Løs disse oppgavene v.h.a. Python

a) Regn ut arealet til en trekant med høyden 5 cm og grunnlinjen 8 cm.

b) Regn ut arealet til et parallelogram som har høyden 7 cm og grunnlinjen 10 cm.

Arealet til et parallelogram er altså lik lengden (grunnlinjen) multiplisert med høyden.

c) Regn ut arealet til en trapes som har høyden 6 cm og lengden til de to parallelle sidene er 8 cm og 12 cm.

Arealet av et trapes regnes ut ved å multiplisere høyden og den gjennomsnittlige lengden av de to parallelle sidene.

Å hente π fra numby-biblioteket

Eksempel 2.4.2: Vi ønsker å bestemme arealet til en sirkel med radius 5 cm.

Her har vi to muligheter Vi kan regne direkte med π som verdien 3.14, og regne direkte på kommandolinjen:

```
1 > 3.14*5**2
2 78.5
```

eller vi kan definere pi som 3.14 og regne ut arealet et skript:

```
1 pi=3.14
2 r=5
3 Areal = pi*r**2
4 print(Areal)
```

Ved å kjøre programkoden får man 78.5.

Vi lærer senere at vi kan importere pi fra *math*- eller *numpy*-biblioteket, der man enten installer hele *math*- eller *numpy*-biblioteket eller bare installerer det man trenger. Du kan bruke *numpy*-bibliotekets *numpy.pi* eller *math*.bibliotekets for *math.pi* og få verdien av den matematiske konstanten π i python.

```
1 import numpy
2 r=5
3 Areal = numpy.pi*r**2
4 print(Areal)
```

```
1 import numpy as np
2
3 r=5
4 Areal = np.pi*r**2
5 print(Areal)
```

Hvis man skal regne ut med kjente verdier som π eller kjente funksjoner som cos, kan man enten installere hele *numpy*-biblioteket:

```
1 from numpy import *
2 r=5
3 Areal = pi*r**2
4 print(Areal)
```

eller bare innstallere det man trenger:

```
1 from numpy import pi
2
3 r=5
4 Areal = pi*r**2
5 print(Areal)
```

Hvis man importerer fra numpy bare det man trenger, kan bare bruke dem uten å bruke np. Tilsvarende gjelder dette også for funksjoner som vises i det neste eksempelet.

2.5 Datatyper

En datatype forteller kompilatoren (tolkeprogrammet) hvilken datatype som behandles, og hvilke regler som skal brukes for hvordan dataene skal behandles. Her har vi fokus bare på enkle datatyper, datatyper som bare inneholder 'en verdi. De mest brukte er:

- int - integer (heltall som 1,2,3)
- float - desimaltall (eks 2.5 - OBS: her må man bruke punktum og ikke komma)
- str (string) - tekst, OBS: skrives mellom anførselstegn
- bool - boolean (True/False - OBS: når man skal gi en variabel verdien True eller False, må de skrives med stor forbokstav)

I tillegg har vi noen sammensatte datatyper, hvor hver datatype kan inneholde flere verdier, f.eks.

- list - lister
- tuple - tupler/tuppel (ligner på lister, men er en ikke-muterbar sekvens - den kan ikke endres og brukes først og fremst på data som ikke skal endres, slik som ukedager og måneder)
- set - mengder
- dict - dictionary (oppslagsliste)

Syntaks	Utfører	Eksempel
str	streng (tekst)	'matematikk'
int	heltall	37
float	desimaltall	3.14
list	liste	['k', 1, 2, 'A']
set	mengde	{1, 3, 5}
bool	boolsk verdi	True, False
np.array	np-array-datastruktur	array([1, 2])

Eksempel 2.5.1: Her vises et eksempel, hvordan strenger oppfører seg med matematiske operasjoner:

```
1 print('tre' + 'ti')
2 print('to' * 3)
```

Utskriften blir da:

```
1 treti
2 tototo
```

2.6 Variabler

Som i matematikk brukes variabler i programmering. Variabler brukes til å huske data til bruk senere i programmet, for eksempel til utregninger, beslutninger og for å gjøre endringer på informasjon. Variabler kan innta verdier eller tekst, og for tekst skal ha med apostrof ('... ') eller anførselstegn ("...") rundt seg. Man kan opprette en variabel i et program ved å skrive en bokstav eller en tekst ettefulgt av et liketstegn og

- verdien som variabelen skal innta hvis variabelen skal være *et tall*
- apostrof ('... ') eller anførselstegn ("...") rundt seg (åpnes og avsluttes) hvis variabel skal være *tekst/streng*.

Som vist nedenfor her:

```
1 x= 2
2 y= 'Tekst'
3 z= "Tekst"
```

I løkkene blir ofte variablene oppdatert ved å legge til et tall til variabelen eller multiplisere variabelen med et tall, for eksempel:

Eksempel 2.6.1: Her settes variabelen n til verdien 2 og blir deretter oppdatert:

```
1 n = 2
2 # det legges til 4
3 n = n+4
4 # Resultatet ganges med 5
5 n=5*n
6 # Resultatet legges til 4
7 n += 4
8 # Resultatet ganges med 5
9 n* = 5
10
11 print('n blir da',n)
```

Her blir verdien til variabelen n satt sammen med 4 og ganget med 5 på to ulike måter, før resultatet blir multiplisert med 5 og blir vist. Svaret er da $((2 + 4) * 5 + 4) * 5 = 170$. $\diamond$

Oppg. 2.6.1: Skriv en programkode som utfører følgende regneoperasjoner ved hjelp av variabler:

- Skriv en programkode som leser inn to tall m og n som tar verdiene 7 og 1 henholdsvis, deretter blir m summert med 3 og n ganges med 5, og til slutt multipliseres m og n sammen og resultatet blir skrevet ut på skjermen som: *Produktet er* og resultatet.
- Skriv en programkode som leser inn to tall a og b som tar tallet 5 og teksten *TALL*, deretter multipliseres a og b med henholdsvis 2 og 3 og de oppdaterte variabelene a og b blir vist på skjermen

Oppg. 2.6.2: Skriv en programkode som beregner volum og overflateareal til en rektangulær rett prisme med lengde 5 meter, bredde 4 meter og høyde 3 meter.

Oppg. 2.6.3: Skrive en programkode som regner ut $y = h - \frac{1}{2}gt^2 + v_0t$ for $h = 3.5$, $g = 9.81$, $v_0 = 20$ og $t = 5$, og skriver på skjermen.

Avrunding

F-literalene bruker vi når vi vil runde av et tall i en *print-kommando*. Rett før høyre mengdeparentes legger vi inn et kolon. Da kan vi styre hvordan utskriften formateres. Vi runder av til to desimaler med formatdefinisjonen «.3f». Her betyr bokstaven «f» at variabelen skal skrives ut med et fiksert antall desimaler, og tallet betyr at det skal være tre desimaler. Hvis du i stedet skriver «.2f», får du to desimaler.

Eksempel 2.6.2: Her er et eksempel, hvordan to tall og deres sum blir avrundet:

```

1 a = 25/7
2 b = 32
3 print(f"Her er a {a:.3f} og b {b:.1f}. Summen er {a+b:.3f}.")

```

Det er vanlig at resultatet fra programmet som skrives til skjermen, inneholder både tekst og verdier av variabler. I programmet bruker vi det som kalles f-literal. Vi setter en «f» rett etter venstreparentesen for å fortelle at det kommer en f-literal. Alt som står inni mengdeparenteser i en f-literal, blir tolket som Python-kode. Det vil si at a blir tolket som verdien av variabelen a.

2.7 Å sende inn datasett i Python-program

1. I dataprogram kan variablene deklarerer med å definere variablene som får inndelt en verdi:

```

1 r=2
2 n=5

```

2. Et dataprogram kan ta imot data fra brukeren hver gang man prøver å kjøre programmet:

```

1 alder=input("Skriv inn alderen din:")
2 print(f''Du er {alder} år.'')

```

Her kunne man angi alderen som tall eller tekst. Hvis man ønsker seg å presisere hvilken datatype (heltall, flyttall og tekst), vil man ta i bruk videre i programmet følgende:

```

1 alder=int(input("Skriv inn alderen din:"))
2 r = float(input("Skriv inn radiusen til sirkelen (et reelt tall) her: "))
3 n = int(input("Skriv inn a (et heltall) her: "))

```

3. Når vi skal behandle datafil i Python, er det første vi må gjøre å laste ned en fil med dataene og lagre den i et hensiktsmessig format. De vanlige formatene er .txt og .csv. Når vi laster ned filer med stort tallmateriale, er det ofte lurt å bruke .csv (se avsnitt 0.14).

I neste eksempel vises hvordan `float(input())` og `float(input())` for å lese inn takk og hvordan `print(f("tekst og {tall}"))` kan brukes for å formatere det som skrives ut på skjermen.

Formatering av tekst og tall

Her tas et eksempel som viser hvordan man kan lese inn hele og reelle tall fra brukeren, og skriver ut på skjermen, og hvordan man kan avrunde og formatere i utskriften:

```

1 print('Her leses inn variablene')
2 a = int(input("Skriv inn a (et heltall) her: "))
3 b = float(input("Skriv inn b (et reelt tall) her: "))
4
5 c= a+b
6 print()
7 print(f'Summen til {a} og {b} er: {c} ')
8 print() # lager en linje mellomrom
9 print(f'Summen til {a} og {b} er:c=', c)
10 print()
11 print('formatering:')
12 print(f'Her er a {a:.2f} og b {b:.1f}. Summen er {c:.3f}.')
13 print(f'Summen til {a} og {b} er: {c:.3e} ')

```

Ved å kjøre programmet får vi da på skjermen

```

1 # Her leses inn variablene
2
3 Skriv inn a (et heltall) her: 11
4 Skriv inn b (et reelt tall) her: 2.35
5
6 Summen til 11 og 2.35 er: 13.35

```

```
7 Summen til 11 og 2.35 er: c =13.35
8
9 formatering:
10 Her er a 11.00 og b 2.4. Summen er 13.350.
11 Summen til 11 og 2.35 er: 1.335e+01
```

Hvis man skulle lese tekst fra brukeren, kunne man bruke `str(input())`.

Oppg. 2.7.1: Skrive en programkode som oppretter tre variabler a , b og c der a blir gitt som input av brukeren, og $b = 3$ og $c = 5$. Programmet skal regne ut $d = \frac{1}{3}ab + c^2$, og skrive resultatet på skjermen.

Oppg. 2.7.2: Skrive en programkode som oppretter to variabler r og h som blir gitt som input av brukeren (radius og høyden til en rett sylinder), og som regner ut $V = \pi r^2 h$ (volumet til sylindere, $\pi = 3.14$), og skriver resultatet på skjermen.

2.8 Importering av pakker (numpy, math, pylab, sympy)

En pakke eller et bibliotek i Python er en samling av moduler, hvor en modul er en fil som inneholder Python-definisjoner (funksjonsdefinisjoner), men inneholder ikke kall til. Modul kan være en en samling av funksjoner og klasser som kan gjenbrukes i mange ulike programmer. Når man bruker en modul fra en pakke, skrives: `pakke.modul`, henter modulen `modul` fra pakken `pakke`. Det finnes mange ulike pakker man kan benytte i Python, og vi skal hovedsakelig bruke pakkene `numpy` (numerisk Python) og `matplotlib` her.

Disse to kommandoene for å importere hele pakken er ekvivalente:

```
1 import numpy
```

```
1 from numpy import *
```

Eller man kan bare importere det man trenger:

```
1 from numpy import pi
```

Noen foretrekker å bruke:

```
1 import numpy as np
```

der for eksempel for å bruke π skriver man i programmet `np.pi`.

Under følger et kort eksempel der vi importerer noen pakker, og bruker dem til en liten beregning av $y(x)$ for en gitt x -verdi.

Eksempel 2.8.1: Vi ønsker å beregne $y(x) = e^{-\pi x}$ for $x = 0.2$.

```
1 import numpy as np
2 x= 0.2
3 y = np.exp(-np.pi*x)
4
5 print('y er lik',y)
```

Eksempel 2.8.2: Man kan importere hele pakken eller deler av pakken:

```
1 import numpy as np          # Importerer hele pakken som får navnet np
2 # Vi vil bruke pakkens cosinus- og pi-funksjoner:
3
4 print("sinus av ",np.pi," er ", np.sin(np.pi))
5 print() # oppretter ekstra linje
6
7 # Vi kan ogsaa bare importere enkeltfunksjoner. Da kan vi kalle paa
  funksjonene uten aa angi pakkens navn.
8
9 from numpy import sin, pi    # importerer vi bare delene vi trenger.
10
11 print("sinus av ",pi," er ", sin(pi))    # Merk at vi ikke trenger pakkens
  adresse.
```

Oppg. 2.8.1: Skrive en programkode som regner ut $y = \sin(\pi x)$ for $x = 0.5$.

Eksempel 2.8.3: Vi vil regne ut $\sin(1) + \cos(4) + e^{-2}$.

```
1 import numpy as np
2 svar=np.sin(1)+np.cos(4)+np.exp(-2)
3 print(svar)
1 from numpy import exp, sin, cos
2 svar=sin(1)+cos(4)+exp(4)
3 print(svar)
```

numpy eller *math*-biblioteket?

numpy-biblioteket inneholder flerdimensjonale array- og matrise-datastrukturer, mens *math*-biblioteket brukes til å utføre matematikk på skalardata, for eksempel trigonometriske beregninger. Hvis man skal regne ut med kjente funksjoner som $\cos$, kan man enten importere hele *numpy*-biblioteket eller bare importere det man trenger:

```
1 from numpy import exp, sin, cos, tan, arcsin, arccos, arctan, exp
2 svar=sin(1)+cos(4)+exp(4)
3 print(svar)
```

Husk at de trigonometriske funksjonene ($\sin$, $\cos$ og $\tan$) tar inn en vinkel i radianer, og inversfunksjonene ($\arcsin$, $\arccos$ og $\arctan$) returnerer en vinkel i radianer.

Oppg. 2.8.2: Regn ut disse v.h.a Python

- a) $\cos(2) + \tan(1.5)$
- b) $e^4 + \log(2) + \ln(0.1)$

pylab-biblioteket

I de nye matematikk-lærebøkene for videregående-trinnet brukes *pylab*-biblioteket, som har mange innbygde funksjoner. *pylab*-pakken inneholder både *numpy* og *matplotlib*.

```
1 from pylab import *
2 print(cos(1))
```

Noen foretrekker *numpy* og at man bruker

```
1 import numpy as np
```

der man for eksempel for å bruke $\sin$ -funksjonen skal skrive *np.sin(x)*. Fordi man har da et bedre oversikt hvor i programmet har brukt pakken.

Oppg. 2.8.3: Skrive en programkode som regner ut $y = \sin(\pi x)$ for $x = 0.5$.

numpy- eller *sympy*-biblioteket?

numpy-biblioteket gjør beregningsmatematikk, mens *sympy*-biblioteket gjør symbolsk matematikk. For eksempel for å derivere eller å integrere:

```
1 from sympy import *
2
3 x = Symbol('x')
4 svar1= diff(x**3, x)
5 svar2= integrate(svar1, x)
6 print(svar2)
```

Andre pakker (*Matplotlib*)

Vi vil ofte ha behov for å visualisere data. Da kan biblioteket *Matplotlib* hjelpe oss. Her skal vi bruke modulen *pyplot* (som er en del av biblioteket *Matplotlib*) for å tegne en graf:

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 x = np.linspace(-3,3, num = 101)
5 f = x**3-3*x
6
7 plt.plot(x,f, '--')
8 plt.grid()
9 plt.xlabel('x')
10 plt.ylabel('y')
11 plt.legend(['f(x)'])
12 plt.savefig('MinPythonFigMappe/eksplot.png')
```

2.9 Indeksering

Vi lager indeksmengder med kommandoen `range`. Kommandoen `range(2, 5)` betyr heltallene fra og med 2 til, men ikke med, 5 (til og med 4).

<code>range(1,n+1)</code>	alle hele tall fra og med 1 til n
<code>range(1,n+1,m)</code>	alle hele tall fra og med 1 til n med m steglengde
<code>range(1,n+1,m)</code>	alle hele tall fra og med 1 til n med m steglengde

Hvis du utelater første argument i `range`, antar Python at første argument er 0. Dersom du legger til et tredje argument, angir dette steglengden i intervallet.

Kommando	Verdi
<code>range(0, 5)</code>	0, 1, 2, 3, 4
<code>range(5)</code>	0, 1, 2, 3, 4
<code>range(2, 5)</code>	2, 3, 4
<code>range(0, 5, 2)</code>	0, 2, 4
<code>range(10, 5, -1)</code>	10, 9, 8, 7, 6

2.10 Løkker (betingelser og gjentagelser)

En løkke brukes for beregninger eller handlinger som skal gjentas flere ganger. Man kan velge et gitt antall ganger eller at det skal gjentas inntil et mål (noe) er nådd. Fordelen med å bruke en løkke er at vi slipper å skrive mange setninger med programkode. Da blir koden enklere å feilsøke og rette, men den blir også mer kompakt, noe som gjør at det tar kortere tid å kjøre gjennom programmet.

if/elif/else-løkke

En if/elif/else-setning brukes når vi ønsker å utføre en blokk med kode når den gitte tilstanden er sant eller usant. Her er et eksempel:

```
1 a = float(input("Skriv inn a (et reelt tall) her: "))
2 if a > 2:
3     print("a er mer enn 2")
4
5 elif a < 2:
6     print("a er mindre enn 2")
7
8 else:
9     print("a er lik 2")
```

Ved å kjøre programmet og taste inn forskjellige verdier for a får vi:

```
1 Skriv inn a (et reelt tall) her: 4
2 a er mer enn 2
3 Skriv inn a (et reelt tall) her: 1
4 a er mindre enn 2
5 Skriv inn a (et reelt tall) her: 2
6 a er lik 2
```

Oppg. 2.10.1: Hvis -7 skrives inn når følgende programkoden kjøres, hva blir resultatet?

```
1 print('Skriv et tall')
2 a = int(input())
3 x=a
4 if x < 0:
5     x = -x
6 print('Absluttverdien til',a,' er', x)
```

Oppg. 2.10.2: Skriv en programkode for følgende:

- Les et tall fra skjermen, og sjekk om tallet er positivt, negativt eller null, og skriv resultatet på skjermen.
- Les et tall fra skjermen, og sjekk om tallet er partall eller oddetall, og skriv resultatet på skjermen.
- Les to tall fra skjermen og si om disse tallene har samme eller motsatt fortegn.

while-løkke

While-løkker brukes i tilfeller hvor man kan si at løkka skal utføres så lenge noe er sant.

```
1 i=0
2 while i < 3:
3     print("i =", i)
4     i += 1
```

Ved å kjøre programmet får man:

```
1 i = 0
2 i = 1
3 i = 2
```

Andre ganger vil vi gjøre bestemte utregninger så lenge en betingelse vi har satt på forhånd, er oppfylt. Vi skal vise bruken av denne type løkke ved hjelp av et eksempel. Legg merke til at $\text{range}(1, n+1)$ er alle hele tall fra og med 1 til n .

Eksempel 2.10.1: Skriv et program som regner ut $a_2, a_3, \dots, a_{10}$ når: $a_1 = 1$ og $a_{n+1} = 0.25a_n + 2$. Hva konvergerer a_n når $n \rightarrow \infty$?

Løsning:

```
1 n = 1
2 for i in range(1,10):
3     0.25*n + 2
4     print(n)
```

Når man kjører programmet for de første 10 leddene, ser man at $a_n \simeq 2.666\dots$ når $n \rightarrow \infty$. $\diamond$

Eksempel 2.10.2: Skriv et program som regner ut $a_2, a_3, \dots, a_{10}$ når: $a_1 = 1$ og $a_{n+1} = \frac{3}{4 - a_n}$. Hva konvergerer a_n når $n \rightarrow \infty$?

Løsning:

```
1 n = 1
2 for i in range(1,10):
3     n = 3/(4-n)
4     print(n)
```

Når man kjører programmet for de første 10 leddene, ser man at $a_n \simeq 1.0$ når $n \rightarrow \infty$. $\diamond$

Eksempel 2.10.3: Betrakt summen $\sum_{n=1}^{\infty} \frac{5}{n}$.

Finn ut hvor mange ledd som trengs for å få summen over 20.

Løsning:

```
1 n = 1 # Definerer n og Sum ved start
2 Sum = 0
3 # Vil regne ut summeringen til summen er mindre eller lik 20
4 while Sum <= 15:
5     Sum += 5/n
6     n += 1
7     print('Summen passerer 15 etter', n, 'ledd. Da er summen', Sum)
```

Ved å kjøre programmet får man:

```
1 Summen passerer 15 etter 32 ledd. Da er summen 20.136225977182594
```

Oppg. 2.10.3: Skriv kort hva følgende programkode utfører:

```
1 innlest = input("Hva er 49+35?")
2 tall = int(innlest)
3
4 if tall != 84:
```

```

5 while tall != 84:
6     innlest = input("Regn igjen!")
7     tall = int(innlest)
8 else:
9     print("Du klarte det!")

```

Oppg. 2.10.4: Skriv en programkode som spør hva summen til $52+19$ er, og leser inn svaret fra brukeren, og hvis svaret er feil, skriver ut *Feil. Regn igjen!*, og hvis det er rett, skriver ut *Du klarte det!*

Oppg. 2.10.5: Følgende programkode spør leseren om å skrive et partall, og deretter sjekker om dette tallet er partall, og bestemmer summen til alle partall til og med dette tallet.

```

1 tall = int(input("Skriv inn et partall: "))
2
3 if tall%2 == 1:
4     print("Tallet er ikke et partall!")
5 else:
6     sum = 0
7     i=0
8     while(i <= tall):
9         sum = sum + i
10        i = i+2
11
12 print(sum)

```

Skriv en programkode som gjør dette for oddetall.

Oppg. 2.10.6: Vi vil prøve å finne det minste heltallet x (heltall) for at $3^x > 81$.

Følgende program vil telle hvor mange ganger, må gange 3 med seg selv for å få mer enn 81.

```

1 x0= 1
2 b = 81
3 x = 0
4 x1 = x0
5
6 while x1 <= b:
7     x += 1
8     x1*= 3
9
10 print(f"Svaret er {x}.")

```

Skriv programmet og sjekk om du får svaret 4.

Modifiser programmet for å gjøre dette for ligningen $4^x = 1024$

for-løkke

for-løkken kan brukes til å gjenta en kodeblokk et kjent/begrenset antall ganger, eller gå gjennom en sekvens, element for element:

```
1 n=2
2 for i in range(1,n+1):
3     b=i**2+1
4     print(b)
```

Ved å kjøre programmet får man:

```
1 2
2 5
```

Legg merke til at $\text{range}(1, n+1)$ betyr indeksmengde fra og med 1 til og med n (og ikke til og med $n+1$).

Oppg. 2.10.7: Skriv hva resultatet av følgende programkode blir:

```
a) i=0
2 for i in range(2):
3     print('A')
4     for i in range(3):
5         print('B')
```

```
b) for x in [50, 60]:
2   for y in [1, 2, 3]:
3       print(x+y, end=' ')
```

```
c) for x in [50, 60]:
2   print()
3   for y in [1, 2, 3]:
4       print(x+y, end=' ')
```

Oppg. 2.10.8: Hva får man av følgende programkoder? Skriv følgende i et script og kjør programmet.

```
a) for i in range(0,10):
2   print(i, end=" ")
```

```
b) for i in range(1,10):
2   print(i, end=" ")
```

```
c) for i in range(0,10,2):
2   print(i, end=" ")
```

```
d) for i in range(10,1,-1):
2   print(i, end=" ")
```

Oppg. 2.10.9: Skriv en programkode for følgende: Det dukker på skjermen: *Hei, vil du fortsette med programmet?* Hvis svaret er nei, avsluttes programmet.

Oppg. 2.10.10: Skriv en programkode som regner ut summen til de første 5 leddene i en geometisk rekke med førsteledd 3 og kvotient 2, og skriver ut på skjermen summen.

Oppg. 2.10.11: Skriv kort hva følgende program utfører:

```
1 for i in range(1,10):
2     for k in range(1,5):
3         print(i,k)
```

Oppg. 2.10.12: Skriv en programkode som regner ut summen til de første 7 leddene i en aritmetisk rekke med førsteledd 5 og kvotient 3, og skriver ut summen på skjermen.

Oppg. 2.10.13: a) Skriv hva resultatet til følgende programkode blir:

```
1 for i in range(1,2):
2     for k in range (1,3):
3         print(i,2*k)
```

b) Skriv også hva `range(5)` sender inn i løkken.
Skriv hva resultatet til programkoden blir

```
1 for i in range(5):
2     if i == 3:
3         print(i, end=" ")
4     else:
5         print(2*i, end=" ")
```

c) Skriv også hva `while i < 4` sender inn i løkken.

```
1 i = 1
2 while i < 4:
3     print(i)
4     if i == 3:
5         break
6     i += 1
```

Hvordan ville du endre løkken i del c) hvis du ville bruke `range`-syntaksen?

d) `= 1`

```
2 while i < 5:
3     print(i)
4     i += 1
5 else:
6     print("i er ikke lenger mindre enn 5")
```

e) Hva skriver følgende programkode ut?

```
1 r = 4
2 for i in range(1, r + 1, 1):
3     for j in range(1, i + 1):
4         print(j, end=' ')
5     print()
```

f) Hva skriver følgende programkode ut?

```
1 n=4;
2 for i in range(n):
3     for j in range(i):
4         print ('o ', end=" ")
5     print(' ')
6
7 for i in range(n,0,-1):
8     for j in range(i):
9         print ('* ', end=" ")
10    print(' ')
```

g) Skriv en programkode som får følgende resultat:

```
1 *
2 * *
3 * * *
4 o o o o
5 o o o
6 o o
7 o
```

Oppg. 2.10.14: Følgende programkode

```
1 n = 4
2 k = 4
3 for i in range(0,n+1):
4     for j in range(k-i,0,-1):
5         print(j,end=' ')
6     print()
```

Skriver ut:

```
1 4 3 2 1
2 3 2 1
3 2 1
4 1
```

a) Gjør de nødvendige endringene i programmet for at svaret blir:

```
1 1
2 1 2
3 1 2 3
4 1 2 3 4
```

b) Gjør de nødvendige endringene i programmet for at svaret blir:

```
1 4 3 2 1
2 3 2 1
3 2 1
4 1
```

c) Gjør de nødvendige endringene i programmet for at svaret blir:

```
1 1
2 2 1
3 3 2 1
4 4 3 2 1
```

d) Gjør de nødvendige endringene i programmet for at svaret blir:

```
1 1
2 1 2
3 1 2 3
4 1 2 3 4
5 1 2 3 4 5
```

2.11 Funksjoner

Funksjoner brukes til å dele opp programmer i mindre deler. I programmering er funksjoner en måte å samle en beregning utført av en eller flere variabler i en sammensatt bolk, såkalte *funksjoner*, slik at det blir lettere å skrive, modifisere, lese koden, bruke andre steder i programmet eller eventuelt i andre program. Syntaksen for funksjoner er kontrollordet *def*, deretter navnet på funksjonen etterfulgt av to parenteser som inneholder variabel (variabler), som inngår funksjonen og et kolon (:).

```
1 def funksjonsnavn(variabler):
2     kodeblokk
```

Husk å trykke tab (knappen rett over caps lock) for linjene i kodeblokken for å få innrykk. Funksjoner gjør programmeringen lettere.

Litt om funksjoner: Hva og hvorfor?

Funksjoner i Python (**Hva?**):

- ◇ består av en gruppe kodelinjer som utfører en spesifikk oppgave
- ◇ kan minne om matematiske funksjoner, men også Python-funksjoner
 - kan inneholde alle slags handlinger, ikke bare matematikk
 - kan returnere et svar (som `mat.funk.`), men kan også la være
 - kan ta argumenter, men kan også la være

Hvorfor bruke funksjoner?

- Dele et større program opp i mindre deler
 - lettere å lese, skrive og vedlikeholde kode
 - Lettere å forstå, enklere å teste, mer effektivt samarbeid
 - gir ryddigere og enklere kode
- Gjenbruk av kode
 - Vi kan bruke Pythons egne funksjoner, f.eks. `print()`, `round()`
 - Vi kan selv definere nye funksjoner og
 - * bruke dem flere steder i ett program,
 - * eller i flere ulike programmer

Her tar vi et eksempel på hvordan vi kan lage en funksjon som kan brukes flere ganger i programkoden. For eksempel kan vi skrive en funksjon som kan regne ut volumet av en sylinder med radius r og høyde h .

```
1 import numpy as np
2
3 def Volumsyylinder(r, h):
4
5     volum=np.pi*(r**2)*h
6     return volum
7
8 V=Volumsyylinder(3,5)
9 print(V)
```

I scriptet regner vi ut `Volumsyylinder(3,5)` volumet av sylindren med radius = 3 og høyde = 5. Funksjoner begynner med kontrollordet `def`, etterfulgt av navnet på funksjonen, parenteser og et kolon, der input-parameterne er gitt i parentes. Dersom du vil at funksjonen skal kunne ta ett eller flere argumenter, skriver du dem inn i parentesene. I dette eksemplet har vi deklartert variablene r (radius) og h (høyde), og det betyr at disse variablene inngår i funksjonen og de må defineres som et tall når funksjon skal regne ut volumet.

Legg merke til at etter funksjonshodet `def Volumsyylinder(r, h):` kommer resten av koden i linjene under med innrykk (bruke TAB-tasten). Så utføres utregningen av volumet, før vi til slutt har med `return volum` som skal returnere svaret funksjonen gir, og vi kan bruke videre i koden. Istedenfor `Volumsyylinder(3,5)`, kunne vi bruke:

```
1 a=3
2 b=5
3 V=Volumsyylinder(a,b)
4 print(V)
```

Ellers variablene kan leses inn i programmet fra skjermen:

```
1 r= float(input('Skriv inn radiusen til en sylinder: '))
2 h= float(input('Skriv inn høyden til en sylinder: '))
3 V=Volumsyylinder(r,h)
```

Eksempel 2.11.1: Her er 3 eksempler med funksjoner uten argument, med argumenter, og med matematiske beregninger og hvordan disse kan brukes (*kalles*).

```
1 def minfunk():
2     print("Her er en funksjon!")
3
4 def minfunkmedarg(Navn, Hilsen):
5     print("Hei %s, %s" %(Navn,Hilsen))
6
7 def minfunktall(x,y):
8     return x**2 + y**2
9
10 minfunk()
11
12 minfunkmedarg("Kari Olsen","Ha en fin dag!")
13
14 a = minfunktall(1,2)
15 print(a)
```

Ved å kjøre programkoden får vi:

```
1 Her er en funksjon!
2 Hei Kari Olsen, Ha en fin dag!
3 5
```

Eksempel 2.11.2: For eksempel setter følgende program opp funksjonen $f(x) = x^3 + 3x$ og lister opp på skjermen x og $f(x)$ under x og $f(x)$:

```
1 def f(x):
2     return x**3 + 3*x
3
4 print(f'x\t f(x)')
5
6 for x in range(10):
7     print(f'{x}\t {f(x)}')
```

Ved å kjøre programmet, ser resultatet slik ut:

```
1 x  f(x)
2 0  0
3 1  4
4 2  14
5 3  36
6 4  76
7 5  140
8 6  234
9 7  364
10 8  536
11 9  756
```

Eksempel 2.11.3: Her tas et eksempel med en funksjon som beregner skalarproduktet til to vektorer:

```
1 def skalarprod(a, b):
2     p= 0
3     for i in range(len(a)):
4         p += a[i]*b[i]
5     return p
6 # Prøv funksjonen:
7 a = [1, 2, 3]
8 b = [0, 3, -1]
9 ab = skalarprod(a,b)
10 print(f'Skalarproduktet av a og b er {ab:.2f}')
```

Ved å kjøre programmet, får man:

```
1 Skalarproduktet av a og b er 3.00
```

Oppg. 2.11.1: Hva får man ved å kjøre følgende program:

```
1 def skalarprod(m, n):
2     d= 0
3     for i in range(len(m)):
4         d += m[i]*n[i]
5     return d
6 # Prøv funksjonen:
7 m = [-1, 3, 0]
8 n=  [2, 1, 5]
9 mn = skalarprod(m,n)
10 print(f'Skalarproduktet av m og n er {mn:.1f}.')
```

Oppg. 2.11.2: a) Skriv et program som setter opp en funksjon for arealet til en sirkel med radius r og regn ut arealet til en sirkel med radius 5.

b) Skriv et program som leser inn radiusen til en sirkel fra brukeren og regner og skriver ut arealet til en sirkel.

Oppg. 2.11.3: Summene nedenfor beregner $\ln x$ når n velges stor nok.

$$s_1(x, n) = \sum_{i=1}^n (-1)^{i-1} \frac{1}{i} (x-1)^i, \quad 0 < x \leq 2$$

$$s_2(x, n) = \sum_{i=1}^n \frac{1}{i} \left(1 - \frac{1}{x}\right)^i, \quad x > \frac{1}{2}$$

Vi ønsker å lage en programkode som hjelper til å beregne disse summene for en gitt x og n . Vi vil studere estimeringsavviket (feilen) man får ved å bruke summen $s(x, n)$ for en gitt x , istedenfor $\ln x$. Estimeringsfeilen for s_1 kan studeres ved to forslag:

- 1) Å bestemme feilen ved å regne ut differansen mellom den eksakte verdien og den estimerte verdien: $\ln(1.5) - s(1.5, n)$ (naiv metode).
- 2) Å Regne ut det først leddet i summen s_1 som ikke er tatt med i beregningen av summen, som kan kalles det numeriske estimeringsavviket. To programkoder er skrevet slik:
For s_2 kan man se nærmere på 1).

```

1 # Programkode 1
2 import numpy as np
3
4 def S1(x, n):
5     S1 = 0
6     for i in range(1, n+1):
7         S1 += (-1)**(i-1) * (x-1)**i / i
8     return S1
9
10 def E1(x, i):
11     E1 = 0
12     E1 = (x-1)**i / i
13     return E1
14
15 x = 1.5
16 n=100
17 e=0.000001
18
19 for i in range(1, n+1):
20     EstimertLog1 = S1(x, i)
21     Estimeringsfeil1 = E1(x, i)
22     EksaktLog = np.log(x)
23     NaivEstimeringsfeil = abs(EstimertLog1 - EksaktLog)
24     if NaivEstimeringsfeil < e:
25         break
26
27 print(f' Ved å bruke {i} første ledd: Estimert= {EstimertLog1}, Eksaktverdi= {
    EksaktLog}, Naiv estimeringsfeil = {NaivEstimeringsfeil}, Estimeringsfeil = {
    Estimeringsfeil1}') )

```

```

1 # programkode 2
2 import numpy as np
3
4 def S2(x, n):
5     S2 = 0
6     for i in range(1, n+1):
7         S2 += (1-1/x)**i / i
8     return S2
9
10 x = 1.5
11 n=100
12 e=0.000000001
13
14 for i in range(1, n+1):
15     EstimertLog2 = S2(x, i)
16     EksaktLog = np.log(x)

```

```

17     NaivEstimeringsfeil = abs(EstimertLog2 -EksaktLog )
18     if NaivEstimeringsfeil < e:
19         break
20 print(f'Ved å bruke {i} første ledd: Estimert= {EstimertLog2}, Eksaktverdi= {
    EksaktLog }, Naiv estimeringsfeil = {NaivEstimeringsfeil}')

```

Ved å kjøre programmeringskoder får man disse to visningene på skjermen (som ikke nødvendigvis er skrevet i samme rekkefølge som programkodene):

```

1 Ved å bruke 15 første ledd: Estimert= 0.4054657568451514, Eksaktverdi=
    0.4054651081081644, Naiv estimeringsfeil = 6.487369870189497e-07,
    Estimeringsfeil = 2.0345052083333333e-06

1 Ved å bruke 16 første ledd: Estimert= 0.40546510744296105, Eksaktverdi=
    0.4054651081081644, Naiv estimeringsfeil = 6.652033368581556e-10

```

- Forklar hvilken programkode som kan brukes til å beregne $\ln(0.25)$, $\ln(1.75)$ og $\ln(2.25)$?
- Hvordan kan man forbedre presisjonen og få bedre nøyaktighet?

Oppg. 2.11.4: Arealet til en trekant med sidelengdene a , b og c er gitt ved:

$$A = \sqrt{s(s-a)(s-b)(s-c)}$$

der $s = \frac{1}{2}(a+b+c)$.

Skriv en programkode som beregner arealet til en trekant for en gitt a , b og c .

Oppg. 2.11.5: Kinetisk energi for en partikkel med masse m [kg] som har hastighet v [m/s] er gitt ved:

$$E = \frac{1}{2}mv^2$$

Skriv en programkode som definerer en funksjon for kinetisk energi, regner ut den kinetiske energien for en partikkel som har massen $m = 0.0001$ [kg] og hastigheten $v = 8$ [m/s].

Å kalle inn en funksjon i en annen funksjon

Eksempel 2.11.4: Her tas et eksempel med en funksjon som finner det største tallet blant tre tall ved å kalle en funksjon i en annen funksjon

```
1 def maksavto( x, y ):
2     if x > y:
3         return x
4     return y
5 def maksavtre( x, y, z ):
6     return maksavto( x, maksavto( y, z ) )
7 print(maksavtre(2, -3, 7))
```

Eksempel 2.11.5: Her tas et eksempel med en funksjon som kalles inn i en annen funksjon. Skriv en programkode for en funksjon $f1(x)$ som multipliserer argumentet x med 4 og en annen funksjon $f2(x)$ som legger til tallet 5 til argumentet x . Utvid programkoden til å regne ut $f1(f2(2))$ og $f2(f1(2))$. Ellers formuleres dette slik også:

Gitt $f(x) = 4x$ og $g(x) = x + 5$. Skriv disse som funksjoner i programkode, og bruk programmet til å regne ut $f(g(2))$ og $g(f(2))$.

```
1 def f1(x):
2     y= 4*x
3     return y
4
5 def f2(x):
6     y=x+5
7     return y
8
9 print(f1(f2(2)))
10 print(f2(f1(2)))
```

Svaret ved å kjøre programmet blir:

```
1 28
2 13
```

Sjekker: $f1(f2(2))$, der $f2(2) = 2 + 5 = 7$ og $f1(f2(2)) = f1(7) = 4 \cdot 7 = 28$.

Sjekker $f2(f1(2))$, der $f2(f1(2)) = f2(8) = 8 + 5 = 13$. ◇

Oppg. 2.11.6: Hva skrives ut som resultat av kjøring av følgende programkode?

```
1 def printut(a):
2     b = 3*a
3     print('b = %g' % b)
4 x= 5
5 print('x = %g' % x)
6 printut(x)
```

2.12 Plotting i Python

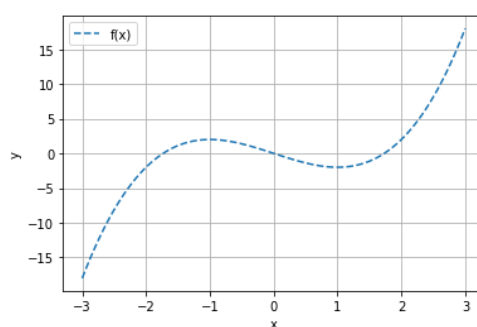
Vi vil ofte ha behov for å visualisere data. Da biblioteket *Matplotlib* hjelpe oss.

Plotting av kurver $y = f(x)$

Her skal vi bruke modulen pyplot (som er en del av biblioteket Matplotlib) for å tegne en graf:

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 x = np.linspace(-3,3, num = 101)
5 plt.yticks([-15,-5,5,15])
6 f = x**3-3*x
7
8 plt.plot(x,f, 'b--')
9 plt.xlabel('x')
10 plt.ylabel('y')
11 plt.legend(['f(x)'], loc='best')
12 plt.grid()
13 plt.show
14 plt.savefig('MinPythonFigMappe/eksplot.png')
```

Kjører vi dette scriptet, vil følgende figur plottes *og lagres*.



Elementene vi har tatt i bruk, her er følgende:

- `plt.plot(x,f, 'b--')` tegner grafen til funksjonen og tar inn x -verdiene definert i `x = np.linspace()`, de tilhørende funksjonsverdiene, og deretter `'b--'`, som skifter utseendet på grafen til en stiplet i stedet for en sammenhengende graf med blåfarge. Flere argumenter kan angis i `plt.plot()` for å spesifisere detaljene til grafen.
- Funksjonen `np.linspace(start, end, num = N+1)` deler intervallet $[start, end]$ inn i N like deler med $N + 1$ jevnt fordelte punkter.
- `plt.yticks([-15,-5,5,15])` spesifiserer tallverdiene på y -aksen.
- `plt.grid()` legger på et gitter bak grafen.
- `plt.xlabel()` og `plt.ylabel()` definerer navn på aksene.
- `plt.legend()` legger til en tekstboks for å beskrive hva grafen heter. `loc='best'`: plasserer tekstboksen der det passer best.
- `plt.show` tegner diagrammet i konsollvinduet.
- `plt.savefig()` Denne funksjonen gjør at vi kan lagre figuren direkte når vi kjører koden. Her må vi legge til riktig path inne i en streng, slik at bildet blir plassert i den mappen vi ønsker.

KONTROLL SPØRSMÅL. Hvordan kan du endre graftykkelsen og farger på grafen?
Hvordan kan du endre programkoden ved å skrive f ved hjelp av funksjonssyntaksen? ■

Oppg. 2.12.1: Følgende programkode er gitt:

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 x = np.linspace(-2,2, num = 101)
5 plt.yticks([-10,-5,5,10])
6 f = x**5-5*x**2
7
8 plt.plot(x,f, 'g--')
9 plt.xlabel('x')
10 plt.ylabel('y')
11 plt.legend(['f(x)'], loc='best')
12 plt.grid()
13 plt.show
14 plt.savefig('MinPythonFigMappe/EksGraf.png')
```

- a) Forklar hva programmet utfører linje for linje.
- b) Gjør de nødvendige endringene for at programkoden tegner grafen til: $g(x) = x^3 - 5x$ når $-2 \leq x \leq 2$.

Oppg. 2.12.2: La f være funksjonen gitt ved $f(x) = x^3 + 6x - 3$.

- a) Skriv en programkode som setter opp verditabell for x -verdiene mellom -2 og 2 (med å dele opp intervallet med steglengde = 0.01), og tegner grafen til f .
Utvid programmet til å tegne grafen til funksjonen
- b) Utvid programmet ditt til å la brukeren velge minste og største verdi for x og trinnene, i tabellen.

Oppg. 2.12.3: Skriv en programkode som tegner grafen til følgende funksjoner i samme koordinatsystem:

$$f(x) = \sin(2x - \pi)$$

$$g(x) = \cos(2x + \pi)$$

i intervallet $[-2\pi, 2\pi]$.

Skisser også $h(x) = \cos(2x + \frac{\pi}{2})$. Hva kan vi si om f og h ?

Utvid programmet for at den kan tegne grafen til funksjonene:

$$f(x) = c + a \sin\left(\frac{2\pi}{T}kx + d\right), \quad 0 \leq x \leq T$$

$$g(x) = c + a \cos\left(\frac{2\pi}{T}kx + d\right), \quad 0 \leq x \leq T$$

gitt $c = 60$, $a = 120$, $T = 365$, $k = 2$ og $d = 182.5$.

Plotting av flater $z = f(x, y)$

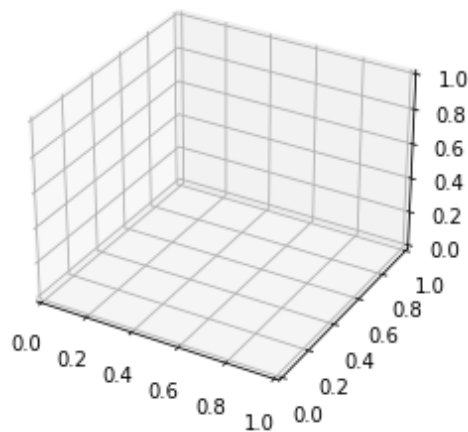
Størrelsen på gridet må tilpasses til hver situasjon. Her i dette eksemplet er brukt 101 punkter (100 oppdelinger) per akse, noe som gjør at flaten ser glatt ut.

To vanlige kommandoer for å plote flater i Python er `mesh` og `surf`. Forskjellen på dem er at `surf` fyller maskene mens `mesh` tegner kun maskenettet. Kommandoene brukes likt:

```
1 mesh(x,y,h)
2 surf(x,y,h)
```

Overflateplottet er en representasjon av tredimensjonale datasett. Den beskriver et funksjonelt forhold mellom to uavhengige variabler X og Y og en utpekt avhengig variabel Z . Det er et ledsagerplott av konturplottet. Dette bidrar til å skape topologien til overflaten som blir visualisert.

```
1 import matplotlib.pyplot as plt
2
3 fig = plt.figure()
4 ax = plt.axes(projection='3d')
```



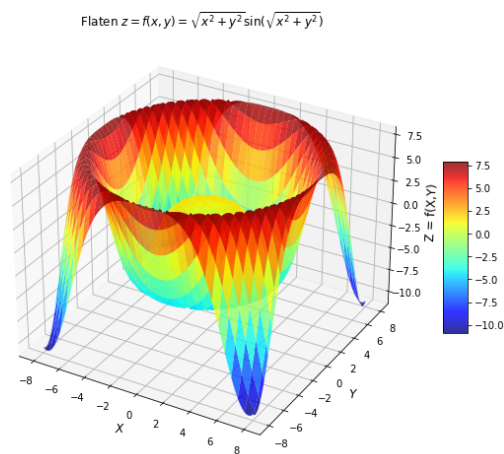
For å endre fargene som vises i `mesh` og `surf` skriv `cmap='jet'` eller istedenfor `jet`, kan man sette `hot`, `bone`, `cool`, osv.

For å se hvilke farger som representerer hvilke verdier kan man skrive `verb|colorbar()`. Til slutt kan vi pynte litt på figuren. For å angi navn på aksene bruker vi

```
1 ax.set_xlabel('$X$', fontsize=12)
2 ax.set_ylabel('$Y$', fontsize=12)
3 ax.set_zlabel('$Z = f(X,Y)$', fontsize=12)
4 # ax.set_title('Flaten $z=f(x,y)$');
5
6 # Justerer plasseringene til titel og aksene
7 ax.set_title(" " * 35 + "Flaten $z=f(x,y)$" + " " * 35)
```

Eksempel 2.12.2: Skisser grafen til $z = \sqrt{x^2 + y^2} \sin(\sqrt{x^2 + y^2})$

```
1 import matplotlib.pyplot as plt
2 import numpy as np
3
4 n=101
5 # Setter opp definisjons området til de uavhengige x og y variablene:
6 X, Y = np.meshgrid(np.linspace(-8, 8, n), np.linspace(-8, 8, n))
7
8 # Setter opp funksjonen
9
10 def f(x, y):
11     t=np.sqrt(x ** 2 + y ** 2)
12     return t*np.sin(t)
13
14 Z= f(X,Y)
15
16 # Det fungerer også med:
17 # t = np.sqrt(X**2 + Y**2)
18 # Z = t*np.sin(t)
19
20 fig = plt.figure(figsize=(10,8))
21 ax = fig.add_subplot(111, projection = '3d')
22
23 # 3D flate
24 surf=ax.plot_surface(X, Y, Z, cmap='jet', alpha=0.8)
25
26 # colorbar
27 fig.colorbar(surf, shrink=0.4, aspect=7)
28
29 # Setter opp tekst på aksene og grafen
30
31 ax.set_xlabel('$X$', fontsize=12)
32 ax.set_ylabel('$Y$', fontsize=12)
33 ax.set_zlabel('$Z = f(X,Y)$', fontsize=12)
34 # ax.set_title('Flaten $z=f(x,y)$');
35
36 # Justerer plasseringene til titel og aksene
37 ax.set_title(" " * 35 + "Flaten $z=f(x,y)$" + " " * 35)
38
39 plt.show()
```



◇

2.13 Lister

Lister er mengden av elementer atskilt med komma mellom hakeparenteser, for eksempel `[1, 2, 3]` og `['Klara', 'Augusta', 7]`. Lister kan brukes til å lagre objekter underveis i programmet. For å lage en liste, oppretter man en tom liste ved å tilordne verdien `[]` til en variabel, for eksempel `liste=[]` eller `xverdier=[]`. Her er variabelnavnene «liste» og «xverdier».

```
liste = ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h']:
```

Kommando	Resultat	Forklaring
<code>liste[0]</code>	<code>'a'</code>	Første element er nr. 0
<code>liste[0:3]</code>	<code>['a', 'b', 'c']</code>	Tar ikke med nr. 3 (fjerde)
<code>liste[:3]</code>	<code>['a', 'b', 'c']</code>	Tar med element på plass nr. 0, 1 og 2
<code>liste[-1]</code>	<code>'h'</code>	Siste element er nr. -1
<code>liste[3:6]</code>	<code>['d', 'e', 'f']</code>	Fra og med nr. 3 til nr. 6
<code>liste[3:-1]</code>	<code>['d', 'e', 'f', 'g']</code>	Fra nr. 3, tar ikke med siste element
<code>liste[3:]</code>	<code>['d', 'e', 'f', 'g', 'h']</code>	Fra nr. 3, tar ikke med siste element

For å legge til elementer i en liste, brukes kommandoen «*append*», og listevariabelen brukes som forstavelse, atskilt med et punktum. I koden nedenfor opprettes det først en tom liste i linje 1, og i de tre neste linjene legger vi inn tallene 3 og 1 og bokstaven «c». I linje 5 skriver vi lista til skjerm.

```
1 liste = []
2 liste.append(3)# legger tallet 3 inn i lista
3 liste.append(1)# legger tallet 1 inn i lista
4 liste.append('c')# legger bokstaven c inn i lista
5 print(liste)
```

Når vi kjører programmet, får vi dette resultatet:

```
1 [3, 1, 'c']
```

Kontrollordet *len* kan gi oss lengden på lista, *len(liste)*, altså hvor mange elementer den inneholder:

```
1 liste=[1,7,2,4,3,3,5,6]# en liste med sju elementer
2 print(len(liste))
```

Svaret blir da 8.

Eksempel 2.13.1: Skriv en programkode som gir `[0, 1, 'e', 'p']`.

```
1 liste = []
2 liste.append(0)
3 liste.append(1)
4 liste.append('e')
5 liste.append('p')
6 print(liste)
```

Man kan også legge til flere elementer samtidig med programkoden

```
1 liste = []
2 liste.extend([0,1,'e','p'])
```

Oppg. 2.13.1: Hva blir resultatet av følgende?

a) Programkoden:

```
1 A=[1,2,3,4,5]
2 B=[2,4,6,8,10]
3 C=A+B
4 print(C)
```

b) Programkoden:

```
1 A=[1,2,3]
2 A+= [4]
3 A = A + [5]
4 print(A)
```

- c) En liste er gitt som: Liste=[1, 2, 3, 4, 5, 6] Bruk en for-løkke til å regne ut produktet av alle elementene i liste.

Oppg. 2.13.2: La f være funksjonen gitt ved $f(x) = -0.2x^2 + 1$. Skriv en programkode som setter opp x -verdiene, mellom -1 og 2 med steglengde 0.25 , og y -verdiene til f inn i to lister.

Oppg. 2.13.3: La f være funksjonen gitt ved $f(x) = -x^3 + x + 2$. Skriv en programkode som setter opp x -verdiene, mellom -2 og 3 med steglengde 0.25 , og y -verdiene til f inn i to lister.

2.14 Python-filhåndtering

I Python er det ikke nødvendig å importere eksternt bibliotek for å lese og skrive filer. Python har en innebygd funksjon for å lage, skrive og lese filer.

Filkommandoer	Forklaring
<code>f = open('filnavn')</code>	Åpner en fil, returnerer filreferanse
<code>f = open('filnavn','tilgang')</code>	Åpner en fil, med spesifisert tilgang
	F.eks. 'w' åpner en fil for skriving (se neste side)
<code>f.read()</code>	Returnerer hele innholdet av filen
<code>f.read(n)</code>	Returnerer <i>n</i> karakterer av innholdet
<code>f.readline()</code>	Returnerer neste linje (før / n)
<code>f.readlines()</code>	Returnerer hele innholdet av fila som en liste
<code>f.write(s)</code>	Skriver strengen <i>s</i> til fil
<code>f.writelines(liste)</code>	Skriver innholdet av liste av strenger til fil
<code>f.seek(offset fra hvor)</code>	Forflytter filpekeren (index) i filen
<code>f.tell()</code>	Returnerer posisjon til filpekeren i filen
<code>f.close()</code>	Lukker filen

Tilgangstyper for open

Filkommandoer	Forklaring	Tidl. data	Hvis filen ikke finnes
'r'	Lesing	Bevares	FEIL
'w'	Skriving	Fjernes	Oppretter ny fil
'a'	Skriving (bakerst)	Bevares	Oppretter ny fil
'r+'	Lesing og skriving	Bevares	FEIL
'w+'	Lesing og skriving	Fjernes	Oppretter ny fil
'a+'	Lesing og skriving (helt bakerst)	Bevares	Oppretter ny fil

```
1 f = open('datafil.txt','r') # aapner fil for lesing
2 f = open('datafil.txt','w') # aapner fil for skriving
3 f = open(r'C:\User\files\fil.txt','a' )
4 # r først inni parentesen fo at \ ikke tolkes som spesielle tegn
5 # deretter leser filen fil.txt fra stien.
```

Eksempel 2.14.1:

```
1 f = open('fil1.txt', 'a')
2 f.write('Denne skrives i filen!')
3 print()
4 f.close()
5 print()
6 #åpen og les fra filen
7 f = open('fil1.txt', 'r')
8 print(f.read())
```

2.15 Laste inn, håndtere og plotte datasett

Vi har lært så langt hvordan vi bruke matematiske funksjoner til å regne funksjonsverdier og plotter disse. I naturvitenskapen vil ofte data / tall / målte verdier være hentet fra eksperimenter og observasjoner. Det kan ofte være store datasett med lange lister med tall. Vi skal nå hvordan vi kan laste inn data fra harddisken til vår pc eller eksterne servere og bruke innbygde Python-funksjoner og pakker for å håndtere slike lister.

pandas-biblioteket

pandas-biblioteket brukes til å analysere data.

```
1 import pandas as pd
2 df = pd.read_csv('data.csv')
```

I det neste eksempelet skal vi lese tid–temperatur-målinger, der vi bruker følgende syntakser for å åpne datafilen *tekopp-temp.csv* der i regnearket hver tekstlinje er en rad og verdiene for de ulike søylene er separert med komma (;). For å indikere dette, bruker man *sep=';'* og for å tolke desimaltall bruker man *decimal='.'*.

```
1 import pandas as pd
2 df = pd.read_csv('http://home.hvl.no/ansatte/ahas/MP/tekopp-temp.csv', sep=';',
3                 decimal='.', comment='#', encoding='latin-1')
4 tid = df['Tid'].tolist()
5 temperatur = df['Temperatur'].tolist()
```

2.16 Modellering: tekopp-temperatur

Prosjekt: Modellering av tekopp-temperatur

Vi ønsker å studere endringen i tekopp temperatur i løpet av en time (60 min).

Læringsmål

- Utforske sammenhengen mellom konstant prosentvis endring, vekstfaktor og eksponentialfunksjoner
- Bruke funksjoner i modellering og argumentere for framgangsmåter og resultater,
- Modellere situasjoner knyttet til reelle datasett, presentere resultatene og argumentere for at modellene er gyldige
- Utforske matematiske egenskaper og sammenhenger ved å bruke programmering
- Tolk og bruke funksjoner i matematisk modellering og problemløsning.

Læringsutbytte

- Studentene skal forstå hvordan kurven som beskriver at temperaturfallet avtar raskest i starten for så å nærme seg temperaturen i rommet som er en asymptote til kurven.
- Studentene skal se at lineær kurvetilpasning ikke kan dekke behovet, spesielt i de første minuttene.

Programmeringsdidaktikk

Teori (det tas litt teori på tavla før eksperimentet begynner):

Newtons avkjølingslov: dette forsøket prøver å lage en matematisk modell som beskriver hvordan temperaturen endres i et objekt (en kopp med vann) som kjøles ned av omgivelsene. Funksjonsuttrykket:

$$y = f(t) = at + b$$

representerer en lineær funksjon som kan beskrive lineær sammenheng mellom to størrelser. Funksjonsuttrykket:

$$y = f(x) = c + ae^{bt}, \quad \text{der } b < 0$$

representerer en eksponentialfunksjon. Ved start ($t=0$) har funksjonen verdien $y = c + a$, og synker eksponentiell mot $y = c$ når tiden (t) går.

Utstyr:

- 10 eller flere kopper (samme type)
- Vannkoker
- En vekt
- En klokke
- Ett eller flere termometer(e)

Tidsbruk:

Forarbeid: 15-20 min og elevene jobber ca. 60 min, 10-15 min refleksjon over detaljene i prosjektet med åpen samtale og eventuelt kontrollspørsmål.

Gjennomføring:

Kok opp tilstrekkelig vann til å fylle opp alle koppene. Sett koppene på vekten etter tur og fyll opp med like mye av det varme vannet. Mål temperaturen med jevne mellomrom i løpet av en time (3600 sek), for eksempel etter 5 min (300 sek), 10 min (600 sek), ... og noter resultatet i tabellen.

Tid (sek)	Temp (Celsius)

For eksempel (data hentet fra time-temp.pdf (southampton.ac.uk))

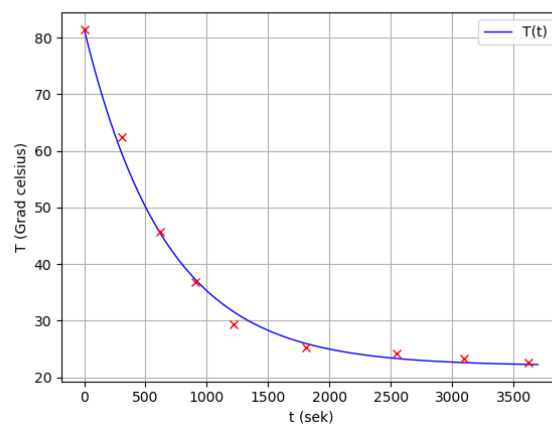
Tid (sek)	Temp (Celsius)
0	81.50
310	62.50
620	45.70
910	36.80
1220	29.40
1810	25.30
2550	24.20
3100	23.30
3620	22.50

Eksempel 2.16.1: I dette eksempelet skal vi nå se på filen ‘tekopp-temperatur.csv’. Hver linje i filen er en temperatur-måling i en tekopp i løpet av en time (3600 sekunder) som er plassert i et rom med temperatur 22 grader celsius. Her bruker vi Newtons avkjølingslov for å studere temperaturendringen i vannkoppen.

$$\frac{dT}{dt} = k(T - T_o)$$

der $T(t)$ ((målt i grader celsius) er vanntemperatur i vannkoppen ved tiden t (målt i sekunder).

```
1 import matplotlib.pyplot as plt
2 import pandas as pd
3
4 df = pd.read_csv('http://home.hvl.no/ansatte/ahas/MP/tekopp-temp.csv', sep=';',
5                  decimal='.', comment='#', encoding='latin-1')
6
7 tid = df['Tid'].tolist()
8 temperatur = df['Temperatur'].tolist()
9 tstart = 0
10 Tstart = temperatur[0]
11 tslutt = 3700
12 deltat = 0.5
13 To = 22 # Rom temperatur
14 k = .0015 # proporsjonalitets koeff.
15 tverdier = []
16 Tverdier = []
17 t = tstart
18 T = Tstart
19 while t <= tslutt:
20     tverdier.append(t)
21     Tverdier.append(T)
22     t = t + deltat
23     deltaT = -deltat * k * (T - To)
24     T = T + deltaT
25
26 plt.plot(tverdier, Tverdier, linewidth=1, color='blue', label='T(t)')
27 plt.plot(tid, temperatur, 'x', color='red')
28 plt.grid()
29 plt.xlabel('t (sek)')
30 plt.ylabel('T (Grad celsius)')
31 plt.legend(loc='best')
```



2.17 Oppsummering

Matematiske operatører

Operator	Utfører	Eksempel	Resultat
+	Addisjon	$5 + 2$	7
-	Subtraksjon	$5 - 2$	3
*	Multiplikasjon	$5 * 2$	10
/	Divisjon	$5 / 2$	2.5
//	Heltalldivisjon	$13 // 3$	4
**	Eksponent	$2 ** 3$	8
÷	Modulo	$13 \div 4$	1

Datatyper

Syntaks	Utfører	Eksempel
str	streng (tekst)	'matematikk'
int	heltall	37
float	desimaltall	3.14
list	liste	['k', 1, 2, 'A']
set	mengde	{1, 3, 5}
bool	boolsk verdi	True, False
np.array	np-array-datastruktur	array([1, 2])

Å lese input fra brukeren og utskrift av tekst og variabel

Her tas et eksempel som viser hvordan man kan lese inn hele og reelle tall fra tastaturet, og skriver ut på skjermen, og hvordan man kan avrunde og formatere i utskriften:

```
1 print('Her leses inn variablene')
2 a = int(input("Skriv inn a (et heltall) her: "))
3 b = float(input("Skriv inn b (et reelt tall) her: "))
4 alder=input("Skriv inn alderen din:")
5 print(f''Du er {alder} år.'')
```

Andre tegn og syntaks

Tegn	betydning
==	lik
!=	ikke lik
>	mer enn
>=	mer eller lik (minst)
<	mindre enn
<=	mindre eller lik (høyst)

Importerer av ulike pakker

```
1 import numpy as np
2 import matplotlib.pyplot as plt
```

Man kan enten importere hele biblioteket eller det man trenger fra biblioteket.

```
1 from numpy import *
2 from numpy import pi sin cos
```

Noen kommandoer

<code>print ()</code>	Skriv på skjerm
<code>range(1,n+1)</code>	Alle hele tall fra og med 1 til n
<code>range(1,n+1,m)</code>	Alle hele tall fra og med 1 til n med m steg lengde
<code>range(1,n+1,m)</code>	Alle hele tall fra og med 1 til n med m steg lengde
<code>if ... elif ... else</code>	Vilkår, if-test.
<code>while</code>	Løkke som kjører til testen er usann.
<code>for</code>	Løkke som kjører et på forhånd kjent antall ganger.
<code>import matplotlib.pyplot as plt</code>	Importerer selektivt pyplot-modulen til matplotlib som <i>plt</i>
<code>plt.plot()</code>	Tegner grafen til to lister.

Funksjoner

Funksjoner brukes til å dele opp programmer i mindre deler.

```
1 def funksjonsnavn(variabler):
2     kodeblokk
```

- Første linje kalles *funksjonshode*, som består av **def**, fulgt av navnet på funksjonen i parenteser og variabler, som inngår i funksjonen et kolon :
- Resten av koden kalles kodeblokk, som hører til funksjonen som må tas med.

Python-filhåndtering

I Python er det ikke nødvendig å importere eksternt bibliotek for å lese og skrive filer. Python har en innebygd funksjon for å lage, skrive og lese filer.

Filkommandoer	Forklaring
<code>f = open('filnavn')</code>	Åpner en fil, returnerer filreferanse
<code>f = open('filnavn', 'tilgang')</code>	Åpner en fil, med spesifisert tilgang
	F.eks. 'w' åpner en fil for skriving (se neste side)
<code>f.read()</code>	Returnerer hele innholdet av filen
<code>f.read(n)</code>	Returnerer n karakterer av innholdet
<code>f.readline()</code>	Returnerer neste linje (før / n)
<code>f.readlines()</code>	Returnerer hele innholdet av fila som en liste
<code>f.write(s)</code>	Skriver strengen s til fil
<code>f.writelines(liste)</code>	Skriver innholdet av liste av strenger til fil
<code>f.seek(offset fra hvor)</code>	Forflytter filpekeren (index) i filen
<code>f.tell()</code>	Returnerer posisjon til filpekeren i filen
<code>f.close()</code>	Lukker filen

Tilgangstyper for open

Filkommandoer	Forklaring	Tidl. data	Hvis filen ikke finnes
'r'	Lesing	Bevares	FEIL
'w'	Skriving	Fjernes	Oppretter ny fil
'a'	Skriving (bakerst)	Bevares	Oppretter ny fil
'r+'	Lesing og skriving	Bevares	FEIL
'w+'	Lesing og skriving	Fjernes	Oppretter ny fil
'a+'	Lesing og skriving (helt bakerst)	Bevares	Oppretter ny fil

Fasit

Fasit 0.1.1:

- a) Vi ønsker å bestemme den deriverte til funksjonen

$$f(x) = \sin\left(\frac{\pi}{2}x\right)$$

i punktet $x = \frac{1}{2}$ ved hjelp av numerisk derivasjon. Programmet nedenfor bruker *endelig differanse* (definisjonen) for å tilnærme den deriverte.

```
1 # importerer numpy pakken
2 import numpy as np
3
4 # Definerer funksjonen f(x) = sin(pi/2 * x)
5 def f(x):
6     return np.sin(np.pi/2 * x)
7
8 # Funksjon for numerisk derivasjon
9 def derivert(a, deltax):
10     return (f(a + deltax) - f(a)) / deltax
11
12 # Punktet vi ønsker å finne den deriverte i
13 a = 0.5
14 deltax = 0.0001
15
16 df = derivert(a, deltax)
17 # Skriv ut den deriverte til f i punktet 0.5
18 print(f"Den deriverte av f i x={a} er tilnærmet: {df}")
```

- b) Den deriverte av $f(x) = \sin\left(\frac{\pi}{2}x\right)$ er

$$f'(x) = \frac{\pi}{2} \cos\left(\frac{\pi}{2}x\right).$$

Setter vi $x = \frac{1}{2}$, får vi

$$\left(\frac{1}{2}\right) = \frac{\pi}{2} \cos\left(\frac{\pi}{4}\right) = \frac{\pi}{2} \cdot \frac{\sqrt{2}}{2} = \frac{\pi\sqrt{2}}{4}.$$

Verdien fra programmet i del a) vil være en tilnærmet verdi til dette eksakte resultatet. Jo mindre Δx vi velger, desto nærmere vil svaret fra koden være den eksakte verdien.

- c) For bedre presisjon kan vi:
- Velge en mindre verdi for `deltax`, for eksempel `deltax = 0.00001`.
 - Bruke symmetrisk differanse (som allerede er brukt) fremfor foroverdifferanse, da dette gir mer nøyaktige resultater (ikke innenfor pensum).
- d) Endringer i koden:

```
1 import numpy as np
2
3 # Definerer funksjonen g(x) = cos(x)
```

```

4 def g(x):
5     return np.cos(x)
6
7 # Funksjon for numerisk derivasjon
8 def derivert(a, deltax):
9     return (g(a + deltax) - g(a - deltax)) / (2*deltax)
10
11 # Punktet vi ønsker å finne den deriverte i
12 a = np.pi/3
13 deltax = 0.0001
14
15 dg = derivert(a, deltax)
16 print(f"Den deriverte av g i x={a} er tilnærmet: {dg}")

```

Fasit 0.1.2:

- La $f(x) = x^3 - x - 1$. Vi har $f(1) = 1^3 - 1 - 1 = -1$ og $f(2) = 2^3 - 2 - 1 = 5$, så siden f er kontinuerlig finnes en rot i $(1, 2)$. Derivasjonen er $f'(x) = 3x^2 - 1$. På $[1, 2]$ er $f'(x) > 0$, så f er strengt voksende og kan ha høyst én rot der. Dermed finnes nøyaktig én løsning.
- Programmet prøver å finne løsningen til $x^3 - x - 1 = 0$ ved Newtons metode:
- Bruk strengere toleranse ($|x_{ny} - x| < \varepsilon$ og $|f(x_{ny})| < \delta$), bedre startverdier, høyere numerisk presisjon og flere iterasjoner. Man kan også bruke linjesøk eller demping for å unngå dårlige steg.
- Vi endrer funksjon og derivasjon og velger startverdi $x_0 = \pi/4$:

```

1 import math
2
3 def g(x):
4     return math.cos(x) - x + 1
5
6 def gderiv(x):
7     return -math.sin(x) - 1
8
9 x = math.pi/4 # startverdi i [0, pi/2]
10 tol = 1e-8
11 maxiter = 100
12
13 for i in range(maxiter):
14     xny = x - g(x)/gderiv(x)
15     if abs(xny - x) < tol:
16         break
17     x = xny
18
19 print("Rot funnet:", x)
20

```

Fasit 0.1.3:

- Beregner integralet

$$\int_0^1 \sin\left(\frac{\pi}{2}x\right) dx$$

ved hjelp av *rektangelmetoden* og dele intervallet i 4 like store deler.

Resultatet: Hvis vi kjører koden, får vi en tilnærmet verdi av integralet.

- Integral av $f(x) = \sin\left(\frac{\pi}{2}x\right)$ er:

$$\int \sin\left(\frac{\pi}{2}x\right) dx = -\frac{2}{\pi} \cos\left(\frac{\pi}{2}x\right) + C$$

Setter vi inn grensene 0 og 1:

$$\int_0^1 \sin\left(\frac{\pi}{2}x\right) dx = -\frac{2}{\pi} \cos\left(\frac{\pi}{2} \cdot 1\right) + \frac{2}{\pi} \cos(0) = \frac{2}{\pi}(1 - 0) = \frac{2}{\pi}.$$

Tilnærmingen med rektangelmetoden vil være litt lavere enn $\frac{2}{\pi}$ siden vi bruker venstre endepunkt.

c) For bedre presisjon kan vi:

- Øke antall delintervaller, for eksempel $n = 100$ i stedet for $n = 4$.
- Bruke midpoint- eller trapezmetoden i stedet for venstre rektangelmetode.

d) Endringer i koden:

```
1 import numpy as np
2
3 def g(x):
4     return np.cos(np.pi/4 * x)
5
6 n = 8
7 a = 0
8 b = 2
9 dx = (b - a)/n
10
11 integral = 0
12 for i in range(n):
13     xi = a + i*dx
14     integral += g(xi)*dx
15
16 print(f"Tilnærmet verdi av integralet: {integral}")
```

Fasit 1.1.1:

```
1 import sympy as sp
2 z1=1+2*sp.I; z2=3-4*sp.I
3 print(z1+z2)
```

Fasit 1.1.2:

```
1 import sympy as sp
2 z1=2+sp.I; z2=1-3*sp.I
3 print(z1*z2)
```

Fasit 1.1.3:

```
1 import sympy as sp
2 z=3+4*sp.I
3 print(abs(z))
```

Fasit 1.1.4:

```
1 import sympy as sp
2 z=2-5*sp.I
3 print(sp.conjugate(z))
```

Fasit 1.1.5:

```
1 import sympy as sp
2 z=7+9*sp.I
3 print(sp.re(z), sp.im(z))
```

Fasit 1.1.6:

```
1 import sympy as sp
2 z=1+2*sp.I
3 print(z**2)
```

Fasit 1.1.7:

- f) Hver del kan løses med SymPy-funksjonene `sp.re`, `sp.im`, `sp.Abs`, `sp.conjugate`, `sp.arg`. Eksempelvis gir $|5 - 12i| = 13$.

Fasit 1.2.1:

- a) Programmet gjør følgende:

- Beregner transponerte A^T av matrisen A .
- Beregner determinanten $\det(A)$ og inversen A^{-1} hvis $\det(A) \neq 0$.
- Lager den utvidede matrisen $[A|b]$ og finner RREF for å løse $Ax = b$.

$$A^T = \begin{bmatrix} 2 & 1 & 3 \\ 1 & 0 & 2 \\ 3 & 2 & 1 \end{bmatrix}$$

- b) Siden $\det(A) = 3 \neq 0$, er A invertibel og systemet har en entydig løsning.

$$c) \mathbf{x} = \begin{bmatrix} x \\ y \\ z \end{bmatrix} = \mathbf{A}^{-1}\mathbf{b} = \begin{bmatrix} -4/3 & 5/3 & 2/3 \\ 5/3 & -7/3 & 1/3 \\ 2/3 & -1/3 & -1/3 \end{bmatrix} \begin{bmatrix} 5 \\ 3 \\ 6 \end{bmatrix} = \begin{bmatrix} 7/3 \\ 2/3 \\ -1/3 \end{bmatrix}$$

- d) Programmet vil skrive ut:

- $A^T = \begin{bmatrix} 2 & 1 & 3 \\ 1 & 0 & 2 \\ 3 & 2 & 1 \end{bmatrix}$
- $\det(A) = 3$
- $A^{-1} = \begin{bmatrix} -4/3 & 5/3 & 2/3 \\ 5/3 & -7/3 & 1/3 \\ 2/3 & -1/3 & -1/3 \end{bmatrix}$
- RREF av $[A|b]$ gir løsningen: $x = 7/3, y = 2/3, z = -1/3$.

Fasit 1.2.2:

- a) Programmet gjør følgende:

- Beregner inversen av A (hvis den eksisterer) og skriver den ut.
- Lager den utvidede matrisen $[A|b]$ og finner $RREF$, som viser hvordan systemet kan løses.

```

1 A^-1 =
2 Matrix([[ -1/4, 3/4, -1/4], [3/4, -1/4, -1/4], [-1/4, -1/4, 3/4]])
3 RREF av [A|b] =
4 (Matrix([
5 [1, 0, 0, 1],
6 [0, 1, 0, -1],
7 [0, 0, 1, 1]]), (0, 1, 2))

```

RREF gir systemet i redusert trappeform, og løsningen av $Ax = b$ blir $x = 1, y = 2, z = 0$.

$$\text{Inversen av } A \text{ er } A^{-1} = \begin{bmatrix} -1/4 & 3/4 & -1/4 \\ 3/4 & -1/4 & -1/4 \\ -1/4 & -1/4 & 3/4 \end{bmatrix}$$

- b) **Fasit 1.2.3:**

- a) Programmet gjør følgende:

- Beregner determinanten til A for å se om matrisen er invertibel.
- Lager den utvidede matrisen $[A|b]$ og finner $RREF$, som viser om systemet har én løsning, ingen løsning eller uendelig mange løsninger.

- b) Vi ser at

$$\det(A) = -8 \neq 0$$

så A er invertibel, og vi kan forvente en entydig løsning.

Når man kjører programmet, viser $RREF$ følgende løsning:

$$x = 1, \quad y = 2, \quad z = 1$$

Fasit 1.2.4:

- a) Programmet gjør følgende:
 - Beregner **inversen av A** (hvis den eksisterer) og skriver den ut.
 - Lager den **utvidede matrisen [A|b]** og finner **RREF**, som viser hvordan systemet kan løses.
- b) RREF gir systemet i redusert trappeform, og løsningen av $Ax = b$ blir:

$$x = 1, \quad y = 2, \quad z = 0$$

- Inversen av A er:

$$A^{-1} = \begin{bmatrix} -1 & 1 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & -1 \end{bmatrix}$$

Fasit 1.2.5:

- a) Programmet beregner **inversen av A**, konstruerer **den utvidede matrisen [A|b]** og finner **RREF** (redusert trappeform). Det skriver også ut **løsningen** x av ligningssystemet $Ax = b$ ved å bruke inversen.

For

$$A = \begin{bmatrix} 3 & 1 & -1 \\ 2 & 4 & 1 \\ 1 & -1 & 2 \end{bmatrix}, \quad b = \begin{bmatrix} 4 \\ 1 \\ 3 \end{bmatrix}$$

får man

Inversen av A:

$$A^{-1} = \begin{bmatrix} \frac{9}{25} & -\frac{1}{25} & \frac{3}{25} \\ -\frac{1}{25} & \frac{7}{25} & -\frac{3}{25} \\ \frac{1}{25} & -\frac{3}{25} & \frac{11}{25} \end{bmatrix}$$

RREF av [A|b]:

$$\left[\begin{array}{ccc|c} 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 2 \end{array} \right]$$

med pivot-kolonner (0, 1, 2).

Løsningen:

$$x = \begin{bmatrix} 1 \\ 0 \\ 2 \end{bmatrix}.$$

- b) **Linje 1:** `import sympy as sp` — importerer SymPy-biblioteket.
Linje 3–6: Definerer matrisen A og vektoren b for systemet $Ax = b$.
Linje 9: `Ainv = A.inv()` — beregner inversen A^{-1} .
Linje 10–11: Skriver ut en overskrift og matrisen A^{-1} .
Linje 14: `Ab = A.row_join(b)` — lager den utvidede matrisen $[A|b]$.
Linje 15: `R, pivots = Ab.rref()` — finner RREF og pivot-kolonner.
Linje 16–18: Skriver ut RREF og pivot-kolonner.
Linje 21: `x = Ainv * b` — løser systemet $x = A^{-1}b$.
Linje 22–23: Skriver ut løsningen x .

Fasit 1.2.6:

- a) Programmet verifiserer at $\det(A) = 1$, beregner **inversen av A**, konstruerer **den utvidede matrisen [A | b]** og finner **RREF** (redusert trappeform). Det skriver også ut **løsningen** x av ligningssystemet $Ax = b$ ved å bruke A^{-1} .
 For

$$A = \begin{bmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \\ 0 & 0 & 1 \end{bmatrix}, \quad b = \begin{bmatrix} 2 \\ -1 \\ 3 \end{bmatrix}$$

får man

Determinant: $\det(A) = 1$.

Inversen av A :

$$A^{-1} = \begin{bmatrix} 1 & -1 & 1 \\ 0 & 1 & -1 \\ 0 & 0 & 1 \end{bmatrix}.$$

RREF av $[A \mid b]$:

$$\left[\begin{array}{ccc|c} 1 & 0 & 0 & 6 \\ 0 & 1 & 0 & -4 \\ 0 & 0 & 1 & 3 \end{array} \right]$$

med pivot-kolonner $(0, 1, 2)$.

Løsningen:

$$x = \begin{bmatrix} 6 \\ -4 \\ 3 \end{bmatrix}.$$

b) **Linje 1:** `import sympy as sp` — importerer SymPy-biblioteket.

Linje 3–6: Definerer matrisen A (med $\det(A) = 1$) og vektoren b for systemet $Ax = b$.

Linje 9: `A.det()` — beregner determinanten $\det(A)$ for å bekrefte at den er 1.

Linje 12: `Ainv = A.inv()` — beregner inversen A^{-1} .

Linje 13–14: Skriver ut en overskrift og matrisen A^{-1} .

Linje 17: `Ab = A.row_join(b)` — lager den utvidede matrisen $[A \mid b]$.

Linje 18: `R, pivots = Ab.rref()` — finner RREF og pivot-kolonner.

Linje 19–20: Skriver ut RREF og pivot-kolonner.

Linje 23: `x = Ainv * b` — løser systemet $x = A^{-1}b$.

Linje 24–25: Skriver ut løsningen x .

Fasit 1.2.7:

a) Programmet bekrefter at $\det(A) = 1$, beregner **inversen av A** , konstruerer **den utvidede matrisen $[A \mid b]$** og finner **RREF**. Deretter finner det **løsningen x** til $Ax = b$ via inversen.

For

$$A = \begin{bmatrix} 2 & -1 & 0 \\ 1 & 2 & 1 \\ 0 & -1 & 1 \end{bmatrix}, \quad b = \begin{bmatrix} 3 \\ 0 \\ 5 \end{bmatrix}$$

får man

Determinant: $\det(A) = 1$.

Inversen av A :

$$A^{-1} = \begin{bmatrix} \frac{1}{3} & \frac{1}{3} & -\frac{1}{3} \\ -\frac{1}{3} & \frac{2}{3} & -\frac{1}{3} \\ \frac{1}{3} & -\frac{1}{3} & \frac{2}{3} \end{bmatrix}.$$

RREF av $[A \mid b]$:

$$\left[\begin{array}{ccc|c} 1 & 0 & 0 & 1 \\ 0 & 1 & 0 & 2 \\ 0 & 0 & 1 & 1 \end{array} \right]$$

med pivot-kolonner $(0, 1, 2)$.

Løsningen:

$$x = \begin{bmatrix} 1 \\ 2 \\ 1 \end{bmatrix}.$$

- b) **Linje 1:** `import sympy as sp` — importerer SymPy-biblioteket.
Linje 3–6: Definerer matrisen A (en 3×3 -matrise med $\det(A) = 1$) og vektoren b .
Linje 9: `A.det()` — beregner determinanten $\det(A)$.
Linje 12: `Ainv = A.inv()` — beregner inversen A^{-1} .
Linje 13–14: Skriver ut A^{-1} .
Linje 17: `Ab = A.row_join(b)` — konstruerer den utvidede matrisen $[A \mid b]$.
Linje 18: `R, pivots = Ab.rref()` — finner RREF og pivot-kolonner.
Linje 19–20: Skriver ut RREF og pivot-kolonner.
Linje 23: `x = Ainv * b` — løser systemet $x = A^{-1}b$.
Linje 24–25: Skriver ut løsningen x .

Fasit 1.2.8:

- a) Det setter opp det utvidede systemet $[A|b]$ og finner RREF for å bestemme hva slags løsning som finnes for ulike verdier av k .
b) Etter rekkereduksjon får vi:

$$\left[\begin{array}{ccc|c} 1 & k & 1 & 1 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & k-1 \end{array} \right].$$

- For alle k : rad 2 forsvinner, men systemet er konsistent.
- Hvis $k \neq 1$: Vi får pivot i kolonne 1 og 3, slik at y er fri parameter. Dermed: *uendelig mange løsninger*.
- Hvis $k = 1$: Den siste likningen blir $z = 0$, og vi får igjen y som fri parameter. Fremdeles: *uendelig mange løsninger*.
- (Hvis høyresiden hadde vært litt annerledes, kunne vi fått en motsigelse $0 = 1$, som ville gitt *ingen løsning*.)

- c) Systemet reduseres til

$$x + 2y + z = 1, \quad z = 1.$$

Da: $x = 0 - 2y$. Dermed

$$(x, y, z) = (-2s, s, 1), \quad s \in \mathbb{R}.$$

Fasit 1.2.9:

- a)

$$12I_1 + 5I_2 + 3I_3 = 31$$

$$10I_1 + 7I_2 + 4I_3 = 36$$

$$15I_1 + 6I_2 + 5I_3 = 42$$

- b) Koeffisientmatrisen for systemet er:

$$A = \begin{bmatrix} 12 & 5 & 3 \\ 10 & 7 & 4 \\ 15 & 6 & 5 \end{bmatrix}$$

For å beregne determinanten, kan vi bruke Python eller Matlab. Her er et eksempel på Python-kode:

```
1 import sympy as sym
2 A = sym.Matrix([[12, 5, 3],
3                 [10, 7, 4],
4                 [15, 6, 5]])
5 print(A.det())
```

Beregningsresultatet viser at determinanten er $\det(A) = 35$, som er forskjellig fra null. Dermed har systemet en entydig løsning.

- c) Bruk den inverse matrisen for å løse systemet. Vi kan bruke Python for å beregne løsningen:

```
1 b = sym.Matrix([[31], [36], [42]])
2 x = A.inv() * B
3 print(x)
```

Dette gir oss løsningen:

$$\begin{bmatrix} I_1 \\ I_2 \\ I_3 \end{bmatrix} = \begin{bmatrix} 1.00 \\ 2.00 \\ 3.00 \end{bmatrix}$$

Dermed finner vi at inntektene fra de forskjellige sektorene er:

- Sektor S_1 genererer 1 million kroner
- Sektor S_2 genererer 2 millioner kroner
- Sektor S_3 genererer 3 millioner kroner

Fasit 1.3.1:

a) $v = \frac{x_4 - x_0}{t_4 - t_0} = \frac{49,2 - 0,0}{1 - 0} = 49,2$ mph (miles per hour).

b) Anta at vi leser av grafen:

Tid, t	Posisjon, x
0.25	2,300
0.26	2,512

b) $v = \frac{x_1 - x_0}{t_1 - t_0} = \frac{2,512 - 2,300}{0,26 - 0,25} = 21,2$ mph (miles per hour)

- c) Tallene er stigningstallene til tangentene (de tre røde linjestykkene).

Fasit 1.3.2:

- a) Koden definerer funksjonen $f(x) = x^3 - 3x + 2$, og setter opp en funksjon som beregner stigningstallet for sekanten i intervallet $[a, a + h]$. Deretter beregner en tilnærmet verdi for funksjonen i punktet $a = 2$.

```
b)1 # Først definerer vi funksjonen f(x)
2 def f(x):
3     return x**3 + 3*x + 2 # eksempel: en kvadratisk funksjon
4
5 # Så definerer vi en funksjon for å beregne den deriverte
6 def derivert(a, deltax):
7     return (f(a + deltax) - f(a)) / deltax
8
9 # Eksempelbruk:
10 a = 2
11 deltax = 0.01 # En liten verdi for delta x
12
13 df=derivert(a, deltax)
14 print("Den deriverte av f i punktet", a, "er tilnærmet:", df)
```

- c) Endre deltax til en mindre verdi som for eksempel $\text{deltax} = 0.001$.

```
d)1 # Først definerer vi funksjonen f(x)
2 def g(x):
3     return 2*x**5 - 5*x + 4 #
4
5 # Så definerer vi en funksjon for å beregne den deriverte
6 def derivert(a, deltax):
7     return (g(a + deltax) - g(a)) / deltax
8
9 # Eksempelbruk:
10 a = -1 # Punktet vi ønsker å finne den deriverte i
11 deltax = 0.01 # En liten verdi for delta x
12
13 dg=derivert(a, deltax)
14 print("Den deriverte av g i punktet", a, "er tilnærmet:", dg)
```

Fasit 1.3.3:

- Programmet finner den deriverte til $f(x) = 3x^4 - x + 1$ i punktet $a = -2$ ved hjelp av numerisk derivasjon.
- Definerer funksjonen $f(x)$, lager en funksjon for numerisk derivasjon, velger punktet $a = -2$, en liten Δx , og printer resultatet.
- Minsk `deltax` til for eksempel 0.0001.
- Importer `math` eller `numpy` (det er lettere å skrive `import numpy as np`, der `np` bare er et kort alias for biblioteket `numpy`), definer $m(x) = \exp(x)$ og sett `a=1`.

```

1 import numpy as np
2
3 # Definerer funksjonen m(x)
4 def m(x):
5     return np.exp(x)
6
7 # Numerisk derivasjon med foroverdifferanse
8 def derivert(a, deltax):
9     return (m(a + deltax) - m(a)) / deltax
10
11 # Punktet der vi vil beregne den deriverte
12 a = 1
13 deltax = 0.0001
14
15 # Beregner den deriverte
16 dm = derivert(a, deltax)
17 print(f"Den deriverte av m(x)=e^x i punktet x={a} er tilnærmet: {dm}")

```

Fasit 1.3.4:

Programmet beregner den deriverte til $f(x) = x^3 - 3x^2$ numerisk i intervallet $[-2, 5]$ og plotter grafen til funksjonen og dens deriverte med å dele intervallet i 500 deler (501 punkter).

Fasit 1.4.1:

Ved hjelp av de 3 differanse-metodene får vi:

$$\begin{aligned}
 f'(0) &\approx \frac{f(x_0 + h) - f(x_0)}{h} = \frac{f(0.25) - f(0)}{0.25} = \frac{0.448 - 0.385}{0.25} \simeq 0.252, & \text{forlengsdifferanse,} \\
 f'(x_0) &\approx \frac{f(x_0) - f(x_0 - h)}{h} = \frac{f(0) - f(-0.25)}{0.25} = \frac{0.385 - 0.273}{0.25} \simeq 0.448, & \text{baklengsdifferanse,} \\
 f'(x_0) &\approx \frac{f(x_0 + h) - f(x_0 - h)}{2h} = \frac{f(0.25) - f(-0.25)}{0.25} = \frac{0.448 - 0.273}{0.50} \simeq 0.350, & \text{sentraldifferanse.}
 \end{aligned}$$

Sentraldifferanse-metoden gir middelveiden av de to andre metodene.

Fasit 1.4.2:

```

1
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 N = 100
6 x = np.linspace(-3,3, num = N)
7 f = np.exp(x) - 4*x #Svart
8 plt.plot(x,f,'k')
9
10 def Funksjon_f(v):
11     return np.exp(v) - 4*v
12
13 x0=2
14 for i in range(1,5):
15     h = 10**(-i)

```

```

16     print('Forlengsdifferanse gir:', (Funksjon_f(x0+h)-Funksjon_f(x0))/h)
17     print('Baklengsdifferanse gir:', (Funksjon_f(x0)-Funksjon_f(x0-h))/h)
18     print('Sentraldifferanse gir:', (Funksjon_f(x0+h)-Funksjon_f(x0-h))/(2*h))
19
20
21 y1 = np.zeros(N)
22 y2 = np.zeros(N)
23 y3 = np.zeros(N)
24 h = 10**(-6)
25
26 for i in range(N):
27     y1[i] = (Funksjon_f(x[i]+h)-Funksjon_f(x[i]))/h
28     y2[i] = (Funksjon_f(x[i])-Funksjon_f(x[i]-h))/h
29     y3[i] = (Funksjon_f(x[i]+h)-Funksjon_f(x[i]-h))/(2*h)
30 plt.plot(x,y1,'x','b')
31 plt.plot(x,y2,'.','r')
32 plt.plot(x,y3,'o')

```

Fasit 1.4.3:

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 N = 1000
5 x = np.linspace(-np.pi,np.pi, num = N)
6 f = np.sin(x) #Svart
7 plt.plot(x,f,'k')
8
9 def Funksjon_f(v):
10     return v**2
11
12 x0=3
13
14 for i in range(1,5):
15     a = 10**(-i)
16     print((Funksjon_f(x0+a)-Funksjon_f(x0))/a)
17
18 y = np.zeros(N)
19 a = 10**(-4)
20
21 for i in range(N):
22     y[i] = (Funksjon_f(x[i]+a)-Funksjon_f(x[i]))/a
23
24 plt.plot(x,y)

```

Fasit 1.4.4:

```

a)1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 n = 6 # Antall delintervaller
5
6 x = np.linspace(1,4,num = n+1)
7 deltax = x[1] - x[0] # storrelsen paa delintervallene
8 f = 4*x**3 - 2*x # funksjonen vi vil integrere
9
10 Sum = 0
11
12 # Trapesmetoden
13 for i in range(len(x)-1):
14     Sum = Sum + ((f[i]+f[i+1])/2)*deltax

```

```

15
16 # print resultatet
17 print(Sum)
18
19 # Definer funksjonen paa ny for glattere graf
20 u = np.linspace(1,4,num = 10000)
21 f1 = 4*u**3 - 2*u
22
23 # Plot funksjonen og tilnaermingen
24 plt.plot(x,f)
25 plt.plot(u,f1, label='f(x)')
26 plt.grid(ls='--')
27 plt.legend()

```

HVA skal være Δx .

b) Med å endre N til en større verdi.

Fasit 1.6.1:

Programkoden bruker Newtonsmetoden for å bestemme løsningen til $x^3 - 3x^2 - 1 = 0$ med startverdi 0.1.

Fasit 1.6.2:

Programkoden bruker Newtonsmetoden for å bestemme løsningen til $x^3 - \cos x = 0$ med startverdi 1.

Fasit 1.6.3:

Skjæringssetningen: $f(x) = x + \sin(x) + 1$, som er kontinuerlig. Vi velger endepunktene gitt av intervallet:

$$f\left(\frac{-\pi}{2}\right) = \frac{-\pi}{2} + \sin\left(\frac{-\pi}{2}\right) + 1 = \frac{-\pi}{2} - 1 + 1 = \frac{-\pi}{2} < 0$$

$$f(0) = 0 + \sin(0) + 1 = 1 > 0$$

Da sier skjæringssetningen at f har minst et nullpunkt i $[-\pi/2, 0]$.

Den derivate: $f'(x) = 1 + \cos(x) \geq 0$ i $[-\pi/2, 0]$. Siden cosinus alltid er mellom -1 og 1, er alltid $f'(x) \geq 0$. I tillegg er cosinus mellom 0 og 1 i intervallet, så $f' > 0$. Det betyr at f er en-entydig, og den kan ikke ha mer enn ett nullpunkt i intervallet.

Newtons metode: $x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$

$$x_1 = x_0 - \frac{f(x_0)}{f'(x_0)} = -1 - \frac{-1 + \sin(-1) + 1}{1 + \cos(-1)} = -0,4536975$$

$$x_2 = x_1 - \frac{f(x_1)}{f'(x_1)} = -0,4536975 - \frac{-0,4536975 + \sin(-0,4536975) + 1}{1 + \cos(-0,4536975)} = -0,510580$$

$$x_3 = x_2 - \frac{f(x_2)}{f'(x_2)} = -0,510580 - \frac{-0,510580 + \sin(-0,510580) + 1}{1 + \cos(-0,510580)} = -0,51097$$

Løsningen er da $x = -0,51$ (her var det ikke presisert hvor mange desimaler man måtte ta med. Det er da nok med 4-5 desimaler).

Fasit 1.6.4:

```

1 def newton(f, Df, x0, epsilon, maks_iter):
2     '''
3     Løsningsmetode for f(x) = 0 ved Newtons metode.
4
5     Parametre
6     -----
7     f : funksjon
8         Funksjonen vi ønsker å finne nullpunktet til.
9     Df : funksjon
10        Den derivate av f(x).
11     x0 : float
12        Startverdi for iterasjonen.
13     epsilon : float

```

```

14     Ønsket nøyaktighet. Iterasjonen stopper når abs(f(xn)) < epsilon.
15     maks_iter : int
16     Maksimalt antall iterasjoner.
17
18     Returnerer
19     -----
20     xn : float eller None
21         En tilnærmet løsning hvis konvergens oppnås, ellers None.
22     '''
23     xn = x0
24     for n in range(maks_iter):
25         fxn = f(xn)
26         print(n, xn, abs(fxn)) # Valgfritt: kan kommenteres ut
27
28         if abs(fxn) < epsilon:
29             print(f'Løsningen er funnet etter {n} iterasjoner.')
30             return xn
31
32         Dfxn = Df(xn)
33         if Dfxn == 0:
34             print('Den deriverte er 0. Ingen løsning funnet.')
35             return None
36
37         xn = xn - fxn / Dfxn
38
39     print('Overskredet maks antall iterasjoner. Ingen løsning funnet.')
40     return None
41
42 p = lambda x: x**3 - x**2 - 3
43 Dp = lambda x: 3*x**2 - 2*x
44 approx = newton(p,Dp,1.5,1e-5,10)
45 print(approx)

```

Med å kjøre programmet, får vi:

```

1 1 2.0 1.0
2 2 1.875 0.076171875
3 3 1.8637931034482758 0.0005794671778258653
4 4 1.8637065329604223 3.4409209703767374e-08
5 Løsningen er funnet etter 4 iterasjoner.
6 1.8637065329604223

```

Fasit 1.7.1:

programmet finner en tilnærmet løsning for ligningen $2x \cos x - 1 = 0$ som ligger på intervallet $(0, 1)$ ved hjelp av halveringsmetoden der estimeringsavviket er mindre enn 0,001. Den tilnærmede løsningen, og antall iterasjon blir også skrevet ut. Ved å kjøre programmet får vi:

```

1 0.60986328125
2 Antall iterasjoner: 10

```

Fasit 1.7.2:

```

a) 1 def f(x):
2     return x**3-5
3
4     a = 1
5     b = 2
6     d = b - a #Finn avstanden mellom a og b
7     m = (a+b)/2
8     e=0.001
9
10    while abs(d) > e:

```

```

11
12     if f(m)*f(a) < 0: #Hvis fm og fa har motsatt fortegn:
13         b = m         #Flytter b til midten
14     elif f(m)*f(b)<0:
15         a = m         #Flytter a til midten
16     elif f(m)==0:
17         break
18     else:
19         print("Feil: f(a) og f(b) skal ligge på hver si side av x-aksen!")
20         break
21 # Sett opp et nytt midtpunkt og reknar ut f-verdi:
22     m = (a+b)/2
23     d=b-a
24 print(f'Nullpunktet med presisjonen {e} er: ', m)

```

Her kan vi legge inn antall iterasjoner:

```

1 def f(x):
2     return x**3-5
3
4 a = 1
5 b = 2
6 d = b - a #Finn avstanden mellom a og b
7 m = (a+b)/2
8 n=0
9 e=0.001
10
11 while abs(d) > e:
12
13     if f(m)*f(a) < 0: #Hvis fm og fa har motsatt fortegn:
14         b = m         #Flytter b til midten
15     elif f(m)*f(b)<0:
16         a = m         #Flytter a til midten
17     elif f(m)==0:
18         break
19     else:
20         print("Feil: f(a) og f(b) skal ligge på hver si side av x-aksen!")
21         break
22 # Sett opp et nytt midtpunkt og reknar ut f-verdi:
23     m = (a+b)/2
24     d=b-a
25     n=n+1
26 print(f'Nullpunktet med presisjonen {e} er: ', m)
27 print("Antall iterasjoner:",n)

```

Ved å kjøre programmet, får vi:

```

1 Nullpunktet med presisjonen 0.001 er: 1.71044921875
2 Antall iterasjoner: 10

```

```

1 def f(x):
2     return x**3 - 5
3 a = 1
4 b = 2
5 n = 0 # Antall iterasjoner
6
7 Nøyaktighet = .001
8 deltax = .01
9 while deltax > Nøyaktighet:
10     if f(a) * f(a + deltax) < 0:
11         deltax = deltax / 10
12     else:

```

```

13     a = a + deltax
14     n = n+1
15
16 print(a)
17 print("Antall iterasjoner:",n)

```

Ved å kjøre programmet, får vi:

```

1 1.70000000000000006
2 Antall iterasjoner: 70

```

b) Det er tydelig at den første metoden er numerisk kostbar.

Fasit 1.7.3:

Skjæringssetningen: $f(x) = x + \sin(x) + 1$, som er kontinuerlig. Vi velger endepunktene gitt av intervallet:
 $f\left(\frac{-\pi}{2}\right) = \frac{-\pi}{2} + \sin\left(\frac{-\pi}{2}\right) + 1 = \frac{-\pi}{2} - 1 + 1 = \frac{-\pi}{2} < 0$

$$f(0) = 0 + \sin(0) + 1 = 1 > 0$$

Da sier skjæringssetningen at f har minst et nullpunkt i $[-\pi/2, 0]$.

Den deriverte: $f'(x) = 1 + \cos(x) \geq 0$ i $[-\pi/2, 0]$. Siden cosinus alltid er mellom -1 og 1, er alltid $f'(x) \geq 0$. I tillegg er cosinus mellom 0 og 1 i intervallet, så $f' > 0$. Det betyr at f er en-entydig, og den kan ikke ha mer enn ett nullpunkt i intervallet.

Newtons metode: $x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}$

$$x_1 = x_0 - \frac{f(x_0)}{f'(x_0)} = -1 - \frac{-1 + \sin(-1) + 1}{1 + \cos(-1)} = -0,4536975$$

$$x_2 = x_1 - \frac{f(x_1)}{f'(x_1)} = -0,4536975 - \frac{-0,4536975 + \sin(-0,4536975) + 1}{1 + \cos(-0,4536975)} = -0,510580$$

$$x_3 = x_2 - \frac{f(x_2)}{f'(x_2)} = -0,510580 - \frac{-0,510580 + \sin(-0,510580) + 1}{1 + \cos(-0,510580)} = -0,51097$$

Løsningen er da $x = -0,51$ (her var det ikke presisert hvor mange desimaler man måtte ta med. Det er da nok med 4-5 desimaler).

Fasit 1.7.4:

Løsning:

a) Ifølge Pytagoras læresetning er: $BC = \sqrt{BD^2 + DC^2} = \sqrt{40^2 + 30^2} = 50\text{m}$.

b) Deler opp problemet i to deler, regner farten på land og i vann:

$$v_1 = \frac{100}{20} = 5 \text{ m/s, og } v_2 = \frac{50}{60} = \frac{5}{6} \text{ m/s.}$$

Forholdet blir da: $\frac{v_1}{v_2} = 5/(5/6) = 6$, det vil si badevakten løper 6 ganger raskere på land enn badevakten svømmer i vann.

c) Problemløsning:

1) Dele opp problemet:

i) Bestem tiden fra A til C: $t(x) = \text{Tiden for å løpe } AB + \text{tiden for å svømme } BC$:

$$t = \frac{140 - x}{5} + \frac{\sqrt{x^2 + 30^2}}{1}.$$

2) Analyserer problemet, og gjenkjenner problemstillingen (minste tid): ii) For å bestemme minste tid, kan vi derivere funksjonen:

$$t'(x) = -\frac{1}{5} + \frac{2x}{2\sqrt{x^2 + 900}}$$

3: Løs problemet

$$t'(x) = -\frac{1}{5} + \frac{2x}{2\sqrt{x^2+9}} = -\frac{1}{5} + \frac{x}{\sqrt{x^2+9}} = 0.$$

$$5x = \sqrt{x^2+900} \text{ som gir } 24x^2 = 900 \text{ og dermed } x = \frac{30}{2\sqrt{6}} = \frac{15}{\sqrt{6}} \simeq 6,1 \simeq 6 \text{ m.}$$

Dette er den minste verdien fordi:

$$t''(x) = \frac{1 - \frac{2x}{2\sqrt{x^2+900}}}{(\sqrt{x^2+900})^2} = \frac{\sqrt{x^2+900} - x}{x^2+900}$$

og $t''(\frac{15}{\sqrt{6}}) > 0$ (krummer nedover) og

dermed er $x = \frac{15}{\sqrt{6}}$ er minimumspunkt og gir minste tiden.

4: Vurdering av løsningen

$x \simeq 6$ ser fornuftig ut, da $t(6) = \frac{140-6}{5} + \sqrt{6^2+900} = \frac{\frac{15}{\sqrt{6}}}{\sqrt{225/6+900}} \simeq 57,4$. Legg merke til at vi brukte her tilnærmet verdi for $x = 6$ og tiden blir litt mer enn det vi har regnet ut. Uansett blir ca. 1 min, og vi ser at denne tiden ser fornuftig liten ut.

Med programmering leses AD og CD fra skjermen, og det settes opp: $t = \frac{AD-x}{5} + \frac{\sqrt{x^2+CD^2}}{1}$. Deretter setter opp deriverte til t med hensyn til x

```
1 x = symbols('x')
2 t = (AD-x)/5 + np.sqrt(x**2+CD**2)/1
3 dt = diff(t, x)
```

og finner nullpunktet til $t'(x)$.

Man kunne sette opp $t''(x)$ og sjekke om $t''(\text{nullpunkt}) > 0$.

d) Programmeringsdel (forklaring og graf):

Programmet gjør følgende steg:

- Leser inn verdier for AD , CD , løpefart v_1 og svømmefart v_2 .
- Definerer funksjonen $t(x) = \frac{AD-x}{v_1} + \frac{\sqrt{x^2+CD^2}}{v_2}$.
- Beregner den numeriske deriverte $t'(x)$ ved differansekvotienten for å finne minimum.
- Skriver ut det optimale punktet x (hvor badevakten bør hoppe i vannet) og den tilhørende minste tiden.
- Tegner grafen til $t(x)$ som funksjon av x , slik at man visuelt kan se hvor minimum oppstår.

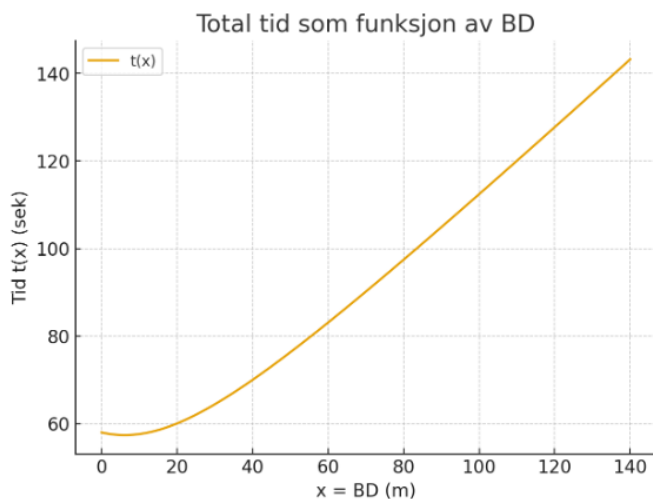
Et eksempel på kode for å tegne grafen i Python er:

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # Parametre
5 AD = 140
6 CD = 30
7 v1 = 5
8 v2 = 1
9
10 # Funksjon for total tid
11 def t(x):
12     return (AD - x)/v1 + np.sqrt(x**2 + CD**2)/v2
13
14 # Lag x-verdier og beregn t(x)
```

```

15 xverdier = np.linspace(0, AD, 500)
16 tverdier = t(xverdier)
17
18 # Plott grafen
19 plt.plot(xverdier, tverdier, label="t(x)")
20 plt.xlabel("x = BD (meter)")
21 plt.ylabel("Tid t(x) (sekunder)")
22 plt.title("Total tid som funksjon av BD")
23 plt.grid(True)
24 plt.legend()
25 plt.show()

```



Grafen viser hvordan total tid varierer med x , og man kan tydelig se hvor minimum oppstår.

Fasit 1.8.1:

- a) Vi implementerer rektangelmetoden i Python og lar N variere.

```

1 import numpy as np
2
3 f = lambda x: x**2 + 1
4 a, b = 0, 2
5 Nverdier = [5, 10, 50]
6
7 print(" N      Left      Mid      Right")
8 print("-----")
9 for N in Nverdier:
10     dx = (b - a)/N
11     x = np.linspace(a, b, N+1)
12
13     Rv = sum(f(x[k]) for k in range(N)) * dx
14     Rh = sum(f(x[k+1]) for k in range(N)) * dx
15     Rm = sum(f((x[k]+x[k+1])/2) for k in range(N)) * dx
16
17     print(f"{N:2d}  {Rv:8.5f}  {Rm:8.5f}  {Rh:8.5f}")

```

For det eksakte integralet:

$$\int_0^2 (x^2 + 1) dx = \left[\frac{x^3}{3} + x \right]_0^2 = \frac{8}{3} + 2 = \frac{14}{3} \approx 4.6667.$$

- b) Kjører vi koden får vi typisk (avrundet):

N	Rv	Rm	Rh
5	3.888	4.630	5.528
10	4.260	4.648	5.036
50	4.554	4.666	4.778

Vi ser at midtmetoden Rm gir verdier mye nærmere det eksakte integralet 4.6667 enn venstre og høyre for samme N .

- c) Når N øker, blir delintervallene smalere og rektangelmetoden gir bedre tilnærming:

$$\lim_{N \rightarrow \infty} Rv = \lim_{N \rightarrow \infty} Rm = \lim_{N \rightarrow \infty} Rh = \int_0^2 (x^2 + 1) dx.$$

Feilen avtar omtrent proporsjonalt med $\Delta x = (b - a)/N$ for venstre/høyre, men midtmetoden har ofte en raskere konvergens (feilen avtar som Δx^2 i mange tilfeller).

Fasit 1.9.1:

- Resultatene viser at $Tn \rightarrow 2$ når N øker, og feilen avtar omtrent som h^2 .
- Ved dobling av N blir h halvert, og feilen reduseres omtrent til en fjerdedel.

Fasit 1.10.1:

- Simpsonmetoden gir langt mindre feil enn trapesmetoden for samme N , fordi feilen avtar som h^4 mot h^2 .
- Simpsonmetoden bruker både rette linjer og parabler for å tilpasse funksjonen, og utnytter mer informasjon om kurven enn trapesmetoden.

Fasit 1.10.2:

- Programmet prøver å beregne det bestemte integralet

$$\int_0^1 \frac{x}{1+x^2} dx$$

både numerisk ved hjelp av den venstre Riemann-summen og eksakt med `scipy.integrate.quad`. Eksakt verdi kan finnes analytisk:

$$\int_0^1 \frac{x}{1+x^2} dx = \left[\frac{1}{2} \ln(1+x^2) \right]_0^1 = \frac{1}{2} \ln 2 \approx 0.34657359.$$

```

b)1 import numpy as np
2 import scipy.integrate as integrate
3
4 # Integrasjonsgrenser
5 a, b = 0, 1
6 def f(x):
7     return x/(1+x**2)
8
9 # Eksakt verdi
10 eksakt, _ = integrate.quad(f, a, b)
11
12 for N in [10, 100]:
13     dx = (b - a)/N
14     x = np.linspace(a, b, N+1)
15     riemann = np.sum(f(x[:-1])) * dx
16     avvik = abs(eksakt - riemann)
17     print(f"N={N}: Riemann = {riemann:.8f}, Feil = {avvik:.8e}")

```

Kjøring gir:

$$\begin{aligned} N = 10 : \quad & \text{Riemann} = 0.3328826, \quad \text{Feil} = 1.36910 \times 10^{-2} \\ N = 100 : \quad & \text{Riemann} = 0.3458894, \quad \text{Feil} = 6.84200 \times 10^{-4} \end{aligned}$$

Vi ser at feilen minker når N øker.

Fasit 1.10.3:

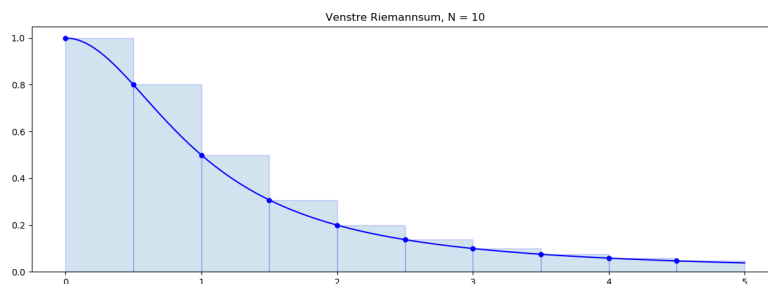
a) $\int_0^5 \frac{1}{x^2+1} dx = [\arctan x]_0^5 = \arctan 5 \simeq 1.37340077$

b) Python-program:

```
1 import scipy.integrate as integrate
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 a = 0
6 b = 5
7 N = int(input("Skriv inn antall patisjoner N: "))
8
9 h = (b - a) / N
10 x = np.linspace(a,b,N+1)
11 f = 1/(1+x**2)
12 ff = lambda x: 1/(1+x**2)
13
14 eksakt,error=integrate.quad(ff, a, b)
15 print(f'Eksaktverdien til integrasjonen er: {eksakt}')
16 I_riemannL = h * sum(f[:N])
17 erRhiemannL =abs(eksakt - I_riemannL)
18
19 print(f'Venstre Riemann-integrasjon med n={N} gir: {I_riemannL}')
20 print(f'Estimeringsavvik ved venstre Riemann-integrasjon med n={N} er: {
    erRhiemannL}')
21
22
23 x = np.linspace(a,b,N)
24 y = ff(x)
25
26 plt.figure(figsize=(15,5))
27
28 plt.plot(x,y,'b')
29 x_right = x[1:] # Venstre endepunkt
30 y_right = y[1:]
31 plt.plot(x_right,y_right,'b.',markersize=10)
32 plt.bar(x_right,y_right,width=-(b-a)/N,alpha=0.2,align='edge',edgecolor='b')
33 plt.title('Nedre Riemann Sum, N = {}'.format(N))
34
35 plt.show()
```

Løsning:

```
1
2 Skriv inn antall patisjoner N: 10
3 Eksaktverdien til integrasjonen er: 1.3734007669450166
4 Venstre Riemann-integrasjon med n=10 gir: 1.6134886966147248
5 Estimeringserror 0.4807692307692308
```



Fasit 1.10.4:

a) Programmet prøver å beregne det bestemte integralet

$$\int_0^2 x e^{-x^2} dx$$

både numerisk ved hjelp av den venstre Riemann-summen og eksakt med `scipy.integrate.quad`. Eksakt verdi kan finnes analytisk ved substitusjon $u = -x^2$:

$$\int_0^2 x e^{-x^2} dx = -\frac{1}{2} \int_0^{-4} e^u du = \frac{1}{2} (1 - e^{-4}).$$

```
b) 1 import numpy as np
2 import scipy.integrate as integrate
3
4 # Integrasjonsgrenser
5 a, b = 0, 2
6 def f(x):
7     return x * np.exp(-x**2)
8
9 # Eksakt verdi
10 eksakt, _ = integrate.quad(f, a, b)
11
12 for N in [10, 100]:
13     dx = (b - a)/N
14     x = np.linspace(a, b, N+1)
15     riemann = 0
16     for k in range(N):
17         riemann += f(x[k]) * dx
18     avvik = abs(eksakt - riemann)
19     print(f"N={N}: Riemann = {riemann:.8f}, Feil = {avvik:.8e}")
```

Kjøring gir typisk:

$$N = 10 : \quad \text{Riemann} = 0.49985382, \quad \text{Feil} = 7.613 \times 10^{-5}$$

$$N = 100 : \quad \text{Riemann} = 0.49999953, \quad \text{Feil} = 4.7 \times 10^{-7}$$

Vi ser at feilen minker når N øker.

Fasit 1.10.5:

Ved trapesmetoden: 0,22070, med estimeringsavvik: $|e_T| < \frac{12}{12 \cdot 4^2} (1 - 0)^3 = 0,0625$

Ved Simpsons formel: 0,20052, med estimeringsavvik: $|e_T| < \frac{24}{180 \cdot 4^2} (1 - 0)^3 \simeq 0,0083$

Fasit 1.10.6:

a) Programmet beregner det bestemte integralet

$$\int_0^2 e^{x^2} dx$$

på to måter:

- **Trapesmetoden:** Summerer arealet av trapesene under kurven over hvert delintervall.
- **Simpsons regel:** Bruker parabeltilnærming (tre punkter) over hvert par av delintervaller, noe som gir høyere presisjon.

Programmet sammenligner begge metodene med en eksakt verdi beregnet numerisk med `scipy.integrate.quad`, og beregner avviket.

Fasit 1.10.7:

```
a) 1 import numpy as np
   2 import scipy.integrate as integrate
   3
   4 # Funksjonen vi integrerer
   5 def f(x):
   6     return np.exp(x**2)
   7
   8 # Integrasjonsgrenser og delintervaller
   9 a, b = 0, 2
  10 n = 4
  11 dx = (b-a)/n
  12 x = np.linspace(a, b, n+1)
  13
  14 # Trapesmetoden
  15 trap = 0
  16 for k in range(n):
  17     trap += (f(x[k]) + f(x[k+1]))/2 * dx
  18
  19 # Eksakt verdi med scipy
  20 eksakt, _ = integrate.quad(f, a, b)
  21
  22 avvik_trap = abs(trap - eksakt)
  23
  24 print("Trapezmetoden:")
  25 print(f"Integral: {trap}, Avvik = {avvik_trap}")
```

Typisk output:

Trapezmetode: 20,64455905 Avvik: 4,19193129

```
b) 1 simp = f(x[0]) + f(x[-1])
   2 for k in range(1, n):
   3     if k % 2 == 0:
   4         simp += 2*f(x[k])
   5     else:
   6         simp += 4*f(x[k])
   7 simp *= dx/3
   8
   9 avviksimp = abs(simp - eksakt)
  10
  11 print("Simpsons regel:")
  12 print(f"Integral: {simp}, Avvik = {avviksimp}")
```

Typisk output:

Simpsons regel: 17.35362645 Avvik: 0,90099869

- c) Simpsons regel benytter 3 punkt fra kurven i hvert estimeringssteg, mens trapesmetoden bruker kun 2 punkt. Dette gir Simpsons regel en mer presis tilnærming. Legg merke til at flere delintervaller øker presisjonen ytterligere.

Fasit 1.10.8:

- a) Programmet beregner det bestemte integralet

$$\int_0^3 x^2 dx$$

på to måter:

- **Trapesmetoden:** Summerer arealet av trapesene under kurven over hvert av de 6 delintervallene.
- **Simpsons regel:** Bruker parabeltilnærming (tre punkter) over hvert par av delintervallene, noe som gir høyere presisjon.

Programmet sammenligner begge metodene med en eksakt verdi beregnet numerisk med `scipy.integrate.quad`, og beregner avviket.

- b) Typiske resultater fra programmet:

Trapesmetoden: Integral: 9.375, Avvik: 0.125

Simpsons regel: Integral: 9.0, Avvik: 0.0

Simpsons regel gir en eksakt tilnærming i dette tilfellet fordi integranden x^2 er et andregardspolynom og Simpsons regel gir eksakt resultat for polynomer opp til tredje grad.

Fasit 2.4.1:

a) `1 print('Denne setningen skrives ut!')`

- b) P i print skal være liten bokstav. Mangler apostrof i enden til setningen.

Fasit 2.4.2:

a) `5+7`

b) `12`

c) `SyntaxError`. Legg merke til at det mangler en apostrof etter 7.

d) Print skal være `print`.

e) `1 5+7`

Fasit 2.4.3:

a) `1 > 3*5+4/5`
`2 15.8`

b) `1 > 2**3+3**2+1.25/4`
`2 17.25`

c) Resultatet til `18%4` blir 2 (restleddet i divisjonen 18 på 4).

Fasit 2.4.4:

- a) Skriver og kjører i console:

```
1 > 5*8/2
2 20.0
```

- b) Skriver og kjører i konsollvinduet:

```
1 > 7*10
2 70.0
```

- c) Skriver og kjører i konsollvinduet:

```
1 > 6*(8+10)/2
2 54.0
```

Fasit 2.6.1:

```
a) 1 m=7
    2 n=1
    3 m=m+3 # eller m+=3
    4 n=n*5 # eller n*=5
    5 p= m*n
    6 print('Produktet blir da',p)
```

Svaret blir da 30.

```
b) 1 a=5
    2 b='TALL'
    3 a=a*2
    4 b=b* 3
    5
    6 print(a)
    7 print(b)
```

Utskriften er da:

```
1 10
2 TALLTALLTALL
```

Fasit 2.6.2:

```
1 l = 5
2 b = 4
3 h = 3
4 volum=l*b*h
5 overflate = 2*l*b + 2*b*h + 2*l*h
6 print(f"Volume er {volum}.")
7 print(f"Overflaten er {overflate}.")
```

Ved å kjøre programmet blir svaret:

```
1 Volume er 60.
2 Overflaten er 94.
```

Fasit 2.6.3:

```
1 h= 3.5
2 g = 9.81
3 v0 = 20
4 t = 5
5
6 y= h - 0.5*g*t**2+v0*t
7
8 print('y er lik',y)
```

Fasit 2.7.1:

```
1 a = float(input("Skriv inn a (et reelt tall) her: "))
2 b= 3
3 c= 4
4 d = (1/3)*a*b+c**2
5
6 print('d er lik',d)
```

Fasit 2.7.2:

```

1 r = float(input("Skriv inn radiusen til en sylinder (et reelt positivt tall)
   her: "))
2 h= float(input("Skriv inn radiusen til en sylinder (et reelt positivt tall) her
   : "))
3 pi=3.14
4 V = pi*r**2*h
5
6 print('Volumet til sylindere er lik',V)

```

Fasit 2.8.1:

```

1 import numpy as np
2
3 x=0.5
4 y = np.sin(np.pi*x)
5
6 print('y er lik',y)

```

Fasit 2.8.2:

a) Programkode:

```

1 import numpy as np
2 svar1=np.cos(2)+np.tan(1.5)
3 print(svar1)

```

b) Programkode:

```

1 import numpy as np
2 svar2=np.exp(4)+np.log10(2)+np.log(0.1)
3 print(svar2)

```

Fasit 2.8.3:

```

1 import pylab
2
3 x=0.5
4 y = sin(pi*x)
5
6 print('y er lik',y)

```

Fasit 2.10.1:

Hvis man kjører programmet, blir man spurt **Skriv et tall**, og når man skriver hvis tallet er negativt, blir den erstattet med det tallet med positiv fortegn. Til slutt skriver programmet ut **Absluttverdien** til a er x, der x er et positivt tall uansett hva input a var. Resultatet blir blir **Absluttverdien** til -7 er 7.

Fasit 2.10.2:

a) Programkode:

```

1 tall = int(input("Skriv inn et tall: "))
2 if tall > 0:
3     print(f"Tallet {tall} er positivt.")
4 elif tall < 0:
5     print(f"Tallet {tall} er negativt.")
6 else:
7     print(f"Tallet {tall} er null.")

```

b) Programkode:

```

1 tall = int(input("Skriv inn et positivt tall: "))
2
3 if tall ==0 or tall <0:
4     print(f"{tall} er ikke positivt, skriv et positivt tall.")

```

```

5 elif tall%2 == 0:
6     print(f"Tallet {tall} er jevntall.")
7 else:
8     print(f"Tallet {tall} er oddetall.")

```

c) Programkode:

```

1 Tall1 = float(input("Skriv inn et tall: "))
2 Tall2 = float(input("Skriv inn et tall: "))
3
4 if Tall1*Tall2 < 0:
5     print(f"Tallene {Tall1} og {Tall2} har motsatt fortegn.")
6 elif Tall1*Tall2 > 0:
7     print(f"Tallene {Tall1} og {Tall2} har samme fortegn.")
8 else:
9     print(f"Et av tallene {Tall1} eller {Tall2} er null.")

```

Fasit 2.10.3:

Programkoden spør hva summen $49 + 35$, og når brukeren svarer, programkoden svarer om svaret ikke er rett (*Regn igjen!*) eller om det er rett (*Du klarte det!*).

Fasit 2.10.4:

```

1 innlest = input("Hva er 52+19?")
2 tall = int(innlest)
3
4 if tall != 71:
5     while tall != 71:
6         innlest = input("Feil. Regn igjen!")
7         tall = int(innlest)
8     else:
9         print("Du klarte det!")

```

Fasit 2.10.5:

```

1 tall = int(input("Skriv inn et oddetall: "))
2
3 if tall%2 != 0:
4     print("Tallet er ikke et oddetall!")
5 else:
6     sum = 0
7     i = 1
8     while(i <= tall):
9         sum = sum + i
10        i = i + 2
11
12 print(sum)

```

```

1 Skriv inn et oddetall: 5
2 9

```

Fasit 2.10.6:

```

1 x0= 1
2 b = 1024
3 x = 0
4 x1 = x0
5
6 while x1 != b:
7     x += 1
8     x1*= 4
9
10 print(f"Svaret er {x}.")

```

```
1 Svaret er 5
```

Fasit 2.10.7:

a) Resultatet blir:

```
1 A
2 B
3 B
4 B
5 A
6 B
7 B
8 B
```

b) Resultatet blir:

```
1 51 52 53 61 62 63
```

c) Resultatet blir:

```
1 51 52 53
2 61 62 63
```

Fasit 2.10.8:

a) Resultatet blir:

```
1 0 1 2 3 4 5 6 7 8 9
```

b) 1 2 3 4 5 6 7 8 9

c) 1 0 2 4 6 8

d) 1 10 9 8 7 6 5 4 3 2

Fasit 2.10.9:

```
1 svar = ""
2
3 while svar != "nei":
4     svar = input("Hei! Vil du fortsette programmet? ")
5
6 print("Du tastet nei, programmet avsluttes")
```

Fasit 2.10.10:

```
1 a1 = 3
2 k = 2
3 n=5
4 Sum=0
5 for i in range(n):
6     an=a1
7     an = a1 * k
8     Sum+=an
9
10 print(f'Summen til {n} ledd i en geometrisk rekke med a1 {a1} og kvotient {k} er {Sum}')
```

Fasit 2.10.11:

Programkoden lister opp for hvert tall fra og med til og med 9, tallet fra (fra og med) 1 til (til og med) 4.

Fasit 2.10.12:

```

1 a1 = 5
2 k = 3
3 n=7
4 Sum=0
5 for i in range(n):
6     an=a1
7     an = a1+k
8     Sum+=an
9
10 print(f'Summen til {n} ledd i en geometrisk rekke med a1 {a1} og kvotient {k}
      er {Sum} ')

```

Fasit 2.10.13:

- a) Programkoden lister opp for hvert tall fra og med til 1 og med 2, tallet fra (fra og med) 2 til (til og med) 6.

```

1 1 2
2 1 4
3 1 6
4 2 2
5 2 4
6 2 6

```

- b) `range(5)` sender inn 0, 1, 2, 3 og 4 og resultatet blir for `i in range(5)`:

```

1 02438

```

- c) `while i < 4` sender inn 1, 2, 3 og 4 og resultatet blir

```

1 1
2 2
3 3

```

while-delen erstattes med `for i in range(1,4)`:

- d)
- ```

1 1
2 2
3 3
4 4
5 i er ikke lenger mindre enn 5

```

- e)
- ```

1 1
2 1 2
3 1 2 3
4 1 2 3 4

```

- f)
- ```

1 o
2 o o
3 o o o
4 * * * *
5 * * *
6 * *
7 *

```

- g)
- ```

1 n=4;
2 for i in range(n):
3     for j in range(i):
4         print ('* ', end=" ")
5     print(' ')
6
7 for i in range(n,0,-1):
8     for j in range(i):
9         print('o ', end=" ")
10    print(' ')

```

Fasit 2.10.14:

a) Programkode:

```

1 n = 4
2 k = 4
3 for i in range(n+1,1,-1):
4     for j in range(1,k-i+3,1):
5         print(j,end=' ')
6     print()

```

b) Programkode:

```

1 n = 4
2 k = 4
3 for i in range(1,n+1,1):
4     for j in range(k-i+1,0,-1):
5         print(j,end=' ')
6     print()

```

c) Programkode:

```

1 n = 4
2 k = 4
3 for i in range(n+1,1,-1):
4     for j in range(k-i+2,0,-1):
5         print(j,end=' ')
6     print()

```

d) Programkode:

```

1 n = 4
2 k = 4
3 for i in range(n+2,0,-1):
4     for j in range(1,k-i+3,1):
5         print(j,end=' ')
6     print()

```

Fasit 2.11.1:

1 man får Skalarproduktet av a og b er 1.0 . Legg merke til $-1 \cdot 0 + 3 \cdot 1 + 0 \cdot 5 = 1$

Fasit 2.11.2:

a) 1 `import numpy as np`
 2
 3 `def A(r):`
 4 `return np.pi*r**2`
 5
 6 `print(A(5))`

b) 1 `import numpy as np`
 2
 3 `r= float(input('Skriv inn radiusen til en sirkel: '))`
 4
 5 `def A(r):`
 6 `return np.pi*r**2`
 7
 8 `print(A(r))`

Fasit 2.11.3:

- a) Programkode 1 (0.25 ligger i konvergensintervall til s_1 . Programkode 1 og 2 (1.75 ligger i konvergensintervall for begge) Programkode 2 (2.25 ligger i konvergensintervall til s_2)
- b) Ved å synke estimeringsnøyaktighet e i programkodene.

Fasit 2.11.4:

Programkode:

```
1 import numpy as np
2
3 def A(a,b,c):
4     s=1/2*(a+b+c)
5     A=np.sqrt(s*(s-a)*(s-b)*(s-c))
6     return A
7
8 a = 3
9 b = 4
10 c = 5
11
12 Areal=A(a,b,c)
13 print(f'Arealen til en trekant med sidelengdene {a}, {b} og {c} er {Areal}')
14 print(Areal)
```

Fasit 2.11.5:

Programkode:

```
1
2 def KE(m,v):
3     """
4     Denne funksjonen bestemmer kinetisk energi for en partikkel gitt:
5     Input:
6         masse (kg)
7         hastighet (m/s)
8
9     Output:
10        Partikkelens kinetisk energi (J)
11        """
12        KE = 0.5 * m * v** 2
13        return KE
14
15 m= 0.0001
16 v=8
17 E=KE(m, v)
18 print(f'Kinetisk energi for en partikkel med massen {m} og hastighet {v} m/s er {E} J')
```

Fasit 2.11.6:

```
1 5
2 15
```

Fasit 2.12.1:

- a)
- `plt.plot(x,f,'g--')` tegner grafen til funksjonen og tar inn x -verdiene definert i `x = np.linspace()`, de tilhørende funksjonsverdiene, og deretter `'b--'` skifter utseendet på grafen til en stiplet i stedet for en sammenhengende graf med grønnfarge. Flere argumenter kan angis i `plt.plot()` for å spesifisere detaljene til grafen.
 - Funksjonen `np.linspace(start, end, num = N+1)` deler intervallet $[start, end]$ inn i N like deler med $N + 1$ jevnt fordelte punkter.
 - `plt.yticks([-15,-5,5,15])` spesifiserer tallverdiene på y -aksen.
 - `plt.grid()` legger på et gitter bak grafen.

- `plt.xlabel()` og `plt.ylabel()` definerer navn på aksene.
- `plt.legend()` legger til en tekstboks for å beskrive hva grafen heter. `loc='best'`): plasserer tekstboksen der det passer best.
- `plt.show` tegner diagrammet i konsoll-vinduet.
- `plt.savefig()` Denne funksjonen gjør at vi kan lagre figuren direkte når vi kjører koden. Her må vi legge til riktig path inne i en streng, slik at bildet blir plassert i den mappen vi ønsker.

b)

Fasit 2.12.2:

```
a) 1 import matplotlib.pyplot as plt
    2 import numpy as np
    3
    4 xstart = -2
    5 xslutt = 2
    6 steglengde = 0.01
    7
    8 def f(x):
    9     return x**3 + 6*x - 3
   10
   11 x = xstart
   12 y = f(x)
   13
   14 while x <= xslutt:
   15     x += steglengde
   16     y = f(x)
   17     print(f'{x:.3f} {y:.3f}')
   18
   19 x = np.arange(xstart, xslutt, steglengde)
   20 y = f(x)
   21
   22 plt.plot(x,y)
```

b) Programkode:

```
1 xstart = float(input('Skriv inn nedre grense for x: '))
2 xslutt = float(input('Skriv inn øvre grense for x: '))
3 steglengde = float(input('Skriv inn steglengden: '))
4
5 def f(x):
6     return x**3 + 6*x - 3
7
8 x = xstart
9 y = f(x)
10
11 while x <= xslutt:
12     x += steglengde
13     y = f(x)
14     print(f'{x:.3f} {y:.3f}')
```

Fasit 2.12.3:

Programkode:

```
1
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 # Her skal vi definere en funksjon, og deretter plotte den
6
7 # Deler inn x-aksen i 100 deler (101 punkt)
```

```

8 x = np.linspace(-2*np.pi,2*np.pi, num=101)
9
10 def f(x):
11     f= np.sin(2*x-np.pi)
12     return f
13
14 plt.plot(x, f(x), color='green')
15
16 def g(x):
17     g= np.cos(2*x+np.pi)
18     return g
19
20 plt.plot(x, g(x), color='blue')
21
22 def h(x):
23     h= np.cos(2*x+np.pi/2)
24     return h
25
26 plt.plot(x, h(x), color='red')
27
28 plt.show()

```

f og h representerer samme funksjon.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 c= 60
5 a=120
6 T= 365
7 k =2
8 d= 182.5
9
10
11 # Deler inn x-aksen i 1000 deler (1001 punkt)
12 x = np.linspace(0,T, num=1001)
13
14 def f(x):
15     f= c+a*np.sin((2*np.pi*k/T)*x+ d)
16     return f
17
18 plt.plot(x, f(x), color='blue')
19 plt.grid()
20 plt.xlabel('x ')
21 plt.ylabel('y')
22
23 def g(x):
24     g= c+a*np.cos((2*np.pi*k/T)*x+d)
25     return g
26
27 plt.plot(x, g(x), color='red')
28
29 plt.legend(loc='best')
30 plt.legend(['f(x)', 'g(x)'])
31
32 plt.show()

```

Fasit 2.13.1:

- a) [1, 2, 3, 4, 5, 2, 4, 6, 8, 10]
- b) A=[1,2,3,4,5]
- c) Programkode:

```

1 liste = [1, 2, 3, 4, 5, 6]
2
3 produkt = 1
4
5 for t in liste:
6     produkt *= t
7
8 print(produkt)

```

Fasit 2.13.2:

```

1 def f(x):
2     return -.2*x**2+1
3
4 xstart = -1
5 xslutt = 2
6 xsteg = .25
7
8 xverdier = []
9 yverdier = []
10
11 x = xstart
12 y = f(x)
13
14 while x <= xslutt:
15     xverdier.append(x)
16     yverdier.append(y)
17     x = x + xsteg
18     y = f(x)
19
20 print(xverdier)
21 print(yverdier)

```

Fasit 2.13.3:

```

1 def f(x):
2     return -x**3+x+2
3
4 xstart = -2
5 xslutt = 3
6 xsteg = .25
7
8 xverdier = []
9 yverdier = []
10
11 x = xstart
12 y = f(x)
13
14 while x <= xslutt:
15     xverdier.append(x)
16     yverdier.append(y)
17     x = x + xsteg
18     y = f(x)
19
20 print(xverdier)
21 print(yverdier)

```

Bibliografi

- [1] Amir M. Hashemi and Arne Hole (2025). Matematikk for ingeniørutdanning, 1. utgave. Fagnbokforlaget 2025, ISBN 9788245052114.

Dette heftet vil fremme læreglede for å lære seg algoritmisk tenkning, modellering og problemløsning, og vil bruke programmering til å vise hvordan man kan løse oppgaver og problemer fra realfag og andre fag i ungdomstrinnet og videregående skole trinnet.

Problem: Skriv en programkode som regner ut volumet til en kule med radius r , og skisserer grafen til $V(r)$, for $0 \leq r \leq 5$.

Matematisk modell: $V(r) = \frac{4}{3}\pi r^3$

Programmering:

```
1 import matplotlib.pyplot as plt
2 from math import pi
3
4
5 # Leser opp radiusen fra skjermen
6
7 radius = float(input('Radius til kule:'))
8
9 # Funksjon som regner ut volumet til en kule med radius r
10
11 def volum(radius):
12
13     return (4/3) * pi * radius **3
14
15 V=volum(radius)
16
17 # Leser opp radiusen fra skjermen
18
19 print(f"Volumet til en kule med radius {radius} er: ", V)
20 r_verdier = np.arange(0,5.2,.2)
21 print(r_verdier)
22
23
24 #for r in r_verdier:
25 #    # formatering    print(f"{x:.2f}\t{f(x):.2f}")
26
27 r=r_verdier
28 V=volum(r)
29
30
31 #print(f'{r:.2f}\t,{V:.2f}')
32
33 plt.plot(r,V,'.-', linewidth=3, label='V(r)')
34 plt.grid()
35 plt.xlabel('r')
36 plt.ylabel('V')
37 plt.legend(loc='best')
38 plt.show()
```

Problem: vi skal se på filen 'tekopp-temperatur.csv'. Hver linje i filen er en temperatur-måling i en te-kopp i løpet av en time (3600 sekunder) som er plassert i et rom med temperatur 22 Grad celsius. Her bruker vi Newtons avkjølingslov for å studere temperaturendringen i vannkoppen.

Matematisk modell:

$$\frac{dT}{dt} = k(T - T_o)$$

der $T(t)$ ((målt i Grad celsius) er vanntemperatur i vannkoppen ved tiden t (målt i sekunder).

Programmering:

```
1 import matplotlib.pyplot as plt
2 import pandas as pd
3
4 df = pd.read_csv('http://home.hib.no/ansatte/ahas/MP/tekopp-temp.csv', sep=
    ';', decimal='.', comment='#')
5
6 tid = df['Tid'].tolist()
7 temperatur = df['Temperatur'].tolist()
8 tstart = 0
9 Tstart = temperatur[0]
10 tslutt = 3700
11 deltat = 0.5
12 To = 22 # Rom temperatur
13 k = .0015 # proporsjonalitets koeff.
14 tverdier = []
15 Tverdier = []
16 t = tstart
17 T = Tstart
18 while t <= tslutt:
19     tverdier.append(t)
20     Tverdier.append(T)
21     t = t + deltat
22     deltaT = -deltat * k * (T - To)
23     T = T + deltaT
24
25 plt.plot(tverdier, Tverdier, linewidth=1, color='blue', label='T(t)')
26 plt.plot(tid, temperatur, 'x', color='red')
27 plt.grid()
28 plt.xlabel('t (sek)')
29 plt.ylabel('T (Grad celsius)')
30 plt.legend(loc='best')
31 plt.show()
```

